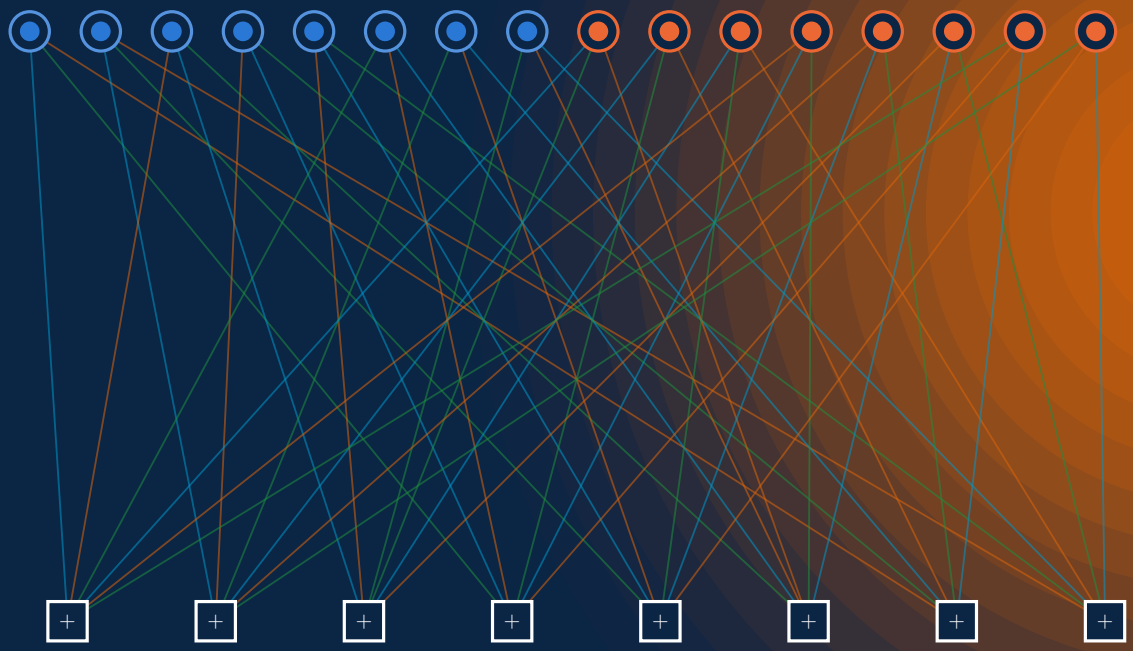


PART TWO • A COMPLETE VISUAL AND MATHEMATICAL GUIDE

LDPC CODES

From Zero to Expert

Parity-check matrices • Tanner graphs • PEG and quasi-cyclic design • Bit flipping, sum-product, min-sum and layered decoding • Density evolution and EXIT charts • 5G NR base graphs • Block diagrams • Tested Python and MATLAB code

Mahmoud Abdelaziz

Military Technical College, Cairo

Version 2 • 2026

LDPC Codes: From Zero to Expert — Part Two of the series that starts with *Polar Codes: From Zero to Expert*

Version 2, October 2026 (adds block diagrams for every system).

All numbers in the worked examples and all simulation results were produced with the Python library printed in Appendix B (NumPy, fixed random seeds). Simulation curves are Monte-Carlo estimates, not theoretical bounds; re-running the code reproduces them. The MATLAB code of Appendix C was run in GNU Octave and checked against the Python library. The 5G NR base-graph tables (TS 38.212, Tables 5.3.2-2 and 5.3.2-3) were read from the open-source Sionna library (Apache-2.0 licence). 3GPP details are summarised for teaching; always consult the current release of TS 38.212 for implementation work.

Typeset with XeLaTeX. Figures drawn with Ti*k*Z and Matplotlib.

How to Use This Book

This is **Part Two** of a series. Part One, *Polar Codes: From Zero to Expert*, built the foundations of digital communication and channel coding and then studied polar codes. This book studies the other great family of modern codes: **low-density parity-check (LDPC) codes**. LDPC codes protect the data channels of 5G NR, Wi-Fi, DVB-S2 satellite television, 10-gigabit Ethernet and many storage systems. Together, the two books cover the two code families of the 5G standard.

You can read this book alone. When a basic idea (BPSK, AWGN, LLR, capacity, linear codes) is needed, Chapter 2 summarises it and gives a pointer such as [Part One, Ch. 2] for the full treatment. Every topic is then taught in the same seven steps:



What you need from Part One

Topic	Used in this book for	Part One
Bits, BPSK, AWGN, E_b/N_0	every simulation	Ch. 1
Linear block codes, $\mathbf{G}$, Hamming distance	parity-check matrices (Ch. 3)	Ch. 2
BEC, BSC, BI-AWGN channels and capacity	thresholds, Shannon limits (Ch. 2, 10)	Ch. 2
LLRs and the box-plus (f) function	the check-node update (Ch. 8, 9)	Ch. 6
BER, BLER/FER, coding gain	performance analysis (Ch. 13)	Ch. 12
Polar codes in 5G NR	comparison with LDPC (Ch. 11, 13)	Ch. 10
Logarithms, probability, GF(2)	throughout	App. A

The colour code of the boxes

Definition 0.1 • Definitions are cyan

A precise statement of a new term. Learn these by heart.

Intuition

Gold boxes explain the idea in everyday words, before any mathematics.

Worked Example 0.1 • Worked examples are green

Real numbers, every step shown. All of them were checked with the Python library. Re-do them with a pencil.

Advanced • Purple is for advanced material

You may skip purple boxes on a first reading.

Important note

Orange boxes hold facts that are easy to forget but important in practice.

! Common mistake

Red boxes show errors that students (and engineers!) really make.

Professional insight

Grey boxes connect the theory to real systems and current research.

Conventions used in the whole book

Code	n code bits, k message bits, m parity checks; rate $R = k/n$
Parity-check matrix	$\mathbf{H}$ is $m \times n$; $\mathbf{c}$ is a codeword $\iff \mathbf{H}\mathbf{c}^T = \mathbf{0}$ (over GF(2))
Generator matrix	$\mathbf{G}$ is $k \times n$; row-vector encoding $\mathbf{c} = \mathbf{u}\,\mathbf{G}$; $\mathbf{H}\mathbf{G}^T = \mathbf{0}$
Tanner graph	variable nodes (circles, ◯) = columns of $\mathbf{H}$; check nodes (squares, □) = rows of $\mathbf{H}$
Degrees	d_v = column weight, d_c = row weight; $\lambda(x)$, $\rho(x)$ in the *edge* perspective
Messages	$L_{v\to c}$ variable-to-check, $L_{c\to v}$ check-to-variable, L_v^{app} a-posteriori LLR
BPSK	bit $0 \mapsto +1$, bit $1 \mapsto -1$; symbol energy $E_s = 1$
LLR	$L = \ln \dfrac{P(\text{bit}=0 \mid y)}{P(\text{bit}=1 \mid y)}$; positive favours 0; BPSK/AWGN: $L = 2y/\sigma^2$
Noise	real AWGN, variance $\sigma^2 = N_0/2$, $E_s/N_0 = R\,E_b/N_0$
Indices	bits, checks and rows are numbered from 0 (as in Python)
Colours of bits	1 information 0 parity 0 punctured or filler

The software

All Python code is in one library, `ldpc.py` (NumPy only), printed in full in Appendix B and explained in Chapter 12. Short scripts produce the worked examples (`examples.py`), the simulations (`run_sims.py`), and the final project (`project.py`). The library was tested before any number was

written into the book: the sum-product decoder gives the exact a-posteriori LLRs on a cycle-free graph, every encoder output satisfies $\mathbf{H}\mathbf{c}^T = \mathbf{0}$, and the 5G NR code construction was checked against the base-graph tables. Appendix C gives the same library in MATLAB, tested against the Python results.

What is new in version 2

Seventeen new **block diagrams**, marked "**Block diagram.**" in their captions, show each system as engineers draw it: the coded link, the syndrome check, the way from $\mathbf{H}$ to an encoder, PEG, quasi-cyclic lifting, the 5G NR encoder, the peeling and bit-flipping decoders, the sum-product decoder, the min-sum check-node unit, the layered decoder, the EXIT model, base-graph and lifting-size selection, the circular buffer with HARQ, the software structure, the Monte-Carlo loop and the LDPC-versus-polar comparison. The theory, the worked examples, the code and all simulation results are unchanged.

Course map

Level	Chapters	Goal
Beginner	1–2	why LDPC codes; the tools from Part One
Intermediate	3–6	parity-check matrices, Tanner graphs, construction, encoding
Advanced	7–10	all decoders, density evolution, EXIT charts, code design
Professional	11–13	5G NR, the software, performance analysis, LDPC versus polar
Expert practice	14–16	solved problems, examination and interview questions, final project

A one-semester postgraduate course can use Chapters 1–10 and 13 in lectures and Chapters 12, 14 and 16 in the laboratory.

Contents

How to Use This Book . . . iii

I Beginner Level: Foundations

1 Welcome to LDPC Codes . . . 2

1.1 An LDPC code in one picture . . . 2
1.2 Why “low density” is the key . . . 3
1.3 A short history . . . 3
1.4 Where are LDPC codes used? . . . 5
1.5 The building blocks of an LDPC link . . . 5
1.6 LDPC codes and polar codes side by side . . . 6
1.7 Road map of this book . . . 6
1.8 Chapter summary . . . 6

2 Essentials from Part One . . . 8

2.1 System model . . . 8
2.2 BPSK over the AWGN channel . . . 9
2.3 Log-likelihood ratios . . . 9
2.4 Channels and capacity . . . 10
2.5 Linear block codes in two descriptions . . . 11
2.6 Measuring performance . . . 11
2.7 Chapter summary . . . 12

II Intermediate Level: Understanding LDPC Codes

3 Parity-Check Matrices . . . 15

3.1 The atom: a single parity check . . . 15
3.2 The parity-check matrix . . . 15
3.3 The syndrome . . . 16

3.4 Rank, redundant checks and the true rate . . . 17
3.5 Gaussian elimination over GF(2). . . . 17
3.6 From **H** to a generator matrix . . . 18
3.7 One code, many matrices . . . 18
3.8 Chapter summary . . . 19

4 Tanner Graphs and Degree Distributions . . . 21

4.1 From matrix to graph . . . 21
4.2 Degrees and the edge-count identity. . . . 22
4.3 Cycles and girth. . . . 22
4.4 Degree distributions . . . 23
4.5 Why irregular codes are better . . . 24
4.6 In Python . . . 25
4.7 Chapter summary . . . 25

5 Constructing LDPC Codes . . . 27

5.1 What makes a good parity-check matrix? . . . 27
5.2 Gallager's construction. . . . 28
5.3 MacKay's random constructions . . . 28
5.4 Progressive edge growth (PEG) . . . 28
5.5 Quasi-cyclic codes . . . 29
5.6 Protographs . . . 31
5.7 Comparison of constructions . . . 32
5.8 In Python . . . 32
5.9 Chapter summary . . . 32

6 Encoding LDPC Codes . . . 35

6.1 The encoding problem . . . 35
6.2 Triangular and dual-diagonal parity parts . . . 36
6.3 The quasi-cyclic dual-diagonal trick . . . 36
6.4 The 5G NR encoder . . . 37
6.5 General matrices: the Richardson–Urbanke method . . . 38
6.6 In Python . . . 38
6.7 Chapter summary . . . 38

III Advanced Level: Decoding and Analysis

7 Erasure Decoding and Bit Flipping . . . 42

7.1 Iterative message passing in one paragraph . . . 42

7.2 Decoding on the binary erasure channel . . . 42
7.3 Bit flipping on the binary symmetric channel . . . 44
7.4 The price of hard decisions . . . 45
7.5 In Python . . . 46
7.6 Chapter summary . . . 46

8 Belief Propagation: the Sum-Product Algorithm . . . 48

8.1 From hard bits to soft beliefs . . . 48
8.2 The goal: bit-wise MAP decoding . . . 48
8.3 The two local rules . . . 49
8.4 The sum-product algorithm . . . 50
8.5 A complete numerical example . . . 51
8.6 BP is exact on trees . . . 51
8.7 Practical issues . . . 52
8.8 In Python . . . 52
8.9 Chapter summary . . . 53

9 Min-Sum and Layered Decoding . . . 56

9.1 The min-sum approximation . . . 56
9.2 Correcting min-sum . . . 57
9.3 Layered decoding . . . 58
9.4 Complexity and implementation notes . . . 60
9.5 In Python . . . 60
9.6 Chapter summary . . . 61

10 Density Evolution and EXIT Charts . . . 63

10.1 The idea: follow the messages, not the codewords . . . 63
10.2 Density evolution on the BEC . . . 64
10.3 Designing irregular codes . . . 65
10.4 Density evolution on the AWGN channel . . . 66
10.5 EXIT charts . . . 67
10.6 What density evolution does not see: error floors . . . 67
10.7 In Python . . . 69
10.8 Chapter summary . . . 69

IV Professional Level: 5G and Practical Implementation

11 LDPC Codes in 5G NR . . . 72

11.1 Where NR uses LDPC codes . . . 72

11.2 The processing chain . . . 72
11.3 The two base graphs . . . 73
11.4 Lifting . . . 74
11.5 Rate matching and redundancy versions . . . 75
11.6 Worked parameter examples . . . 75
11.7 Decoding NR LDPC codes in practice . . . 76
11.8 In Python . . . 76
11.9 Chapter summary . . . 78

12 Python Implementation . . . 80

12.1 Organisation of the software . . . 81
12.2 The data structure: an edge list . . . 82
12.3 The library, block by block . . . 82
12.4 Demonstration with outputs . . . 85
12.5 The simulation script . . . 86
12.6 How the code was tested . . . 88
12.7 Chapter summary . . . 88

13 Performance Analysis and Comparison with Polar Codes . . . 91

13.1 Simulation set-up . . . 91
13.2 Comparing decoders . . . 92
13.3 Iterations . . . 93
13.4 Block length . . . 94
13.5 Code rate . . . 94
13.6 LDPC versus polar codes . . . 95
13.7 Complexity, latency and throughput . . . 97
13.8 Practical design trade-offs . . . 97
13.9 Current research directions . . . 97
13.10 Chapter summary . . . 98

V Expert Practice

14 Fully Solved Problems . . . 101

14.1 Basic level . . . 101
14.2 Intermediate level . . . 102
14.3 Advanced level . . . 104
14.4 Summary of methods . . . 105

15 Interview and Examination Questions . . . 106

15.1 Beginner level . . . 106

15.2 Intermediate level . . . 107
15.3 Advanced level . . . 108
15.4 Engineering interview level . . . 109
15.5 How to prepare . . . 110

16 Final Project: A 5G NR LDPC Link with HARQ . . . 111

16.1 Project specification . . . 111
16.2 Design decisions . . . 112
16.3 The complete program . . . 112
16.4 Results . . . 115
16.5 Extensions for students and researchers . . . 116
16.6 Project checklist . . . 117
16.7 Final words . . . 118

Appendix A — Mathematical Prerequisites . . . 119

Appendix B — Python Source Code . . . 121

Appendix C — MATLAB Implementation . . . 139

Appendix D — Glossary . . . 160

Appendix E — Bibliography . . . 162

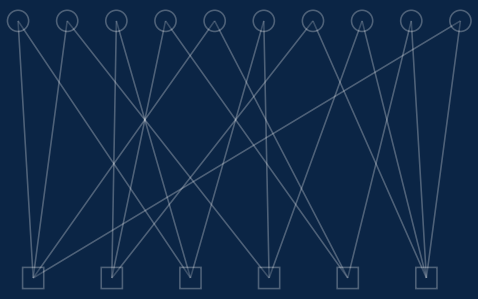
PART I

Beginner Level: Foundations

Why LDPC codes matter, where they are used, and the tools from Part One that we need: channels, LLRs, capacity and linear codes.

CHAPTER

1



Welcome to LDPC Codes

IN THIS CHAPTER YOU WILL LEARN

- what a low-density parity-check (LDPC) code is, in one picture;
- why "low density" is the secret of the decoder;
- the history: from Gallager (1962) to 5G NR;
- where LDPC codes are used today;
- how an LDPC link is built, block by block;
- how LDPC codes differ from the polar codes of Part One;
- the road map of this book.

1.1 An LDPC code in one picture

Intuition

Imagine six friends who each hold one secret bit. Instead of one big rule, we give them four small rules, and each rule involves only three friends: "the bits of friends 0, 1 and 3 must contain an even number of ones", and so on. If one bit is changed by noise, some small rules are broken, and the broken rules point to the bit that is wrong. An LDPC code is exactly this: *many small parity rules, each touching only a few bits.* "Low density" means that each rule is small and each bit appears in only a few rules.

The rules are written as the rows of a **parity-check matrix** $\mathbf{H}$. A 1 in row j and column i means "bit i takes part in check j". Here is the small code that we will use in many chapters:

$$\mathbf{H} = \begin{bmatrix} 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 & 0 & 1 \end{bmatrix} \qquad \begin{aligned} c_0 \oplus c_1 \oplus c_3 &= 0 \\ c_1 \oplus c_2 \oplus c_4 &= 0 \\ c_0 \oplus c_4 \oplus c_5 &= 0 \\ c_2 \oplus c_3 \oplus c_5 &= 0 \end{aligned} \tag{1.1}$$

A word $\mathbf{c} = (c_0, \dots, c_5)$ is a **codeword** when all four checks are satisfied. For example, $\mathbf{c} = (0, 0, 1, 0, 1, 1)$ satisfies every row. The same rules can be drawn as a graph (Fig. 1.1): circles are

the bits, squares are the checks, and a line joins a bit to every check it takes part in. This picture is called a **Tanner graph**, and the decoder works by sending messages along its lines.

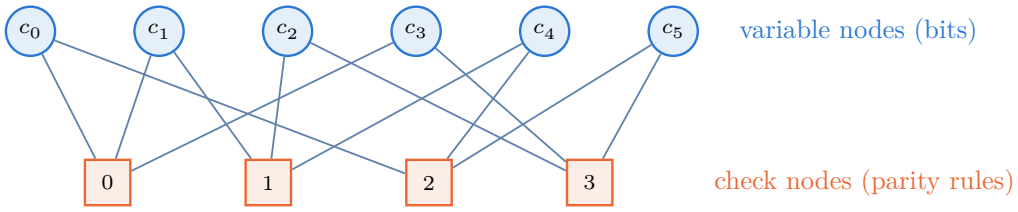


Figure 1.1. Tanner graph of the matrix (1.1): each bit is in two checks, each check contains three bits.

Definition 1.1 • LDPC code

A **low-density parity-check code** is a linear block code defined by a parity-check matrix $\mathbf{H}$ in which only a small fraction of the entries are 1. If every column has d_v ones and every row has d_c ones, the code is (d_v, d_c)**-regular**; otherwise it is **irregular**. The number of ones grows only linearly with the code length n, not with n^2.

Worked Example 1.1 • How sparse is "low density"?

In Chapter 5 we build a code of length $n = 1008$ with $m = 504$ checks and $d_v = 3$. Its matrix $\mathbf{H}$ has $504 \times 1008 = 508\,032$ entries but only $3 \times 1008 = 3024$ ones: a density of 0.6%. A generator matrix $\mathbf{G}$ of the same code (computed in Chapter 6) has 127 514 ones out of 504×1008 entries, a density of 25.1%. The *code* is the same; only the parity-check description is sparse — and the decoder only needs that one.

1.2 Why "low density" is the key

A decoder must decide which codeword was sent. Trying all 2^k codewords is impossible for $k = 504$ ($2^{504} \approx 10^{152}$). Gallager's idea was to decode with *local* work: each check node looks only at its few bits, each bit node listens only to its few checks, and they exchange opinions again and again. The work per iteration is proportional to the number of ones in $\mathbf{H}$, so a sparse $\mathbf{H}$ gives a decoder whose cost grows only linearly with n. Sparseness has a second gift: the graph has few short loops, so the opinions that a bit receives are almost independent, and the simple local rules work almost as well as the optimal (but impossible) decoder.

Two promises of sparseness

1. **Low complexity:** decoding cost $\propto$ (number of ones in $\mathbf{H}$) $\times$ (number of iterations).
2. **Good performance:** few short cycles $\Rightarrow$ nearly independent messages $\Rightarrow$ iterative decoding comes very close to capacity for long codes.

1.3 A short history

- **1962–1963.** Robert Gallager defined LDPC codes and two iterative decoders in his doctoral work at MIT (paper 1962, book 1963). Computers of the time could not simulate long codes, and the idea was almost forgotten for thirty years.

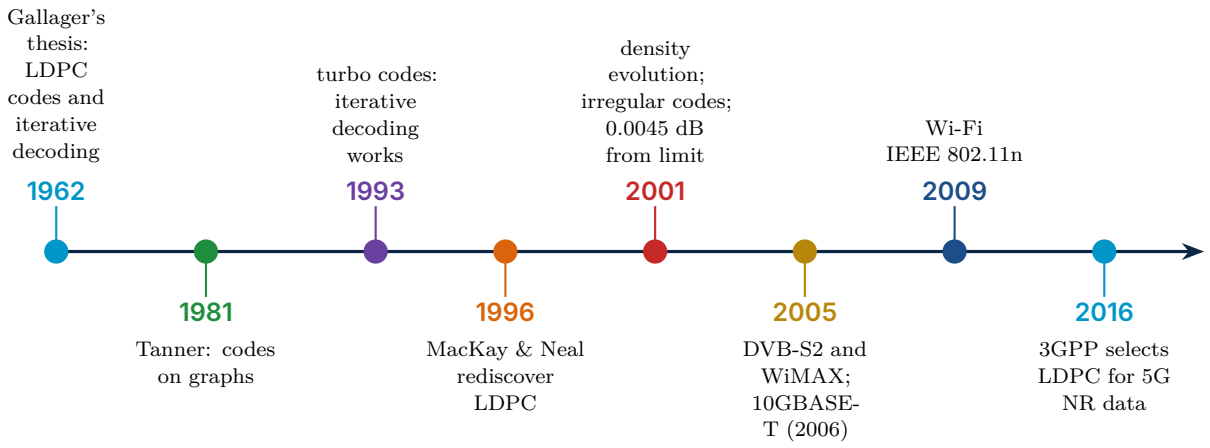


Figure 1.2. Milestones of LDPC codes.

- **1981.** Michael Tanner described codes by bipartite graphs — the Tanner graphs of this book.
- **1993.** Turbo codes showed that iterative decoding can work within about 1 dB of the Shannon limit.
- **1996–1999.** David MacKay and Radford Neal rediscovered LDPC codes and showed that they perform as well as turbo codes.
- **2001.** Richardson and Urbanke developed **density evolution**, a method to compute the limit of iterative decoding; with Shokrollahi they designed irregular codes. Chung, Forney, Richardson and Urbanke reported an irregular code of very long length within 0.0045 dB of the Shannon limit (in simulation, 0.04 dB).
- **2005–2009.** LDPC codes entered standards: DVB-S2 satellite television (2005), IEEE 802.16e WiMAX, IEEE 802.3an 10GBASE-T Ethernet (2006) and IEEE 802.11n Wi-Fi (2009).
- **2016–2018.** 3GPP chose LDPC codes for the data channels of 5G NR (agreed in 2016, specified in Release 15, TS 38.212) and polar codes for the control channels (Part One).

1.4 Where are LDPC codes used?

5G NR	data channels (PDSCH, PUSCH); two quasi-cyclic base graphs, lifting sizes $2 \le Z \le 384$, code blocks up to 8448 bits (Chapter 11)
Wi-Fi (IEEE 802.11n/ac/ax)	quasi-cyclic codes of length 648, 1296 and 1944 bits, rates 1/2, 2/3, 3/4, 5/6
DVB-S2 / S2X, ATSC 3.0	long codes (16 200 and 64 800 bits) for broadcasting, followed by an outer BCH code
10GBASE-T Ethernet	a (2048, 1723) LDPC code built from Reed–Solomon codes
Space (CCSDS)	an (8160, 7136) near-earth code and the AR4JA family for deep space
Storage	flash memory (SSD) controllers use LDPC codes with soft reading

Professional insight

Why did so many standards choose LDPC codes? Three reasons. (1) *Performance*: long LDPC codes are within a fraction of a decibel of capacity. (2) *Parallelism*: all check nodes can work at the same time, so hardware decoders reach tens of gigabits per second. (3) *Structure*: quasi-cyclic codes (Chapter 5) are described by a small base matrix and a lifting size, which makes one decoder serve many lengths and rates.

1.5 The building blocks of an LDPC link

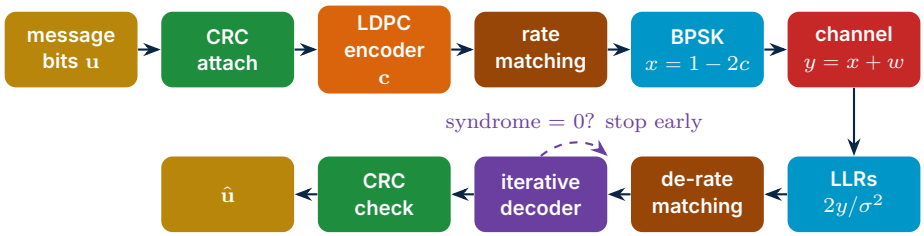


Figure 1.3. An LDPC link as used in 5G NR data channels (simplified; Chapter 16 builds it in Python).

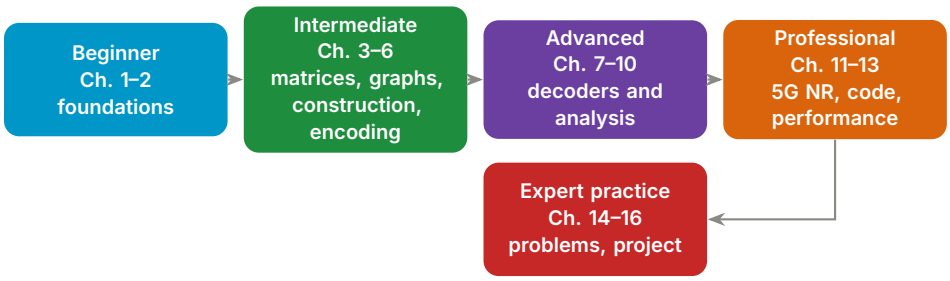
Each block has its own chapter: the encoder (Chapter 6), the decoders (Chapters 7–9), rate matching and the CRC (Chapter 11) and the complete link (Chapter 16). One block is new compared with Part One: the decoder can test the syndrome $\mathbf{H}\hat{\mathbf{c}}^T$ after each iteration and *stop early* when it is zero. This saves energy and time at high signal-to-noise ratio.

1.6 LDPC codes and polar codes side by side

	Polar codes (Part One)	LDPC codes (this book)
Defined by	generator $\mathbf{G}_N = \mathbf{F}^{\otimes n}$ and a frozen set	a sparse parity-check matrix $\mathbf{H}$
Length	$N = 2^n$ (then rate matching)	any n; QC lifting gives many lengths
Main decoder	successive cancellation (list), sequential	belief propagation, parallel and iterative
Strong at	short blocks, with CRC-aided list decoding	medium and long blocks, high throughput
Theory	channel polarization	density evolution, EXIT charts
In 5G NR	control channels	data channels

Both families are linear codes, both use LLRs, and both use the same *box-plus* operation $2\tanh^{-1}\big(\tanh\frac{a}{2}\tanh\frac{b}{2}\big)$ — in polar codes it is the f function [Part One, Ch. 6]; in LDPC codes it is the check-node update (Chapter 8). Chapter 13 compares the two families with real simulations at the same length and rate.

1.7 Road map of this book

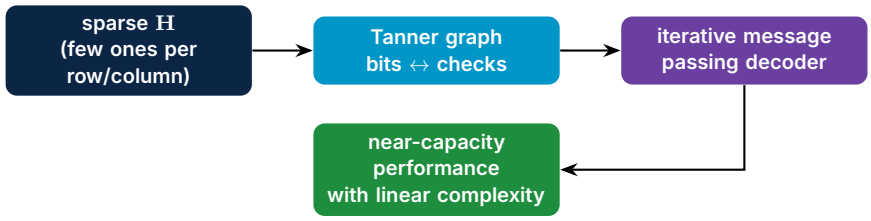


1.8 Chapter summary

Key concepts

- An LDPC code is a linear code with a *sparse* parity-check matrix $\mathbf{H}$.
- $\mathbf{c}$ is a codeword if and only if $\mathbf{H}\mathbf{c}^T = \mathbf{0}$: every parity rule is satisfied.
- The Tanner graph draws $\mathbf{H}$: variable nodes (bits), check nodes (rules) and edges (the ones of $\mathbf{H}$).
- Sparseness gives linear decoding complexity and nearly independent messages.
- LDPC codes are used in 5G NR data channels, Wi-Fi, DVB-S2, Ethernet, space links and storage.

Visual recap • Chapter 1



Common student mistakes

! Common mistake

- Thinking that "low density" means a low code rate. It describes the matrix $\mathbf{H}$, not the rate. A rate-8/9 code can be an LDPC code.
- Thinking that the generator matrix of an LDPC code is also sparse. Usually it is dense (25% ones in our example).
- Believing that LDPC codes were invented for 5G. They are older than the Internet: 1962.

Practice problems

Problem 1.1. Is $\mathbf{c} = (1, 1, 0, 0, 1, 0)$ a codeword of (1.1)?

Solution Check 0: $1 \oplus 1 \oplus 0 = 0$; check 1: $1 \oplus 0 \oplus 1 = 0$; check 2: $1 \oplus 1 \oplus 0 = 0$; check 3: $0 \oplus 0 \oplus 0 = 0$. All checks are satisfied: yes.

Problem 1.2. A $(3, 6)$-regular code has $n = 2000$. How many checks and how many ones does $\mathbf{H}$ have? What is the density?

Solution Ones $= 3n = 6000$; checks $m = 6000/6 = 1000$; density $6000/(1000 \cdot 2000) = 0.3\%$.

Problem 1.3. Why does the cost of one decoding iteration grow linearly with n?

Solution Every edge carries two messages per iteration, and the number of edges is $d_v n$ (a constant times n).

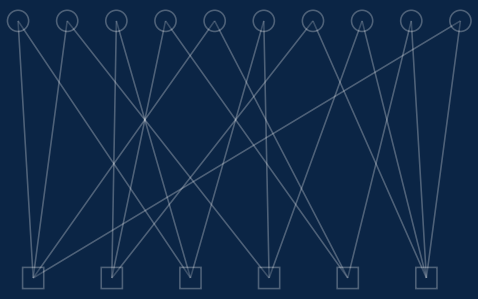
Self-assessment quiz

1. Who invented LDPC codes, and when?
2. What do the circles and the squares of a Tanner graph represent?
3. Which 5G channels use LDPC codes and which use polar codes?
4. Name two benefits of a sparse parity-check matrix.

Answers: 1) R. G. Gallager, 1962. 2) Bits (variables) and parity checks. 3) LDPC: data channels; polar: control channels. 4) Linear decoding complexity; few short cycles, so iterative decoding works well.

CHAPTER

2



Essentials from Part One

IN THIS CHAPTER YOU WILL LEARN

- the system model and notation of the whole book;
- BPSK over AWGN and the meaning of E_b/N_0;
- log-likelihood ratios (LLRs) and their two key rules;
- the BEC, BSC and BI-AWGN channels and their capacities;
- the Shannon limit for each code rate;
- linear block codes: $\mathbf{G}$, $\mathbf{H}$, syndrome and distance;
- BER, FER, waterfall and error floor.

How to read this chapter

This chapter is a compact summary of Part One. If you have read Part One, read only the boxes and the tables. If you have not, every section gives the pointer to the full treatment.

2.1 System model

A message $\mathbf{u} = (u_0, \dots, u_{k-1})$ of k bits is encoded into a codeword $\mathbf{c} = (c_0, \dots, c_{n-1})$ of n bits. The code rate is $R = k/n$. Each bit is mapped to a BPSK symbol $x_i = 1-2c_i$ (bit $0 \mapsto +1$, bit $1 \mapsto -1$) and sent over a channel. The receiver computes one reliability number per bit, the LLR L_i, and the decoder uses these numbers to find $\hat{\mathbf{c}}$ and $\hat{\mathbf{u}}$ [Part One, Ch. 1].

Fig. 2.1 draws this model. The decoder never sees y_i directly, only the LLRs; this is the interface between the channel (this chapter) and the decoders (Chapters 7–9).

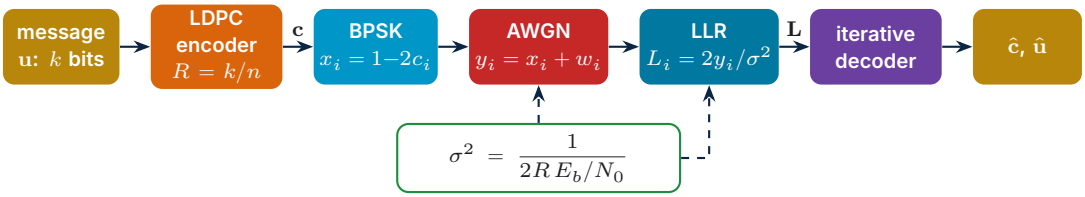


Figure 2.1. Block diagram. The coded link used in the whole book. The same σ^2 sets the noise and scales the LLRs.

2.2 BPSK over the AWGN channel

The received sample is $y_i = x_i + w_i$ with Gaussian noise $w_i \sim \mathcal{N}(0, \sigma^2)$. With symbol energy $E_s = 1$ and energy per information bit $E_b = E_s/R$:

$$\frac{E_s}{N_0} = R\,\frac{E_b}{N_0}, \qquad \sigma^2 = \frac{N_0}{2} = \frac{1}{2R\,(E_b/N_0)} \quad (E_b/N_0 \text{ linear}).$$

Worked Example 2.1 • Noise level for a rate-1/2 code

$R = \frac{1}{2}$, $E_b/N_0 = 2$ dB $= 10^{0.2} = 1.585$. Then $\sigma^2 = 1/(2 \cdot 0.5 \cdot 1.585) = 0.631$ and $\sigma = 0.794$.

! Common mistake

Always compare codes of different rates in E_b/N_0, not in E_s/N_0. A low-rate code spends more symbols per information bit; E_b/N_0 charges it for that energy.

2.3 Log-likelihood ratios

Definition 2.1 • LLR

For a bit c and an observation y,

$$L(c) = \ln \frac{P(c = 0 \mid y)}{P(c = 1 \mid y)}.$$

The **sign** is the hard decision ($L > 0 \Rightarrow \hat{c} = 0$), and the **magnitude** $|L|$ is the confidence.

2.3.1 Channel LLRs

For BPSK over AWGN with equally likely bits:

$$L = \ln \frac{e^{-(y-1)^2/2\sigma^2}}{e^{-(y+1)^2/2\sigma^2}} = \frac{(y+1)^2 - (y-1)^2}{2\sigma^2} = \frac{2y}{\sigma^2}.$$

Channel	Output	Channel LLR
BI-AWGN	$y \in \mathbb{R}$	$2y/\sigma^2$
BSC(p)	$y \in \{0, 1\}$	$(1 - 2y) \ln \frac{1-p}{p}$
BEC(ϵ)	$y \in \{0, 1, ?\}$	$+\infty$ (0), $-\infty$ (1), 0 (erasure ?)

Worked Example 2.2 • From a sample to a probability

$y = 0.8$, $\sigma^2 = 0.5$: $L = 2(0.8)/0.5 = 3.2$. Since $P(c = 0 \mid y) = 1/(1 + e^{-L})$, the probability that a 0 was sent is $1/(1 + e^{-3.2}) = 0.961$.

2.3.2 The two rules that make LDPC decoding work

Rule 1 (same bit, independent observations): add the LLRs.

$$L(c \mid y_1, y_2) = L(c \mid y_1) + L(c \mid y_2).$$

Rule 2 (XOR of independent bits): the box-plus ("tanh") rule.

$$L(c_1 \oplus c_2) = 2\tanh^{-1}\Big(\tanh\frac{L_1}{2}\tanh\frac{L_2}{2}\Big) \approx \operatorname{sign}(L_1)\operatorname{sign}(L_2)\min(|L_1|,|L_2|).$$

Rule 1 is used by the *variable nodes* (a bit hears several checks). Rule 2 is used by the *check nodes* (a check knows that the XOR of its bits is 0). In Part One, Rule 2 was the f function of the SC decoder and Rule 1 was the g function with a known partial sum [Part One, Ch. 6]. Chapter 8 derives both rules again and builds the LDPC decoder from them.

2.4 Channels and capacity

Definition 2.2 • Channel capacity

The capacity C is the largest rate (bits per channel use) at which reliable communication is possible. For rates $R < C$ codes exist with error probability as small as we like; for $R > C$ they do not [Part One, Ch. 2].

Channel	Capacity	Example
BEC(ϵ)	$C = 1 - \epsilon$	$\epsilon = 0.5 \Rightarrow C = 0.5$
BSC(p)	$C = 1 - H_2(p)$, $H_2(p) = -p\log_2 p - (1-p)\log_2(1-p)$	$p = 0.11 \Rightarrow C = 0.500$
BI-AWGN(σ)	$C = 1 - \mathbb{E}\left[\log_2(1 + e^{-L})\right]$, $L = \frac{2}{\sigma^2}(1 + \sigma w)$	$\sigma = 0.9787 \Rightarrow C = 0.5$

The BI-AWGN capacity has no closed form; we computed it by Gauss–Hermite numerical integration. Setting $C = R$ and converting σ to E_b/N_0 gives the **Shannon limit** for BPSK at each rate:

Rate R	1/5	1/4	1/3	1/2	2/3	3/4	5/6	8/9
BPSK (BI-AWGN) limit, dB	−0.96	−0.79	−0.50	0.19	1.06	1.63	2.36	3.03
Gaussian-input limit, dB	−1.29	−1.21	−1.08	−0.82	−0.55	−0.41	−0.28	−0.19

Professional insight

These limits are the "finish line" of every simulation in this book. For example, a rate-1/2 code with BPSK can never work reliably below $E_b/N_0 = 0.19$ dB, however long it is and however good its decoder is. In Chapter 10 we show that a $(3, 6)$-regular LDPC code with belief propagation reaches about 1.1 dB for infinite length, and in Chapter 13 a real 5G code of 4096 information bits reaches a frame error rate of 10^{-3} at about 1.3 dB.

2.5 Linear block codes in two descriptions

A binary linear (n, k) code is a k-dimensional subspace of $\{0,1\}^n$ [Part One, Ch. 2]. It has two descriptions:

	Generator matrix G $(k \times n)$	**Parity-check matrix H** $(m \times n)$
Defines the code as	all combinations $\mathbf{c} = \mathbf{uG}$	all solutions of $\mathbf{Hc}^T = \mathbf{0}$
Good for	encoding	checking and decoding
Size	k rows	$m \geq n - k$ rows (rank $n - k$)
LDPC codes	usually dense	sparse (by design)

The two matrices satisfy $\mathbf{HG}^T = \mathbf{0}$. The **syndrome** of a received hard word $\mathbf{r}$ is $\mathbf{s} = \mathbf{Hr}^T$: it is zero if and only if $\mathbf{r}$ is a codeword. The **minimum distance** $d_{\min}$ is the smallest Hamming weight of a non-zero codeword; a code corrects all patterns of up to $\lfloor (d_{\min} - 1)/2 \rfloor$ errors with a maximum-likelihood decoder.

Worked Example 2.3 • The (7,4) Hamming code from the parity-check side

In one common form, the (7,4) Hamming code has $\mathbf{H} = \begin{bmatrix} 1 & 1 & 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 & 0 & 1 \end{bmatrix}$ in one common form. A single error in bit i gives a syndrome equal to column i of $\mathbf{H}$; all columns are different and non-zero, so every single error is located. LDPC codes use the same principle — unsatisfied checks point to wrong bits — but with many more, much smaller checks.

2.6 Measuring performance

- **Bit error rate (BER)**: fraction of wrong information bits.
- **Frame error rate (FER)**, also called block error rate (BLER): fraction of codewords with at least one wrong information bit. 5G link adaptation usually targets FER $= 10^{-1}$ for the first transmission; low-latency services need 10^{-5} or lower.
- **Coding gain**: the reduction of E_b/N_0 that the code gives for the same error rate, compared with uncoded BPSK [Part One, Ch. 12].

Definition 2.3 • Waterfall and error floor

The **waterfall** is the region where the error rate falls very fast as E_b/N_0 grows; its position is predicted by density evolution (Chapter 10). The **error floor** is the region at high E_b/N_0 where the curve becomes flatter. For LDPC codes it is caused by small harmful subgraphs (stopping sets on the BEC, trapping sets on the AWGN channel) and by low-weight

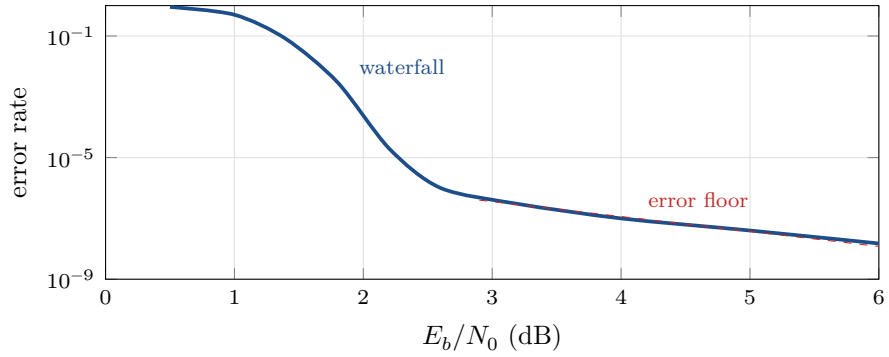


Figure 2.2. Schematic shape of an LDPC error-rate curve (not a simulation): a steep *waterfall* region and, at high E_b/N_0, a flatter *error floor*.

codewords (Chapters 7 and 10).

2.7 Chapter summary

Important formulas

Noise variance	$\sigma^2 = 1/(2R\,E_b/N_0)$
BPSK LLR	$L = 2y/\sigma^2, \quad P(c=0 \mid y) = 1/(1+e^{-L})$
Same bit	$L = L_1 + L_2$
XOR of bits	$L = 2\tanh^{-1}(\tanh\frac{L_1}{2}\tanh\frac{L_2}{2}) \approx \operatorname{sign}(L_1)\operatorname{sign}(L_2)\min(\lvert L_1\rvert, \lvert L_2\rvert)$
Capacities	BEC $1-\epsilon$; BSC $1-H_2(p)$; BI-AWGN numerical
Codes	$\mathbf{c} = \mathbf{uG}$, $\mathbf{Hc}^T = \mathbf{0}$, $\mathbf{HG}^T = \mathbf{0}$

Common student mistakes

! Common mistake

- Using $\sigma^2 = 1/(2\,E_b/N_0)$ and forgetting the rate R.
- Using E_b/N_0 in dB inside formulas. Convert first: $10^{x/10}$.
- Feeding the decoder y instead of $2y/\sigma^2$. Min-sum decoders do not care about a positive scale, but the sum-product decoder does.
- Mixing the sign convention: in this book $L > 0$ means bit 0.

Practice problems

Problem 2.1. Find σ for $R = 1/3$ at $E_b/N_0 = 1$ dB.

Solution $10^{0.1} = 1.259$; $\sigma^2 = 1/(2 \cdot \frac{1}{3} \cdot 1.259) = 1.191$; $\sigma = 1.091$.

Problem 2.2. Two independent observations of the same bit give $L_1 = 2.0$ and $L_2 = -0.5$. What is the combined LLR and the decision?

Solution $L = 1.5 > 0$, decide 0.

Problem 2.3. Bits c_1, c_2 have LLRs 2.0 and -3.0. Find the exact and approximate LLR of $c_1 \oplus c_2$.

Solution $\tanh(1.0)\tanh(-1.5) = 0.7616 \cdot (-0.9051) = -0.6893$; $2\tanh^{-1}(-0.6893) = -1.693$. Min-sum: -2.0.

Problem 2.4. What is the capacity of a BSC with $p = 0.05$?

Solution $H_2(0.05) = 0.05\log_2 20 + 0.95\log_2(1/0.95) = 0.2161 + 0.0703 = 0.2864$; $C = 0.7136$.

Self-assessment quiz

1. What does $|L| = 0$ mean?
2. Why is the min-sum approximation always larger in magnitude than the exact box-plus?
3. What is the Shannon limit for BPSK at $R = 1/2$?
4. Which matrix of an LDPC code is sparse?

Answers: 1) No information (both bit values equally likely, like an erasure). 2) The exact rule multiplies two numbers below 1 in magnitude, so the result is less reliable than the weakest input. 3) 0.19 dB. 4) The parity-check matrix $\mathbf{H}$.

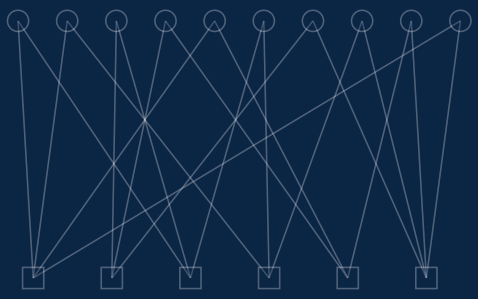
PART II

Intermediate Level: Understanding LDPC Codes

Parity-check matrices, Tanner graphs, degree distributions, code construction (PEG and quasi-cyclic) and efficient encoding.

CHAPTER

3



Parity-Check Matrices

IN THIS CHAPTER YOU WILL LEARN

- single parity checks, the atoms of LDPC codes;
- how $\mathbf{H}$ defines a code as a null space;
- the syndrome and what it tells the decoder;
- rank, redundant rows and the true code rate;
- Gaussian elimination over GF(2), step by step;
- how to obtain a generator matrix from $\mathbf{H}$;
- why one code can have many parity-check matrices.

3.1 The atom: a single parity check

Intuition

A *single parity check* is the simplest error detector: add one bit so that the total number of ones is even. If one bit flips, the count becomes odd and we know that something is wrong — but not where. An LDPC code uses *many* single parity checks that overlap. Each bit takes part in several checks, so when one bit flips, the set of failed checks acts like a fingerprint that points at the bit.

Definition 3.1 • Single parity-check (SPC) constraint

A set of bits $\{c_{i_1}, \dots, c_{i_d}\}$ satisfies a parity check if $c_{i_1} \oplus c_{i_2} \oplus \cdots \oplus c_{i_d} = 0$. The number d is the **degree** of the check.

3.2 The parity-check matrix

Collect m parity checks on n bits into an $m \times n$ binary matrix $\mathbf{H}$: row j has a 1 in every column that takes part in check j. Then all checks are satisfied together exactly when

$$\mathbf{H}\,\mathbf{c}^T = \mathbf{0} \pmod 2.$$

Definition 3.2 • Code defined by H

The code $\mathcal{C} = \{\mathbf{c} \in \{0,1\}^n : \mathbf{H}\mathbf{c}^T = \mathbf{0}\}$ is the **null space** of $\mathbf{H}$ over GF(2). It is linear: the XOR of two codewords is a codeword, because $\mathbf{H}(\mathbf{c}_1 \oplus \mathbf{c}_2)^T = \mathbf{H}\mathbf{c}_1^T \oplus \mathbf{H}\mathbf{c}_2^T = \mathbf{0}$.

Worked Example 3.1 • All codewords of the toy code

For the matrix (1.1) of Chapter 1, we tested all $2^6 = 64$ words. Exactly 8 satisfy $\mathbf{H}\mathbf{c}^T = \mathbf{0}$:

```
000000  001011  010111  011100  100101  101110  110010  111001
```

So $k = \log_2 8 = 3$, the rate is $R = 3/6 = 1/2$, and the smallest non-zero weight is 3: $d_{\min} = 3$.

3.3 The syndrome

Definition 3.3 • Syndrome

For a received hard-decision word $\mathbf{r}$, the syndrome is $\mathbf{s} = \mathbf{H}\mathbf{r}^T$. Entry $s_j = 1$ means that check j is **unsatisfied**. If $\mathbf{r} = \mathbf{c} \oplus \mathbf{e}$ (codeword plus error pattern), then $\mathbf{s} = \mathbf{H}\mathbf{e}^T$: the syndrome depends only on the errors, not on the codeword.

Worked Example 3.2 • Locating one error

The codeword $\mathbf{c} = (0,0,1,0,1,1)$ is sent and $\mathbf{r} = (0,0,1,1,1,1)$ is received. Then

$$\mathbf{s} = \mathbf{H}\mathbf{r}^T = (1,0,0,1)^T :$$

checks 0 and 3 fail. Which bit is in checks 0 and 3 and in no other check? Column 3 of $\mathbf{H}$ is $(1,0,0,1)^T$ — exactly the syndrome. Flipping bit 3 gives back the codeword. This is the idea of the bit-flipping decoder of Chapter 7.

Fig. 3.1 shows the syndrome as a block of the receiver: a cheap test that every iterative decoder runs after each iteration.

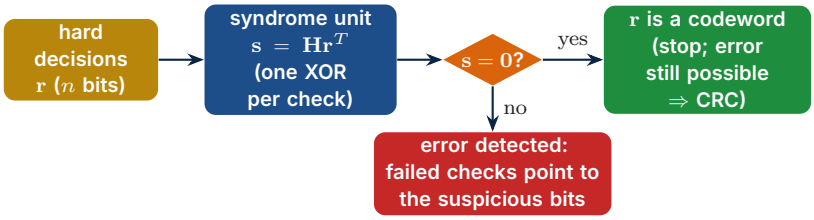


Figure 3.1. Block diagram. The syndrome check: error detection with $\mathbf{H}$ only.

! Common mistake

A zero syndrome means "$\mathbf{r}$ is *a* codeword", not "$\mathbf{r}$ is the codeword that was sent". If the noise turns one codeword into another one, the syndrome is zero and the error is undetected. This is why 5G adds a CRC on top of the LDPC code (Chapter 11).

3.4 Rank, redundant checks and the true rate

The matrix $\mathbf{H}$ has m rows, but some rows may be sums of others. Only the *rank* counts:

$$k = n - \operatorname{rank}_2(\mathbf{H}), \qquad R = \frac{k}{n} \;\geq\; R_{\text{design}} = 1 - \frac{m}{n}.$$

Worked Example 3.3 • A redundant row

In (1.1) every column has exactly two ones, so the sum of all four rows is the zero row: one row is redundant. Hence $\operatorname{rank}\mathbf{H} = 3$, $k = 6 - 3 = 3$ and $R = 1/2$, while the design rate is $1 - 4/6 = 1/3$.

Worked Example 3.4 • Gallager's construction always loses rank

A $(3, 6)$-regular Gallager matrix with $n = 1008$ (Chapter 5) has $m = 504$ rows, but it is built from 3 bands and the rows of each band add up to the all-ones row. So at least 2 rows are redundant. The library finds rank 502, so $k = 506$ and $R = 0.502$, slightly above the design rate 0.5.

Redundant rows are not useless

For *decoding*, a redundant check is extra information: it is another parity rule that the decoder can use. Some codes (for example the Euclidean-geometry LDPC codes) have many redundant rows on purpose, and this helps iterative decoding.

3.5 Gaussian elimination over GF(2)

To find the rank, the message positions and a generator matrix, we bring $\mathbf{H}$ to **reduced row echelon form** (RREF). Over GF(2) the only row operations needed are swapping two rows and adding (XOR) one row to another.

Worked Example 3.5 • RREF of the toy matrix, step by step

Write the rows as bit strings: $r_0 = 110100$, $r_1 = 011010$, $r_2 = 100011$, $r_3 = 001101$.

1. **Column 0.** Pivot r_0. Clear column 0 in r_2: $r_2 \leftarrow r_2 \oplus r_0 = 010111$.
2. **Column 1.** Pivot $r_1 = 011010$. Clear column 1 elsewhere: $r_0 \leftarrow 110100 \oplus 011010 = 101110$, $r_2 \leftarrow 010111 \oplus 011010 = 001101$.
3. **Column 2.** Pivot $r_2 = 001101$. Clear column 2 elsewhere: $r_0 \leftarrow 101110 \oplus 001101 = 100011$, $r_1 \leftarrow 011010 \oplus 001101 = 010111$, $r_3 \leftarrow 001101 \oplus 001101 = 000000$.

$$\text{RREF} = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 0 & 1 \end{bmatrix}, \qquad \text{pivot columns } \{0, 1, 2\}, \quad \text{free columns } \{3, 4, 5\}.$$

The zero row disappeared: rank $= 3$. This agrees with `gf2_rref` in the library.

3.6 From H to a generator matrix

The free columns can take any values: they carry the message. Each RREF row then gives one pivot bit as a sum of free bits:

$$c_0 = c_4 \oplus c_5, \qquad c_1 = c_3 \oplus c_4 \oplus c_5, \qquad c_2 = c_3 \oplus c_5.$$

Setting the message (c_3, c_4, c_5) to 100, 010 and 001 gives the three rows of $\mathbf{G}$:

$$\mathbf{G} = \begin{bmatrix} 0 & 1 & 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 0 \\ 1 & 1 & 1 & 0 & 0 & 1 \end{bmatrix}, \qquad \mathbf{H}\mathbf{G}^T = \mathbf{0}.$$

Fig. 3.2 summarises the way from $\mathbf{H}$ to an encoder.

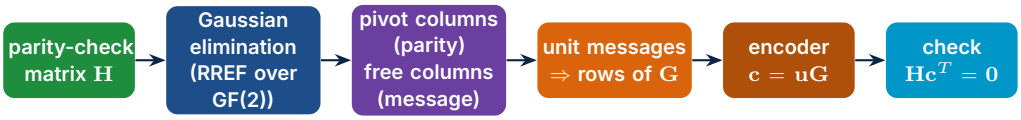


Figure 3.2. Block diagram. From the parity-check matrix to a generator-matrix encoder (`generator_from_H`).

Worked Example 3.6 • Encoding with G

Message $\mathbf{u} = (1, 0, 1)$ placed in bits $3, 4, 5$: $\mathbf{c} = \mathbf{u}\mathbf{G} = \text{row}_0 \oplus \text{row}_2 = 011100 \oplus 111001 = 100101$. Check: $\mathbf{H}\mathbf{c}^T = (1 \oplus 0 \oplus 1,\ 0 \oplus 0 \oplus 0,\ 1 \oplus 0 \oplus 1,\ 0 \oplus 1 \oplus 1) = (0, 0, 0, 0)$. ✓

Definition 3.4 • Systematic form

If the columns are reordered so that the message bits come first, $\mathbf{G} = [\mathbf{I}_k \mid \mathbf{P}]$ and $\mathbf{H}' = [\mathbf{P}^T \mid \mathbf{I}_{n-k}]$. The function `generator_from_H` does not reorder columns; instead it returns the list `info_pos` of the columns that hold the message (here $\{3, 4, 5\}$).

In Python

```python
import numpy as np
from ldpc import gf2_rref, gf2_rank, generator_from_H, encode_G
H = np.array([[1,1,0,1,0,0],[0,1,1,0,1,0],[1,0,0,0,1,1],[0,0,1,1,0,1]])
print(gf2_rank(H))                  # 3
G, info = generator_from_H(H)       # info = [3 4 5]
print(encode_G(G, [1, 0, 1]))       # [[1 0 0 1 0 1]]
```

The functions are in Appendix B (lines 11–55); Chapter 12 explains them line by line.

3.7 One code, many matrices

Adding one row of $\mathbf{H}$ to another does not change the null space, so a code has many parity-check matrices. For algebraic decoders this does not matter. For LDPC decoders it matters a lot: the decoder works on the Tanner graph of the *given* $\mathbf{H}$, and a denser $\mathbf{H}$ means more short cycles and more work.

Worked Example 3.7 • Same code, worse graph

Replace row $r_0 = 110100$ of (1.1) by $r_0 \oplus r_1 = 101110$. The code is unchanged, but the new row has weight 4 instead of 3, and rows 0 and 1 now share bits 2 and 4: a cycle of length 4 appears (Chapter 4). A good LDPC code needs a good *matrix*, not only a good code.

Professional insight

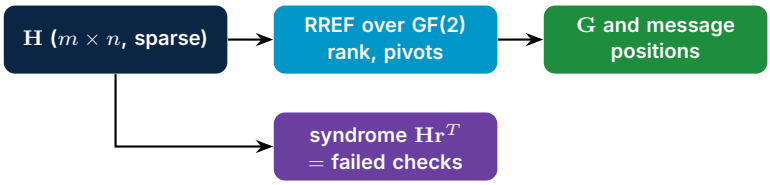
This is why LDPC design is done directly on $\mathbf{H}$: we choose a sparse matrix with good graph properties, and the generator matrix is only a by-product for encoding — and even that is avoided in practice by structured encoders (Chapter 6).

3.8 Chapter summary

Key concepts

- $\mathbf{H}\mathbf{c}^T = \mathbf{0}$ defines the code; each row is a single parity check.
- The syndrome $\mathbf{H}\mathbf{r}^T = \mathbf{H}\mathbf{e}^T$ shows which checks fail.
- $k = n - \operatorname{rank}_2 \mathbf{H}$; the true rate may exceed the design rate $1 - m/n$.
- RREF over GF(2) gives the rank, the message positions and a generator matrix.
- Different matrices of the same code give different iterative decoders.

Visual recap • Chapter 3



Common student mistakes

! Common mistake

- Computing the rate as $1 - m/n$ without checking the rank.
- Using ordinary (real) Gaussian elimination: over GF(2), $1 + 1 = 0$ and there is no division.
- Assuming that the first k bits of $\mathbf{c}$ are always the message: they are the free (information) positions, which depend on the pivots.
- Forgetting that $\mathbf{H}\mathbf{G}^T = \mathbf{0}$ must be checked: it is the quickest test of an encoder.

Practice problems

Problem 3.1. Write the $\mathbf{H}$ of the length-4 repetition code and of the length-4 single parity-check code.

Solution Repetition: $\mathbf{H} = \begin{bmatrix} 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 \end{bmatrix}$ (codewords 0000, 1111). SPC: $\mathbf{H} = [1\ 1\ 1\ 1]$ (all even-weight words). Each is the generator matrix of the other: they are dual codes.

Problem 3.2. With the toy $\mathbf{H}$, find the syndrome of $\mathbf{r} = (1, 0, 1, 1, 1, 1)$ and the most likely single error.

Solution $\mathbf{s} = (1 \oplus 0 \oplus 1,\ 0 \oplus 1 \oplus 1,\ 1 \oplus 1 \oplus 1,\ 1 \oplus 1 \oplus 1) = (0, 0, 1, 1)$. Column 5 of $\mathbf{H}$ is $(0, 0, 1, 1)^T$: flip bit 5, $\hat{\mathbf{c}} = (1, 0, 1, 1, 1, 0) = 101110$, a codeword.

Problem 3.3. Encode $\mathbf{u} = (1, 1, 0)$ with the $\mathbf{G}$ of this chapter.

Solution $\text{row}_0 \oplus \text{row}_1 = 011100 \oplus 110010 = 101110$. Check: it is in the list of codewords.

Problem 3.4. An $\mathbf{H}$ of size 600×1200 has rank 598. What are k, R and the design rate?

Solution $k = 1200 - 598 = 602$, $R = 602/1200 = 0.5017$, design rate 0.5.

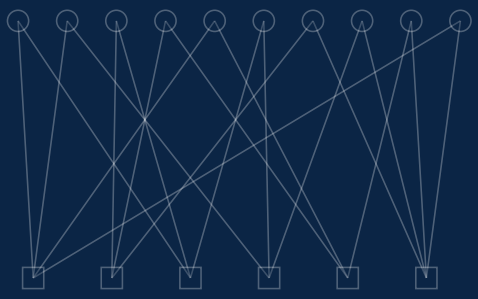
Self-assessment quiz

1. Why is the code defined by $\mathbf{H}$ always linear?
2. What does the syndrome depend on?
3. Why does every (d_v, d_c)-regular Gallager matrix have redundant rows?
4. Can two different matrices define the same code? Does it matter for LDPC decoding?

Answers: 1) A null space is closed under XOR. 2) Only on the error pattern. 3) In each band the rows sum to the all-ones row, so the sum of two bands is zero. 4) Yes; yes, because the decoder runs on the graph of the matrix.

CHAPTER

4



Tanner Graphs and Degree Distributions

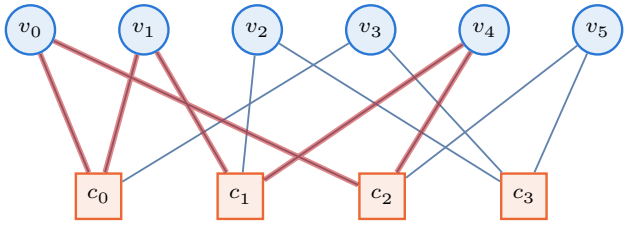
IN THIS CHAPTER YOU WILL LEARN

- how to draw any $\mathbf{H}$ as a bipartite Tanner graph;
- node degrees and the edge-count identity;
- cycles and girth, and why short cycles hurt;
- how to see a 4-cycle directly in $\mathbf{H}$;
- degree distributions in node and edge perspective;
- the design-rate formula $1 - \int \rho / \int \lambda$;
- why irregular codes beat regular codes.

4.1 From matrix to graph

Definition 4.1 • Tanner graph

The Tanner graph of an $m \times n$ matrix $\mathbf{H}$ has n **variable nodes** $v_0, \dots, v_{n-1}$ (one per column, one per code bit), m **check nodes** $c_0, \dots, c_{m-1}$ (one per row) and an **edge** between c_j and v_i whenever $H_{ji} = 1$. Edges join only nodes of different types: the graph is **bipartite**.



$$\mathbf{H} = \begin{bmatrix} 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 & 0 & 1 \end{bmatrix}$$

Figure 4.1. Tanner graph of the toy code. Red: a cycle of length 6, $v_0 - c_0 - v_1 - c_1 - v_4 - c_2 - v_0$.

Matrix ↔ graph dictionary

Matrix **H**	Tanner graph
column i	variable node v_i (bit c_i)
row j	check node c_j (parity rule j)
entry $H_{ji} = 1$	edge v_i–c_j
column weight	variable-node degree d_v
row weight	check-node degree d_c
number of ones	number of edges E

4.2 Degrees and the edge-count identity

Every edge has one variable end and one check end. Counting the edges from both sides gives

$$E = \sum_{i=0}^{n-1} d_v(i) = \sum_{j=0}^{m-1} d_c(j), \qquad \text{regular code: } n\,d_v = m\,d_c \;\Rightarrow\; R_{\text{design}} = 1 - \frac{m}{n} = 1 - \frac{d_v}{d_c}.$$

Worked Example 4.1 • Degrees of the toy code

Every column of **H** has weight 2 and every row has weight 3: the code is $(2,3)$-regular, $E = 6 \cdot 2 = 4 \cdot 3 = 12$ and $R_{\text{design}} = 1 - 2/3 = 1/3$ (the true rate is $1/2$ because of the redundant row, Chapter 3).

4.3 Cycles and girth

Definition 4.2 • Cycle and girth

A **cycle** is a closed path that starts and ends at the same node without using an edge twice. In a bipartite graph every cycle has even length ≥ 4. The **girth** g is the length of the shortest cycle.

Intuition

During decoding, each node sends its opinion to its neighbours, and opinions travel around the graph. On a cycle, a node's own opinion comes back to it after a few steps, disguised as "new" evidence. The node then becomes too confident — it is listening to an echo of itself. Short cycles create echoes early; long cycles create them late, when the decoder has usually already finished.

4.3.1 How to see a 4-cycle in H

A 4-cycle $v_a - c_j - v_b - c_l - v_a$ exists exactly when rows j and l both have ones in columns a and b: the four ones form the corners of a rectangle in $\mathbf{H}$.

$$\begin{bmatrix} \cdots & 1 & \cdots & 1 & \cdots \\ & \vdots & & \vdots & \\ \cdots & 1 & \cdots & 1 & \cdots \end{bmatrix} \iff \text{two rows share two columns.}$$

So a matrix is free of 4-cycles if and only if *any two rows overlap in at most one position.* The number of 4-cycles is $\sum_{j<l} \binom{o_{jl}}{2}$, where o_{jl} is the overlap of rows j and l (the entries of $\mathbf{H}\mathbf{H}^T$ over the integers). This is how `count_4cycles` works.

Worked Example 4.2 • Girth of the toy code

Rows of (1.1) overlap in at most one position (for example rows 0 and 1 share only column 1), so there is no 4-cycle. The red path in Fig. 4.1 is a 6-cycle. Therefore $g = 6$, which the library function `girth` confirms.

4.3.2 The computation tree

To see why girth matters, unroll the graph from one variable node: its checks, their other variables, and so on (Fig. 4.2). After ℓ decoding iterations, the decision for the root depends on the nodes in the first 2ℓ levels of this tree. As long as these levels contain no repeated node (no cycle closes inside them), all incoming messages are statistically independent, and the decoder computes exact probabilities for that sub-tree. A larger girth keeps the tree a true tree for more iterations.

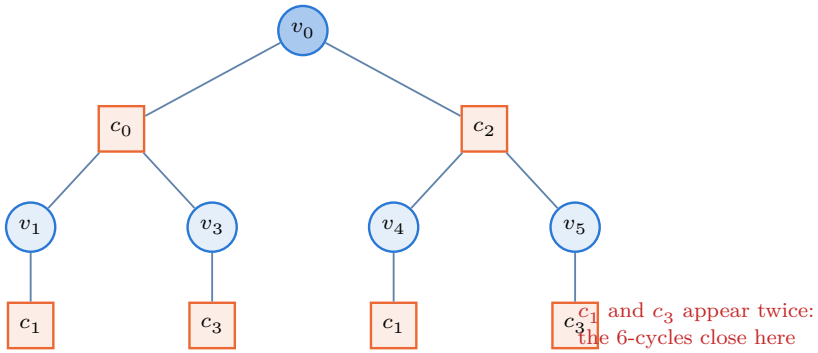


Figure 4.2. Computation tree of v_0 in the toy code. Up to depth 2 all nodes are different; at depth 3 checks repeat because the girth is 6.

! Common mistake

Do not over-rate girth. A girth-8 code is not automatically better than a girth-6 code: the number of short cycles, how they connect, and the degree distribution all matter. But 4-cycles should almost always be avoided.

4.4 Degree distributions

Irregular codes have nodes of different degrees. We describe the mix with polynomials.

Definition 4.3 • Node and edge perspectives

- **Node perspective**: L_i = fraction of variable nodes of degree i, R_i = fraction of check nodes of degree i; $L(x) = \sum_i L_i x^i$, $R(x) = \sum_i R_i x^i$.
- **Edge perspective**: λ_i = fraction of *edges* that end at a variable node of degree i, ρ_i = fraction of edges that end at a check node of degree i; $\lambda(x) = \sum_i \lambda_i x^{i-1}$, $\rho(x) = \sum_i \rho_i x^{i-1}$.

A node of degree i owns i edges, so

$$\lambda_i = \frac{i\,L_i}{\sum_j jL_j}, \qquad L_i = \frac{\lambda_i/i}{\sum_j \lambda_j/j}, \qquad R_{\text{design}} = 1 - \frac{\int_0^1 \rho(x)\,dx}{\int_0^1 \lambda(x)\,dx} = 1 - \frac{\sum_i \rho_i/i}{\sum_i \lambda_i/i}.$$

Advanced • Why the rate formula holds

$\sum_i \lambda_i/i$ is the number of variable nodes per edge (n/E), and $\sum_i \rho_i/i$ is the number of check nodes per edge (m/E). Their ratio is m/n, so $1 - m/n$ is the design rate.

Worked Example 4.3 • An irregular rate-1/2 ensemble

Take $n = 10$ variable nodes: 5 of degree 2, 4 of degree 3 and 1 of degree 8; and $m = 5$ check nodes of degree 6. Edges: $5 \cdot 2 + 4 \cdot 3 + 1 \cdot 8 = 30 = 5 \cdot 6$. ✓

- Node perspective: $L(x) = 0.5x^2 + 0.4x^3 + 0.1x^8$, $R(x) = x^6$.
- Edge perspective: $\lambda_2 = 10/30$, $\lambda_3 = 12/30$, $\lambda_8 = 8/30$, so $\lambda(x) = \frac{1}{3}x + 0.4x^2 + \frac{4}{15}x^7$, $\rho(x) = x^5$.
- Rate: $\int \lambda = \frac{1}{3}\cdot\frac{1}{2} + 0.4\cdot\frac{1}{3} + \frac{4}{15}\cdot\frac{1}{8} = 0.3333$, $\int \rho = \frac{1}{6}$, $R = 1 - 0.1667/0.3333 = 0.5$.
- A value used later: $\lambda(0.5) = 0.26875$.

4.5 Why irregular codes are better

Intuition

In a regular code all bits are equally protected, and they all "wake up" at the same speed. In an irregular code, high-degree variable nodes receive many messages and become reliable after one or two iterations. They then send strong messages to their checks, which helps the low-degree nodes. The correction spreads like a *wave* from the strong nodes to the weak ones. Degree-2 nodes are cheap (they cost few edges) but must not be too many, or the wave stops (the stability condition of Chapter 10).

Worked Example 4.4 • The 5G NR base graphs are strongly irregular

Counting the base matrices of TS 38.212 (Chapter 11):

- **BG1** (46×68, 316 non-zero entries): variable degrees from 1 to 30. The two first columns (punctured, never transmitted) have degrees 30 and 28; the 42 extension parity columns have degree 1. Check degrees range from 3 to 19.
- **BG2** (42×52, 197 non-zero entries): variable degrees from 1 to 23, check degrees from

3 to 10.

The very high-degree punctured columns are the "strong nodes" of the intuition above: they are never sent, but they collect information from many checks and spread it.

4.6 In Python

The library stores a code as the edge list of its Tanner graph (lines 60–103): arrays `cn` and `vn` give the check and the variable of each edge, sorted by check. Degrees are counts:

```
from ldpc import LDPCCode, girth, count_4cycles
code = LDPCCode.from_H(H)
print(code.E, code.dv, code.dc)          # 12 [2 2 2 2 2 2] [3 3 3 3]
print(girth(code), count_4cycles(code))  # 6 0
```

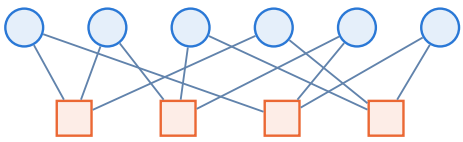
`girth` runs a breadth-first search from every variable node and records the first time two search paths meet; `count_4cycles` uses the row overlaps $\mathbf{H}\mathbf{H}^T$.

4.7 Chapter summary

Key concepts

- Tanner graph: variable nodes = columns, check nodes = rows, edges = ones.
- $E = \sum d_v = \sum d_c$; a regular code has $R_{\text{design}} = 1 - d_v/d_c$.
- A 4-cycle is a rectangle of ones in $\mathbf{H}$; girth = shortest cycle.
- Short cycles make messages dependent; on a tree the decoder is exact.
- $\lambda(x), \rho(x)$ count edges; $L(x), R(x)$ count nodes; $R = 1 - \int \rho / \int \lambda$.
- Irregular degree distributions give better thresholds (Chapter 10).

Visual recap • Chapter 4



columns → circles, rows → squares, ones → edges;
girth = shortest cycle (no rectangles of ones $\Rightarrow g \geq 6$);
$\lambda(x), \rho(x)$: edge fractions; $R = 1 - \int \rho / \int \lambda$.

Common student mistakes

! Common mistake

- Mixing the node and the edge perspective: λ_i is *not* the fraction of nodes of degree i.
- Writing $\lambda(x) = \sum \lambda_i x^i$: the exponent is $i - 1$ (the number of *other* edges of the node).
- Thinking a cycle of length 3 or 5 can exist: Tanner graphs are bipartite, all cycles are

even.

Practice problems

Problem 4.1. A $(4, 8)$-regular code has $n = 1200$. Find m, E and the design rate.

Solution $E = 4 \cdot 1200 = 4800$, $m = 4800/8 = 600$, $R = 1 - 4/8 = 0.5$.

Problem 4.2. Convert $L(x) = 0.6x^2 + 0.4x^4$ to the edge perspective.

Solution $\sum jL_j = 1.2 + 1.6 = 2.8$; $\lambda_2 = 1.2/2.8 = 0.4286$, $\lambda_4 = 1.6/2.8 = 0.5714$; $\lambda(x) = 0.4286x + 0.5714x^3$.

Problem 4.3. Find the design rate for $\lambda(x) = 0.4286x + 0.5714x^3$ and $\rho(x) = x^7$.

Solution $\int \lambda = 0.4286/2 + 0.5714/4 = 0.3571$; $\int \rho = 1/8 = 0.125$; $R = 1 - 0.125/0.3571 = 0.65$.

Problem 4.4. Does $\mathbf{H} = \begin{bmatrix} 1 & 1 & 1 & 0 \\ 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \end{bmatrix}$ contain a 4-cycle?

Solution Rows 0 and 1 share columns 0 and 1: yes, $v_0 - c_0 - v_1 - c_1 - v_0$.

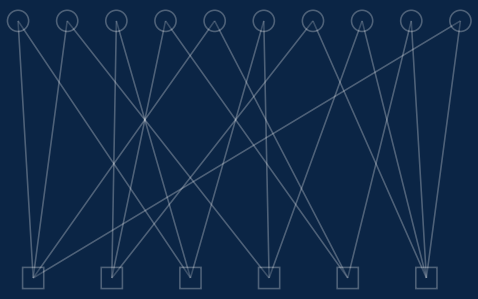
Self-assessment quiz

1. Why are all cycles of a Tanner graph of even length?
2. What is the edge-perspective degree distribution of a $(3, 6)$-regular code?
3. Why do short cycles hurt iterative decoding?
4. Which columns of 5G BG1 have the highest degree, and are they transmitted?

Answers: 1) The path alternates variable and check nodes. 2) $\lambda(x) = x^2$, $\rho(x) = x^5$. 3) A node's own information returns to it and is counted twice. 4) Columns 0 and 1 (degrees 30 and 28); they are punctured, not transmitted.

CHAPTER

5



Constructing LDPC Codes

IN THIS CHAPTER YOU WILL LEARN

- the design goals for a parity-check matrix;
- Gallager's band construction and its rank loss;
- MacKay's random constructions;
- progressive edge growth (PEG), with an example;
- circulant permutation matrices and lifting;
- quasi-cyclic (QC) codes and the 4-cycle condition;
- protographs, the idea behind 5G base graphs.

5.1 What makes a good parity-check matrix?

Sparse	few ones per row and column, so decoding is cheap
Good degree distribution	chosen by density evolution (Chapter 10) to give a low threshold
Few short cycles	large girth, no 4-cycles, few 6-cycles: good waterfall and low error floor
Known rank	so that the true rate is the wanted rate
Structure	cheap storage, parallel hardware, linear-time encoding (Chapter 6)

These goals pull in different directions: randomness helps the waterfall, structure helps the hardware. Modern codes (5G, Wi-Fi) are *structured* codes whose structure was optimised with random-like methods.

5.2 Gallager's construction

Definition 5.1 • Gallager (d_v, d_c) construction

Let n be a multiple of d_c. The first *band* is an $(n/d_c) \times n$ matrix whose row j has ones in columns $jd_c, \dots, jd_c + d_c - 1$. The other $d_v - 1$ bands are random column permutations of the first band. Stacking the d_v bands gives a (d_v, d_c)-regular $\mathbf{H}$ with $m = d_v n/d_c$ rows.

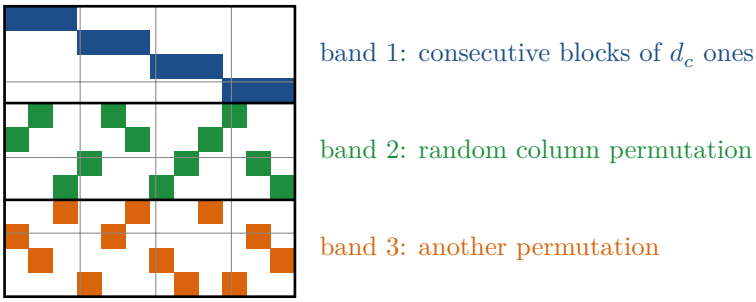


Figure 5.1. Gallager construction for $n = 12$, $d_v = 3$, $d_c = 3$ (illustration).

Worked Example 5.1 • A Gallager code of length 1008

`gallager_H(1008, 3, 6, seed=3)` gives a 504×1008 matrix. The library reports rank 502 (two redundant rows, as predicted in Chapter 3), girth 4, and 32 four-cycles. Random permutations do not care about cycles, so a few 4-cycles appear by chance.

5.3 MacKay's random constructions

MacKay and Neal built matrices column by column with a fixed column weight, placing the ones at random but rejecting choices that create a 4-cycle (two columns sharing two rows) and balancing the row weights. Variants put the weight-2 columns in a "staircase" so that they never form cycles among themselves. These codes showed in the 1990s that LDPC codes can match turbo codes, but the random placement makes storage and hardware wiring expensive.

5.4 Progressive edge growth (PEG)

Intuition

Build the graph one edge at a time. Before adding an edge to bit v, look at the part of the graph that v can already reach, and connect v to a check that is *as far away as possible*. A far-away check closes only a long cycle (or none). This greedy rule, proposed by Hu, Eleftheriou and Arnold (2005), gives a large girth for short and medium lengths.

Definition 5.2 • PEG algorithm

For each variable node v_j (low degrees first) and for each of its $d_v(j)$ edges:

1. **First edge:** connect v_j to a check of currently lowest degree.
2. **Other edges:** expand a breadth-first tree from v_j through the current graph, level by

level, until (a) the set of reached checks stops growing, or (b) all checks are reached.

3. In case (a) choose among the checks *not* reached; in case (b) choose among the checks reached only in the last level (the farthest ones).
4. Among the candidates, take one of lowest current degree (ties broken at random).

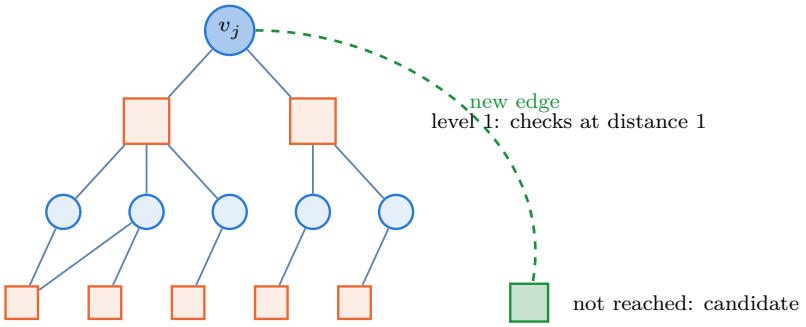


Figure 5.2. PEG: the new edge of v_j goes to a check that the breadth-first tree has not reached.

Worked Example 5.2 • PEG at two sizes

Small: `peg(12, 6, 2, seed=0)` gives (girth 6):

$$\mathbf{H} = \begin{bmatrix} 0&1&0&1&0&0&0&1&0&0&0&0 \\ 0&0&1&1&0&0&1&0&0&0&0&1 \\ 0&1&0&0&1&0&0&0&1&0&1&0 \\ 1&0&0&0&1&0&1&0&0&1&0&0 \\ 0&0&1&0&0&1&0&0&1&1&0&0 \\ 1&0&0&0&0&1&0&1&0&0&1&1 \end{bmatrix}.$$

Medium: `peg(1008, 504, 3, seed=1)` gives 3024 edges, check degrees 5 (22 checks), 6 (460) and 7 (22), full rank 504, **girth 8** and no 4-cycles. This is the code used for the decoder comparisons of Chapters 9 and 13. Compare with the Gallager matrix of the same size: girth 4 and 32 four-cycles.

Fig. 5.3 shows the PEG algorithm as a flow chart.

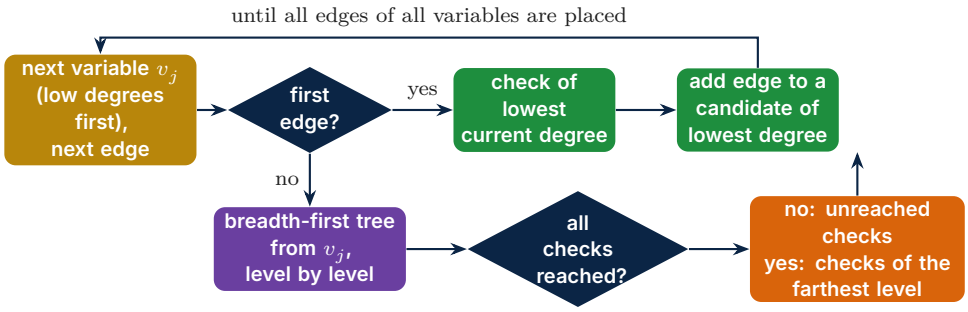


Figure 5.3. **Block diagram.** Progressive edge growth: each new edge goes as far away as possible in the current graph.

5.5 Quasi-cyclic codes

5.5.1 Circulant permutation matrices

Definition 5.3 • Circulant permutation matrix

$\mathbf{P}^s$ is the $Z \times Z$ identity matrix with its columns cyclically shifted to the right by s: row r has its single one in column $(r+s) \bmod Z$. By convention $\mathbf{P}^{-1}$ in a base matrix means the $Z \times Z$ *zero* block (not an inverse). Multiplying a length-Z vector by $\mathbf{P}^s$ is a cyclic shift — a cheap "barrel shifter" in hardware.

$$\mathbf{P}^0 = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}, \qquad \mathbf{P}^2 = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \end{bmatrix} \quad (Z = 5).$$

5.5.2 Lifting a base matrix

Definition 5.4 • QC-LDPC code

A **base (exponent) matrix** $\mathbf{V}$ of size $m_b \times n_b$ has entries -1 or $0 \le s < Z$. Replacing each entry by the block $\mathbf{P}^s$ (or the zero block) gives an $m_b Z \times n_b Z$ parity-check matrix. Z is the **lifting size**. The Tanner graph is Z copies of the small base graph whose edges are "rewired" cyclically.

Worked Example 5.3 • Lifting with $Z = 5$

$\mathbf{V} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 1 & 2 & 3 \end{bmatrix}$ lifted with $Z = 5$ gives a 10×20 matrix with column weight 2 and row weight 4. The library finds **girth 8**. If we change one shift, $\mathbf{V}' = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 3 \end{bmatrix}$, the girth drops to 4 and there are exactly 5 four-cycles.

Fig. 5.4 shows lifting as a processing chain. Only the small base matrix and Z are stored; the large matrix is generated when needed (in hardware it is never built at all: each entry is a cyclic shifter).

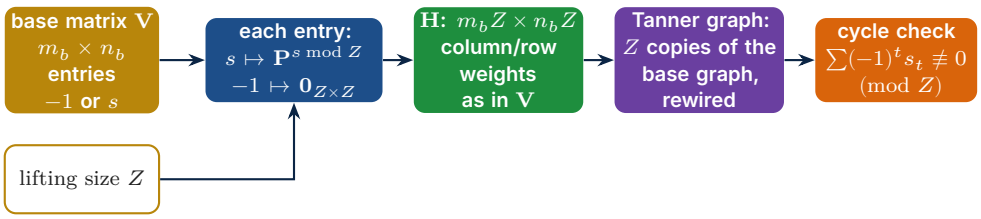


Figure 5.4. Block diagram. Lifting a base matrix into a quasi-cyclic parity-check matrix.

5.5.3 The cycle condition

A cycle of length 2ℓ in the base graph, passing through entries $s_1, s_2, \ldots, s_{2\ell}$ (alternately moving along a row and along a column), becomes cycles of length 2ℓ in the lifted graph if and only if

$$\sum_{t=1}^{2\ell} (-1)^t \, s_t \equiv 0 \pmod{Z}.$$

For a 4-cycle (rows a, b, columns i, j) the condition is $s_{ai} - s_{bi} + s_{bj} - s_{aj} \equiv 0 \pmod{Z}$ (Fossorier, 2004). In $\mathbf{V}'$ above, rows $0, 1$ and columns $1, 2$ give $0-1+1-0=0$: every lifted copy is a 4-cycle, which is why we found exactly $Z = 5$ of them. In $\mathbf{V}$, the same columns give $0-1+2-0=1 \neq 0$.

Professional insight

QC codes are the standard in practice (Wi-Fi, WiMAX, 5G NR, CCSDS). Benefits: (1) only the small base matrix and Z are stored; (2) the decoder processes Z rows in parallel with identical logic, connected by cyclic shifters; (3) one base matrix gives many lengths by changing Z (5G NR uses 51 lifting sizes, Chapter 11); (4) a dual-diagonal parity part gives linear-time encoding (Chapter 6). The price: girth of a QC code is at most 12 when all base entries are non-negative, and the structure must be optimised carefully.

5.6 Protographs

A **protograph** is a small base Tanner graph, possibly with *parallel* edges, and a **lifting** is Z copies of it whose edges are permuted among the copies. QC lifting is the special case with cyclic permutations. Protograph codes are designed by optimising the small base graph with density evolution (often with some punctured, high-degree nodes); the 5G NR base graphs are protograph designs with single edges, two punctured high-degree columns and a dual-diagonal core. The deep-space AR4JA codes are protograph codes as well.

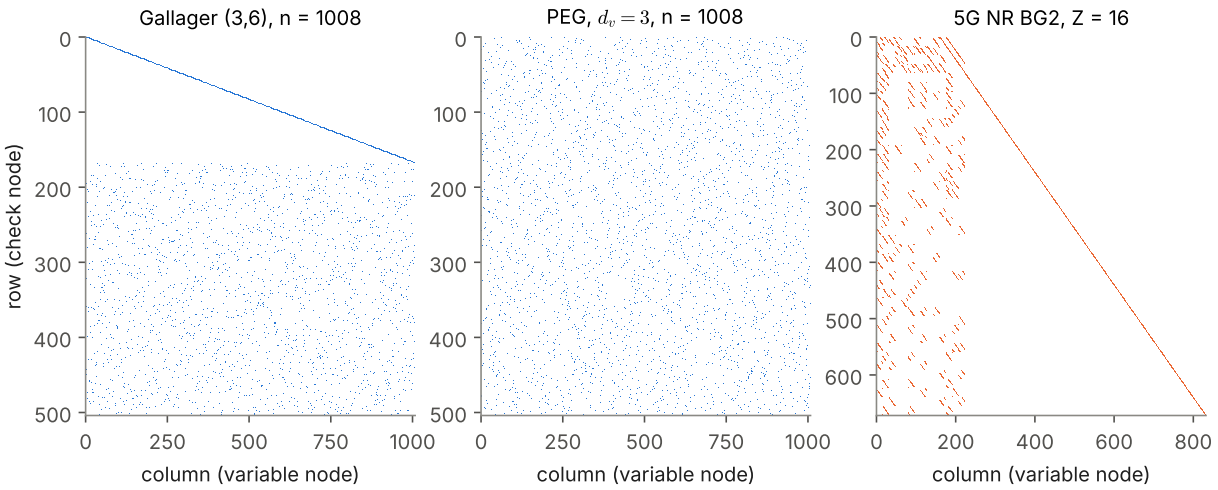


Figure 5.5. Positions of the ones in three parity-check matrices: a Gallager $(3, 6)$ code ($n = 1008$), the PEG code ($n = 1008$), and the 5G NR BG2 code lifted with $Z = 16$ (672×832).

5.7 Comparison of constructions

	Gallager	PEG	QC lifting	Protograph (5G)
Girth	random (4 possible)	large (8 at $n = 1008$)	controlled by shifts	optimised
Rank	loses $d_v - 1$	usually full	depends on shifts	designed
Storage	full list of edges	full list of edges	base matrix $+Z$	base matrix $+Z$
Hardware	irregular wiring	irregular wiring	cyclic shifters	cyclic shifters
Encoding	via $\mathbf{G}$ (dense)	via $\mathbf{G}$ or ALT	dual-diagonal	dual-diagonal core

5.8 In Python

All constructions are in lines 108–197 of `ldpc.py`. The QC lifting is short enough to show here:

ldpc.py • lines 154--164

```
154 def qc_lift(base, Z):
155     """Expand a base (exponent) matrix: -1 -> Z x Z zero block, s >= 0 ->
156     identity cyclically shifted to the right by s (row r has a 1 in column (r+s) mod Z)."""
157     base = np.asarray(base)
158     r = np.arange(Z)
159     cn, vn = [], []
160     for i, j in zip(*np.nonzero(base >= 0)):
161         cn.append(i * Z + r)
162         vn.append(j * Z + (r + base[i, j]) % Z)
163     return LDPCCode(base.shape[0] * Z, base.shape[1] * Z,
164                     np.concatenate(cn), np.concatenate(vn), f"QC(Z={Z})")
```

```
1 from ldpc import gallager_H, peg, qc_lift, girth, count_4cycles, LDPCCode
2 g = LDPCCode.from_H(gallager_H(1008, 3, 6, seed=3))
3 p = peg(1008, 504, 3, seed=1)
4 q = qc_lift([[0, 0, 0, 0], [0, 1, 2, 3]], 5)
5 for code in (g, p, q):
6     print(girth(code), count_4cycles(code))   # 4 32 / 8 0 / 8 0
```

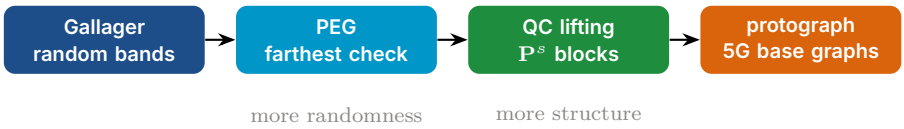
5.9 Chapter summary

Key concepts

- A good $\mathbf{H}$ is sparse, has a good degree distribution, few short cycles and useful structure.
- Gallager: stacked permuted bands (rank loss $d_v - 1$). MacKay: random columns avoiding 4-cycles.

- PEG adds each edge to the farthest check: girth 8 at $n = 1008$, $d_v = 3$.
- QC codes: base matrix of shifts, lifting size Z, circulant blocks; 4-cycle iff $s_{ai} - s_{bi} + s_{bj} - s_{aj} \equiv 0$.
- Protographs: small optimised base graphs lifted to any size; 5G NR uses them.

Visual recap • Chapter 5



Common student mistakes

! Common mistake

- Reading -1 in a base matrix as "shift by -1". It means a zero block.
- Forgetting to reduce shifts modulo Z: in 5G the table values V_{ij} are reduced as $P_{ij} = V_{ij} \bmod Z$.
- Testing the 4-cycle condition with ordinary sums instead of sums modulo Z.
- Believing that a large girth alone guarantees a good code.

Practice problems

Problem 5.1. For $Z = 7$, does the base block $\begin{bmatrix} 0 & 0 \\ 1 & 4 \end{bmatrix}$ create a 4-cycle? And $\begin{bmatrix} 2 & 5 \\ 6 & 2 \end{bmatrix}$?

Solution $0 - 1 + 4 - 0 = 3 \not\equiv 0$: no. $2 - 6 + 2 - 5 = -7 \equiv 0 \pmod 7$: yes, 7 four-cycles.

Problem 5.2. A base matrix of size 46×68 is lifted with $Z = 384$. What is the size of $\mathbf{H}$?

Solution $17\,664 \times 26\,112$ (this is 5G NR BG1 at its largest lifting size).

Problem 5.3. Why can a QC code with all base entries ≥ 0 never have girth larger than 12?

Solution A known result (Fossorier 2004): in a fully connected base matrix, a particular cycle of length 12 in the base graph always has an alternating shift sum equal to 0, so it lifts to 12-cycles for every choice of shifts.

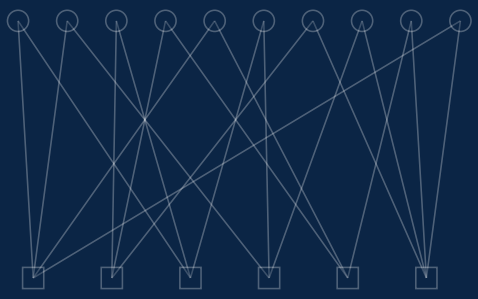
Self-assessment quiz

1. What does PEG choose when every check is already reachable from v_j?
2. What is stored in memory for a QC code?
3. Why does one 4-cycle in the base matrix give Z four-cycles in $\mathbf{H}$?
4. Name two standards that use QC-LDPC codes.

Answers: 1) A check reached only at the last level (the farthest), with lowest degree. 2) The base matrix and Z. 3) The Z copies of the cycle all satisfy the same shift condition. 4) Wi-Fi 802.11n and 5G NR (also WiMAX, CCSDS).

CHAPTER

6



Encoding LDPC Codes

IN THIS CHAPTER YOU WILL LEARN

- why encoding with a dense $\mathbf{G}$ is expensive;
- triangular parity parts and back-substitution;
- the dual-diagonal (staircase) structure;
- the quasi-cyclic dual-diagonal trick of Wi-Fi and 5G, with a full numerical example;
- the 5G NR encoder in three steps;
- the Richardson–Urbanke method for general $\mathbf{H}$.

6.1 The encoding problem

The decoder needs only the sparse $\mathbf{H}$. The encoder needs a rule that maps k message bits to $n-k$ parity bits. The general rule $\mathbf{c} = \mathbf{uG}$ costs about $k(n-k)/2$ XOR operations, because $\mathbf{G}$ is dense.

Worked Example 6.1 • Cost of G-encoding

For the PEG code of Chapter 5 ($n = 1008$, $k = 504$) the generator matrix from `generator_from_H` has 127 514 ones, against 3024 ones in $\mathbf{H}$: encoding with $\mathbf{G}$ costs about 42 times more XORs than one pass over $\mathbf{H}$. For a 5G code with $K = 8448$ and $n = 26\,112$ a dense $\mathbf{G}$ would have tens of millions of ones and would have to be stored for every one of the many code sizes. This is not practical.

The solution is to choose $\mathbf{H}$ with a parity part that can be solved quickly. Write the codeword as $\mathbf{c} = (\mathbf{s}, \mathbf{p})$ (systematic bits $\mathbf{s}$, parity bits $\mathbf{p}$) and split $\mathbf{H} = [\mathbf{A} \mid \mathbf{B}]$. Then

$$\mathbf{Hc}^T = \mathbf{As}^T \oplus \mathbf{Bp}^T = \mathbf{0} \quad \Longrightarrow \quad \mathbf{Bp}^T = \mathbf{As}^T =: \lambda^T.$$

The vector $\lambda = \mathbf{As}^T$ is cheap (sparse $\mathbf{A}$). Everything depends on how easy it is to solve $\mathbf{Bp}^T = \lambda^T$.

6.2 Triangular and dual-diagonal parity parts

If $\mathbf{B}$ is lower triangular with ones on the diagonal, the equations are solved one after another (**back-substitution**): $p_0 = \lambda_0$, then p_1 from row 1, and so on. The simplest useful case is the **dual-diagonal** (staircase) matrix

$$\mathbf{B} = \begin{bmatrix} 1 & & & & \\ 1 & 1 & & & \\ & 1 & 1 & & \\ & & \ddots & \ddots & \\ & & & 1 & 1 \end{bmatrix}, \qquad p_0 = \lambda_0, \quad p_j = p_{j-1} \oplus \lambda_j.$$

The parity bits are a running XOR (an *accumulator*) of λ: linear time, no storage. This is the structure of repeat-accumulate (RA) and irregular repeat-accumulate (IRA) codes, and the parity columns have degree 2.

6.3 The quasi-cyclic dual-diagonal trick

In a QC code each entry of $\mathbf{B}$ is a $Z \times Z$ circulant. Wi-Fi and 5G use a core of the form

$$\mathbf{B} = \begin{bmatrix} \mathbf{P}^a & \mathbf{I} & & \\ \mathbf{P}^b & \mathbf{I} & \mathbf{I} & \\ \vdots & & \ddots & \ddots \\ \mathbf{P}^a & & & \mathbf{I} \end{bmatrix}$$

(the exact pattern varies): the first parity block-column has weight 3 with the *same* shift a at the top and the bottom, and the other block-columns form a dual diagonal of identities.

Advanced • Why adding all block rows solves for $\mathbf{p}_0$

Add all block rows of $\mathbf{B}\mathbf{p}^T = \lambda^T$. Each identity block-column appears exactly twice and cancels; in the first block-column, the two $\mathbf{P}^a$ terms cancel too. What remains is $\mathbf{P}^b\mathbf{p}_0^T = \sum_r \lambda_r^T$, so

$$\mathbf{p}_0 = \mathbf{P}^{-b}\Big(\sum\nolimits_r \lambda_r\Big),$$

which is only XORs and one cyclic shift (no shift at all if $b = 0$). Then each remaining $\mathbf{p}_j$ follows from one block row by back-substitution.

Worked Example 6.2 • A complete QC encoding with $Z = 4$

Base matrix (3×6: 3 systematic and 3 parity block-columns), lifted with $Z = 4$ to a 12×24 $\mathbf{H}$ of girth 6:

$$\mathbf{V} = \left[\begin{array}{ccc|ccc} 1 & 1 & -1 & 1 & 0 & -1 \\ -1 & 2 & 3 & 0 & 0 & 0 \\ 3 & -1 & 1 & 1 & -1 & 0 \end{array}\right].$$

Parity block-column 0 has shifts $(1, 0, 1)$: top and bottom equal, middle $b = 0$. Message (three blocks of four bits): $\mathbf{s} = (\,1001 \mid 0110 \mid 1010\,)$. Recall $(\mathbf{P}^s\mathbf{x})_r = x_{(r+s) \bmod 4}$.

1. **Step 1, $\lambda = \mathbf{A}\mathbf{s}^T$ per block row** (computed with the library): $\lambda_0 = 1111$, $\lambda_1 = 1100$, $\lambda_2 = 1001$.

2. **Step 2, first parity block:** $\mathbf{p}_0 = \lambda_0 \oplus \lambda_1 \oplus \lambda_2 = 1111 \oplus 1100 \oplus 1001 = 1010$ (here $b = 0$, no shift).
3. **Step 3, back-substitution.** Row 0: $\lambda_0 \oplus \mathbf{P}^1\mathbf{p}_0 \oplus \mathbf{p}_1 = \mathbf{0}$. $\mathbf{P}^1\mathbf{p}_0 = 0101$, so $\mathbf{p}_1 = 1111 \oplus 0101 = 1010$. Row 2: $\mathbf{p}_2 = \lambda_2 \oplus \mathbf{P}^1\mathbf{p}_0 = 1001 \oplus 0101 = 1100$.
4. **Check row 1:** $\lambda_1 \oplus \mathbf{p}_0 \oplus \mathbf{p}_1 \oplus \mathbf{p}_2 = 1100 \oplus 1010 \oplus 1010 \oplus 1100 = 0000$. ✓

Codeword: $\mathbf{c} = (1001\,0110\,1010 \mid 1010\,1010\,1100)$. The library confirms $\mathbf{H}\mathbf{c}^T = \mathbf{0}$ and the class `DualEncoder` returns the same word.

6.4 The 5G NR encoder

Definition 6.1 • Structure of the 5G NR parity-check matrix

With $k_b = 22$ (BG1) or 10 (BG2) systematic block-columns, the lifted matrix has the block form

$$\mathbf{H} = \begin{bmatrix} \mathbf{A} & \mathbf{B} & \mathbf{0} \\ \mathbf{C}_1 & \mathbf{C}_2 & \mathbf{I} \end{bmatrix},$$

where $\mathbf{A}$ is $4Z \times k_b Z$, $\mathbf{B}$ is the $4Z \times 4Z$ dual-diagonal core, $[\mathbf{C}_1\ \mathbf{C}_2]$ are the extension rows, and $\mathbf{I}$ is an identity: every extension parity bit appears in exactly one check (degree-1 columns).

Worked Example 6.3 • The real 5G cores

From the TS 38.212 tables (set index $i_{LS} = 0$), the core block of BG1 (rows 0–3, columns 22–25) and of BG2 (rows 0–3, columns 10–13) are

$$\mathbf{B}_{\text{BG1}} : \begin{bmatrix} 1 & 0 & -1 & -1 \\ 0 & 0 & 0 & -1 \\ -1 & -1 & 0 & 0 \\ 1 & -1 & -1 & 0 \end{bmatrix}, \qquad \mathbf{B}_{\text{BG2}} : \begin{bmatrix} 0 & 0 & -1 & -1 \\ -1 & 0 & 0 & -1 \\ 1 & -1 & 0 & 0 \\ 0 & -1 & -1 & 0 \end{bmatrix}.$$

BG1: shifts $(1, 0, 1)$ in the first column, so $\mathbf{p}_0 = \sum_r \lambda_r$. BG2: shifts $(0, 1, 0)$, so $\mathbf{p}_0 = \mathbf{P}^{-1}\sum_r \lambda_r$. (For BG1 with $i_{LS} = 6$ the table has the value 105 in the middle, so the shift is 105 mod Z; for BG2 with $i_{LS} = 3, 7$ the pattern is $(1, 0, 1)$.)

5G NR LDPC encoding in three steps

1. $\lambda = \mathbf{A}\mathbf{s}^T$ (sparse, 4 block rows)
2. $\mathbf{p}_c = \mathbf{B}^{-1}\lambda$ (core parity, $4Z$ bits, by the trick above)
3. $\mathbf{p}_e = \mathbf{C}_1\mathbf{s}^T \oplus \mathbf{C}_2\mathbf{p}_c^T$ (extension parity: one XOR sum per row)

Step 3 is pleasant: because of the identity block, each extension parity bit is simply the XOR of the other bits of its check. The extension rows can be computed only as far as needed for the chosen code rate.

Fig. 6.1 shows the encoder as a block diagram. All operations are XORs of cyclically shifted blocks of Z bits.

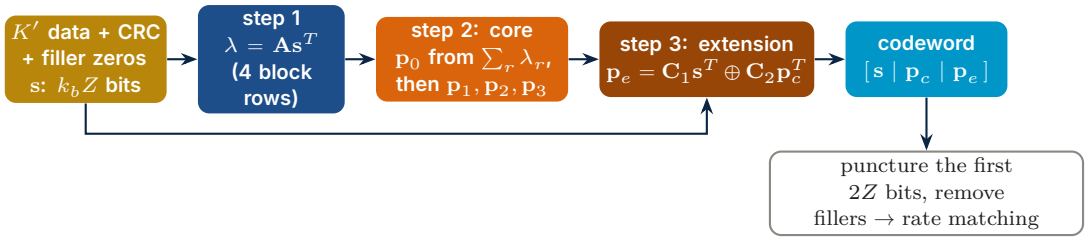


Figure 6.1. Block diagram. The three-step 5G NR LDPC encoder (`DualEncoder`).

Implementation in this book

`DualEncoder` (lines 283–319) follows the three steps. For step 2 it uses the inverse of the small $4Z \times 4Z$ core $\mathbf{B}$ over GF(2), computed once with `gf2_inv`. This is simple and fast enough in software; hardware uses the shift-and-XOR trick instead. We tested it for many (K, E) pairs of both base graphs, including the largest case $K = 8448$, $Z = 384$: every codeword satisfied $\mathbf{H}\mathbf{c}^T = \mathbf{0}$.

6.5 General matrices: the Richardson–Urbanke method

For an $\mathbf{H}$ without special structure, Richardson and Urbanke (2001) bring $\mathbf{H}$ by row and column permutations to an **approximate lower triangular** (ALT) form

$$\mathbf{H} = \begin{bmatrix} \mathbf{A} & \mathbf{B} & \mathbf{T} \\ \mathbf{C} & \mathbf{D} & \mathbf{E} \end{bmatrix},$$

where $\mathbf{T}$ is lower triangular and the "gap" g (the number of rows of $[\mathbf{C}\ \mathbf{D}\ \mathbf{E}]$) is small. With $\phi = \mathbf{E}\mathbf{T}^{-1}\mathbf{B} \oplus \mathbf{D}$ ($g \times g$, inverted once), the parity parts are

$$\mathbf{p}_1^T = \phi^{-1}(\mathbf{E}\mathbf{T}^{-1}\mathbf{A} \oplus \mathbf{C})\mathbf{s}^T, \qquad \mathbf{p}_2^T = \mathbf{T}^{-1}(\mathbf{A}\mathbf{s}^T \oplus \mathbf{B}\mathbf{p}_1^T),$$

and the cost is $O(n + g^2)$. For good LDPC ensembles the gap is a small fraction of n. The 5G structure is the special case $g = 4Z$ with a core that is easy to invert.

6.6 In Python

```python
from ldpc import NRCode, DualEncoder, qc_lift, encode_G, generator_from_H
nr = NRCode(K=1024, E=2048)          # BG2, Z = 104
c, x = nr.encode(u)                  # full codeword and the E transmitted bits
assert not nr.code.syndrome(c).any() # H c^T = 0 for every frame
```

For small codes without structure, `G, info = generator_from_H(H)` and `encode_G(G, u)` are enough (Chapter 3).

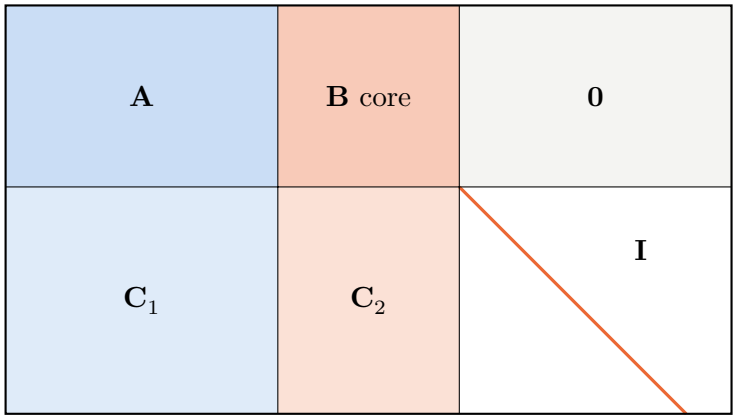
6.7 Chapter summary

Key concepts

- Dense $\mathbf{G}$ encoding costs $O(kn)$; structured $\mathbf{H}$ gives linear-time encoding.

- Split $\mathbf{H} = [\mathbf{A} \mid \mathbf{B}]$: solve $\mathbf{B}\mathbf{p}^T = \mathbf{A}\mathbf{s}^T$.
- Dual-diagonal $\mathbf{B}$: parity = running XOR (accumulator).
- QC core with a weight-3 column: sum all block rows to get $\mathbf{p}_0$, then back-substitute.
- 5G NR: $\lambda = \mathbf{A}\mathbf{s}$, core parity, extension parity by single XOR sums.
- General $\mathbf{H}$: approximate lower triangular form, cost $O(n + g^2)$.

Visual recap • Chapter 6



1) $\lambda = \mathbf{A}\mathbf{s}^T$
2) core: $\mathbf{p}_0 = \mathbf{P}^{-b} \sum \lambda_r$, then back-substitute
3) $\mathbf{p}_e = \mathbf{C}_1\mathbf{s}^T \oplus \mathbf{C}_2\mathbf{p}_c^T$

Common student mistakes

! Common mistake

- Building $\mathbf{G}$ for a long code "because the textbook formula is $\mathbf{c} = \mathbf{u}\mathbf{G}$". Use the structure of $\mathbf{H}$.
- Shifting in the wrong direction: with $\mathbf{P}^s$ defined as a right shift of the identity columns, $(\mathbf{P}^s\mathbf{x})_r = x_{(r+s) \bmod Z}$.
- Forgetting that in 5G the first $2Z$ systematic bits are encoded but *not transmitted* (punctured).

Practice problems

Problem 6.1. A dual-diagonal code has $\lambda = (1, 0, 0, 1, 1)$. Find $\mathbf{p}$.

Solution $p_0 = 1$, $p_1 = 1 \oplus 0 = 1$, $p_2 = 1 \oplus 0 = 1$, $p_3 = 1 \oplus 1 = 0$, $p_4 = 0 \oplus 1 = 1$: $\mathbf{p} = (1, 1, 1, 0, 1)$.

Problem 6.2. In the $Z = 4$ example, change the message to $\mathbf{s} = (1101 \mid 0110 \mid 0011)$, for which the library gives $\lambda_0 = 0111$, $\lambda_1 = 0000$, $\lambda_2 = 1000$. Find $\mathbf{p}_0, \mathbf{p}_1, \mathbf{p}_2$.

Solution $\mathbf{p}_0 = 0111 \oplus 0000 \oplus 1000 = 1111$; $\mathbf{P}^1\mathbf{p}_0 = 1111$; $\mathbf{p}_1 = 0111 \oplus 1111 = 1000$; $\mathbf{p}_2 = 1000 \oplus 1111 = 0111$. Row 1: $0000 \oplus 1111 \oplus 1000 \oplus 0111 = 0000$ ✓.

Problem 6.3. How many XOR operations does step 3 of the 5G encoder need for one extension row of degree d?

Solution The parity bit is the XOR of the other $d - 1$ bits: $d - 2$ XOR operations.

Self-assessment quiz

1. Why is the generator matrix of an LDPC code usually dense?
2. What is an accumulator, and which parity structure gives it?
3. In the QC dual-diagonal core, why must the top and bottom shifts of the weight-3 column be equal?
4. What does "gap" mean in the Richardson–Urbanke method?

Answers: 1) $\mathbf{G}$ spans the null space of $\mathbf{H}$; nothing forces it to be sparse. 2) A running XOR; dual-diagonal $\mathbf{B}$. 3) So that they cancel when all block rows are added. 4) The number of rows that are not in triangular form; the cost grows with its square.

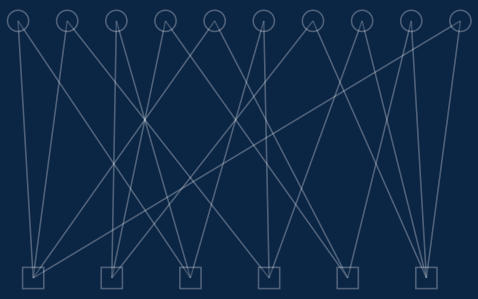
PART III

Advanced Level: Decoding and Analysis

From bit flipping and peeling to sum-product, min-sum and layered decoding, and the theory that predicts their limits: density evolution and EXIT charts.

CHAPTER

7



Erasure Decoding and Bit Flipping

IN THIS CHAPTER YOU WILL LEARN

- the idea of iterative message passing;
- the peeling decoder for the erasure channel;
- stopping sets: exactly when peeling fails;
- why one bad pair of columns creates an error floor;
- Gallager's bit-flipping decoder on the BSC;
- how much hard decisions cost compared with soft LLRs.

7.1 Iterative message passing in one paragraph

Every LDPC decoder in this book has the same shape: variable nodes and check nodes take turns. In each half-iteration a node reads the messages on its edges, applies a small *local rule*, and writes a new message on each edge. After each iteration the decoder makes tentative decisions and stops if the syndrome is zero. The decoders differ only in what a message is (an erasure flag, a bit, an LLR) and in the local rules. This chapter starts with the two simplest message types: erasures and hard bits.

The extrinsic principle

The message that node a sends to its neighbour b must not use the message that b just sent to a. Otherwise b would hear its own opinion again (the "echo" of Chapter 4). Every local rule below excludes the target edge.

7.2 Decoding on the binary erasure channel

On the BEC each bit arrives either correctly or as an erasure "?". Nothing arrives wrong. A parity check with exactly *one* erased bit can therefore recover it: the erased bit equals the XOR of the other bits of the check.

Definition 7.1 • Peeling decoder

Repeat until no erasure is left or no progress is made:

1. find every check that has exactly one erased neighbour;
2. set that erased bit to the XOR of the known bits of the check.

In graph language: remove ("peel") the known variable nodes and their edges; a check of residual degree one resolves its last neighbour.

Worked Example 7.1 • Peeling three erasures

Codeword $\mathbf{c} = (0, 0, 1, 0, 1, 1)$ of the toy code, received $\mathbf{y} = (?, 0, ?, 0, 1, ?)$ (bits 0, 2, 5 erased). Erasures per check at the start: $c_0 : 1$, $c_1 : 1$, $c_2 : 2$, $c_3 : 2$.

- **Round 1** (all checks in parallel): $c_0 = \{0, 1, 3\}$ gives $c_0 \leftarrow y_1 \oplus y_3 = 0 \oplus 0 = 0$; $c_1 = \{1, 2, 4\}$ gives $c_2 \leftarrow y_1 \oplus y_4 = 0 \oplus 1 = 1$.
- **Round 2**: $c_2 = \{0, 4, 5\}$ now has one erasure: $c_5 \leftarrow 0 \oplus 1 = 1$ (c_3 gives the same value).

Result $\hat{\mathbf{c}} = (0, 0, 1, 0, 1, 1)$ after 2 rounds, as returned by `peel`.

Fig. 7.1 shows the peeling decoder as a flow chart.

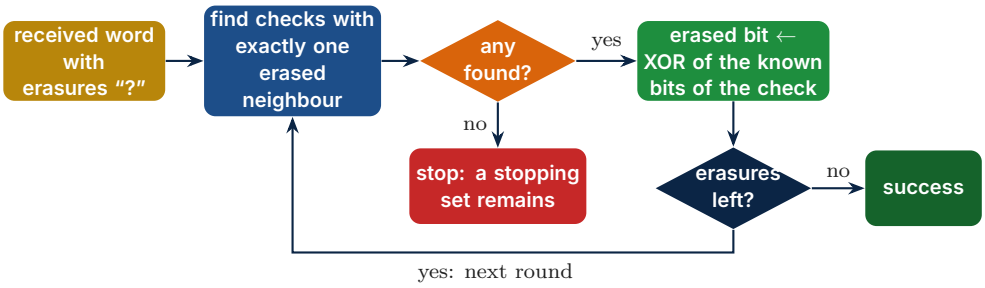


Figure 7.1. Block diagram. Peeling decoder on the binary erasure channel.

7.2.1 Stopping sets

Peeling stops when every check that touches an erased bit touches *at least two* erased bits.

Definition 7.2 • Stopping set

A set $\mathcal{S}$ of variable nodes is a **stopping set** if every check node connected to $\mathcal{S}$ is connected to $\mathcal{S}$ at least twice. The peeling decoder fails if and only if the set of erased positions contains a non-empty stopping set; it then stops with exactly the largest such set still erased.

Worked Example 7.2 • Stopping sets of the toy code

Searching all subsets, the smallest stopping sets have size 3: $\{0, 1, 4\}$, $\{0, 3, 5\}$, $\{1, 2, 3\}$, $\{2, 4, 5\}$. For $\{0, 1, 4\}$: check c_0 contains bits $0, 1$; c_1 contains $1, 4$; c_2 contains $0, 4$; c_3 contains none. Each of these sets is also the support of a weight-3 codeword (110010, 100101, 011100, 001011). Erasing only bits $\{1, 3\}$ (no stopping set inside) is always recovered.

Professional insight

The support of every codeword is a stopping set, but not every stopping set is a codeword. If a codeword's support is erased, even the optimal (maximum-likelihood) decoder cannot decide. If a stopping set that is not a codeword is erased, ML could succeed but peeling cannot: this is the price of the simple iterative decoder.

7.2.2 Threshold behaviour and error floors

Fig. 7.2 shows the frame error rate of peeling for $(3, 6)$-type codes of several lengths (all-zero codeword, which is enough because the code is linear and the BEC is symmetric). Two effects are visible:

- **Threshold.** As n grows the curves become steeper around $\epsilon \approx 0.43$. Chapter 10 computes the exact limit for infinitely long $(3, 6)$ codes: $\epsilon^* = 0.4294$, below the capacity limit 0.5. At $\epsilon = 0.41$ the $n = 8064$ code failed in only 1 of 600 frames; at $\epsilon = 0.43$ it failed in 61% of them.
- **Error floor.** The Gallager code with $n = 504$ has an FER of about 0.1 even at $\epsilon = 0.30$. The reason: its random construction produced two *identical columns* (230 and 233). They form a stopping set of size 2 (and a codeword of weight 2), so the frame fails whenever both bits are erased, with probability ϵ^2 (dotted line). A single bad pair of columns ruins the code. The PEG code has no such defect.

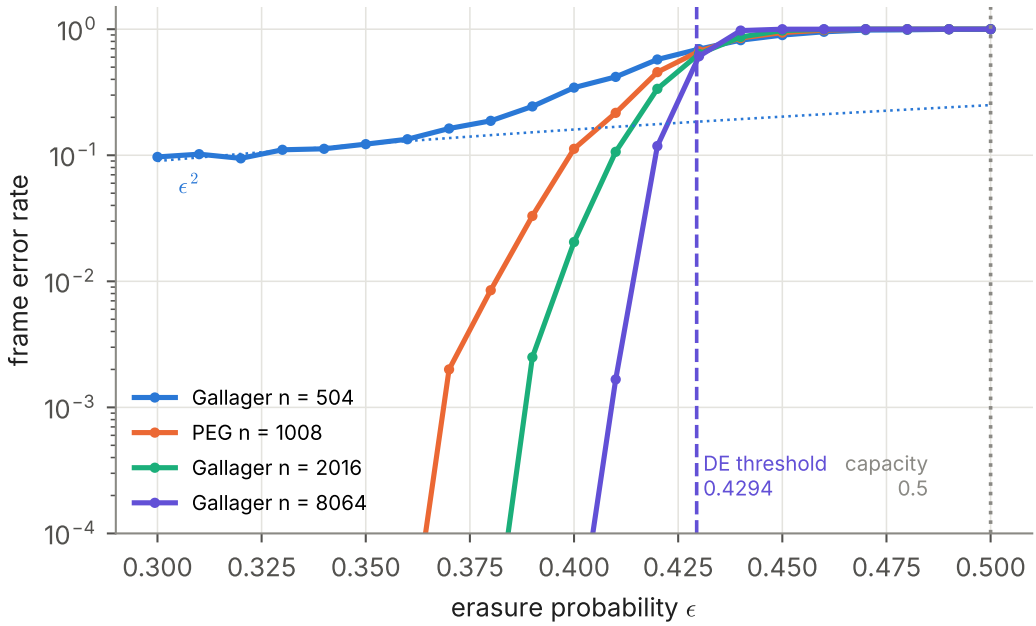


Figure 7.2. Peeling decoder on the BEC (Monte-Carlo, 600–2000 frames per point).

7.3 Bit flipping on the binary symmetric channel

On the BSC the decoder receives hard bits, some of them wrong. Gallager proposed a simple rule: a bit that sits in many unsatisfied checks is suspicious — flip it.

Definition 7.3 • Parallel bit-flipping (BF) decoder

Start with $\hat{\mathbf{c}} = \mathbf{r}$. Repeat up to $I_{\max}$ times:

1. compute the syndrome $\mathbf{s} = \mathbf{H}\hat{\mathbf{c}}^T$; stop if $\mathbf{s} = \mathbf{0}$;
2. for each bit i count its failed checks $f_i = \sum_{j:H_{ji}=1} s_j$;
3. flip every bit whose f_i equals the largest count $\max_i f_i$.

Fig. 7.3 shows the same steps as a block diagram with its feedback loop.

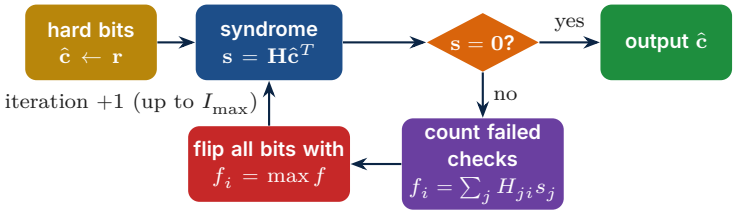


Figure 7.3. Block diagram. Parallel bit-flipping decoder.

Worked Example 7.3 • Bit flipping on the toy code

Sent $\mathbf{c} = (0,0,1,0,1,1)$, received $\mathbf{r} = (0,0,1,1,1,1)$ (bit 3 wrong). Syndrome $\mathbf{s} = (1,0,0,1)$. Failed checks per bit: $\mathbf{f} = (1,1,1,2,0,1)$ (bit 3 is in both failed checks c_0 and c_3). The maximum 2 belongs only to bit 3; flipping it gives a zero syndrome after one iteration.

Advanced • Why one error is always corrected when $d_v = 3$ and $g \geq 6$

If only bit i is wrong, all its $d_v = 3$ checks fail, so $f_i = 3$. Any other bit j shares at most one check with i (no 4-cycles), so $f_j \leq 1$. The maximum is unique and the right bit is flipped.

7.3.1 Variants

- **Gallager A and B** are message-passing versions: each variable node sends to each check its received bit unless (A) all other checks, or (B) at least b other checks, disagree with it. They can be analysed exactly by density evolution.
- **Weighted bit flipping** uses the magnitudes $|y_i|$: a failed check is weighted by the reliability of its least reliable bit, and the bit with the largest weighted sum is flipped. It narrows part of the gap to soft decoding.

7.4 The price of hard decisions

Worked Example 7.4 • Hard versus soft Shannon limits

Making hard decisions turns the BI-AWGN channel into a BSC with $p = Q(\sqrt{2R\,E_b/N_0})$. For $R = 1/2$, the BSC capacity equals $1/2$ at $p = 0.110$, i.e. $E_b/N_0 = 1.77$ dB, compared with 0.19 dB for soft BPSK: **about 1.6 dB is lost** before any decoder is chosen (for $R = 1/3$: 1.21 versus -0.50 dB; for $R = 3/4$: 3.01 versus 1.63 dB).

Simple BF loses even more because its rule is crude. In Chapter 13 the BF decoder on the PEG code needs about 3.6 dB more than the sum-product decoder at FER 10^{-2} (5.71 versus 2.11 dB).

That is why every modern receiver uses soft LLRs and the decoders of Chapters 8 and 9. Bit flipping survives where power is extremely limited (some optical and storage systems) and as a teaching tool.

7.5 In Python

Both decoders are in lines 343–379 of `ldpc.py` and work on a batch of frames at once:

```
from ldpc import peel, bit_flip
x, rounds = peel(code, y)            # y: 0, 1 or -1 (erasure); x: -1 where still erased
x, iters  = bit_flip(code, r, 50)    # r: hard bits; x: decoded bits
```

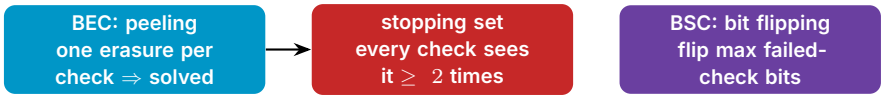
`peel` counts the erased edges per check with one `np.add.reduceat`, and fills all solvable bits of all frames in one vectorised step per round.

7.6 Chapter summary

Key concepts

- Messages never return the information received from the target edge (extrinsic principle).
- BEC: a check with one erasure resolves it; peeling fails exactly on stopping sets.
- Small stopping sets (and low-weight codewords) cause error floors.
- BSC: bit flipping flips the bits that sit in the most failed checks.
- Hard decisions cost about 1.6 dB at $R = 1/2$ before decoding even starts.

Visual recap • Chapter 7



Common student mistakes

! Common mistake

- Updating a check in the same round with a value that was just found by another check and calling it "parallel". Both schedules work on the BEC, but they need different numbers of rounds.
- Believing that peeling failure means ML failure: only stopping sets that are codeword supports defeat ML.
- Flipping *all* bits that have at least one failed check: this oscillates and rarely converges.

Practice problems

Problem 7.1. Toy code, received $(0, ?, 1, ?, 1, 1)$. Decode by peeling.

Solution Erased bits 1 and 3. Check $c_1 = \{1, 2, 4\}$ has one erasure: $c_1 \leftarrow 1 \oplus 1 = 0$. Check $c_3 = \{2, 3, 5\}$: $c_3 \leftarrow 1 \oplus 1 = 0$. Result $(0, 0, 1, 0, 1, 1)$ in one round.

Problem 7.2. Is $\{0, 1, 2, 3\}$ a stopping set of the toy code?

Solution Check $c_2 = \{0, 4, 5\}$ meets the set only in bit 0 (once): no. (Peeling recovers bit 0 from c_2.)

Problem 7.3. Toy code, received $\mathbf{r} = (1, 0, 1, 1, 1, 0)$. Run bit flipping.

Solution $\mathbf{s} = (1 \oplus 0 \oplus 1,\ 0 \oplus 1 \oplus 1,\ 1 \oplus 1 \oplus 0,\ 1 \oplus 1 \oplus 0) = (0, 0, 0, 0)$. It is already a codeword (101110): nothing to flip.

Problem 7.4. A BEC with $\epsilon = 0.1$ is used with a code that has one stopping set of size 2 (and no other small one). Estimate its frame error floor.

Solution About $\epsilon^2 = 0.01$.

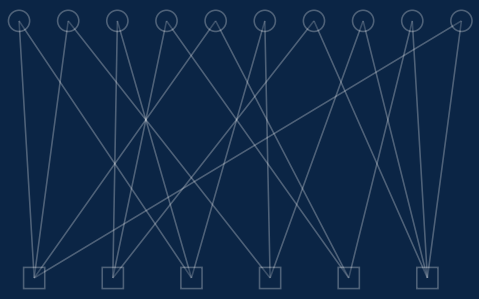
Self-assessment quiz

1. Why can the peeling decoder never make a wrong decision?
2. What is the relation between codewords and stopping sets?
3. Why does bit flipping correct any single error when $d_v = 3$ and the girth is at least 6?
4. How many dB does the hard decision itself cost at $R = 1/2$?

Answers: 1) On the BEC every known bit is correct and parity is exact. 2) Every codeword support is a stopping set; not conversely. 3) The wrong bit has 3 failed checks; any other bit at most 1. 4) About 1.6 dB.

CHAPTER

8



Belief Propagation: the Sum-Product Algorithm

IN THIS CHAPTER YOU WILL LEARN

- what the optimal bit decision is (bit-MAP);
- the variable-node rule, derived from Bayes;
- the check-node rule, derived from the XOR of independent bits (tanh rule);
- the ϕ-function form used in practice;
- the complete flooding sum-product algorithm;
- a full numerical example on the toy code;
- why BP is exact on trees and over-confident on cycles.

8.1 From hard bits to soft beliefs

Intuition

Bit flipping only asks "is this check satisfied?". Belief propagation asks "*how sure* are we about each bit?". Every node keeps a belief (an LLR) and tells its neighbours how sure *it* is, given everything else it knows. A check node says to a bit: "looking at my other bits and their reliabilities, I think you are a 1 with confidence 2.3". A bit node adds up the advice of its checks and its own channel observation. After a few rounds, the beliefs agree with all checks.

8.2 The goal: bit-wise MAP decoding

The best decision for a single bit c_i, given all received samples $\mathbf{y}$, uses its a-posteriori LLR

$$L_i^{\text{MAP}} = \ln \frac{P(c_i = 0 \mid \mathbf{y})}{P(c_i = 1 \mid \mathbf{y})} = \ln \frac{\sum_{\mathbf{c} \in \mathcal{C}:\, c_i = 0} \prod_j p(y_j \mid c_j)}{\sum_{\mathbf{c} \in \mathcal{C}:\, c_i = 1} \prod_j p(y_j \mid c_j)}.$$

The sums run over 2^{k-1} codewords each: impossible for real codes. Belief propagation computes this quantity *exactly* when the Tanner graph has no cycles, and a very good approximation when the cycles are long.

8.3 The two local rules

8.3.1 Variable-node rule

A variable node v has its channel LLR L_v^{ch} and one incoming message $L_{c\to v}$ from each of its checks. If these pieces of evidence are independent, their LLRs add (Rule 1 of Chapter 2). The message to check c must leave out what c itself said:

$$L_{v\to c} = L_v^{\text{ch}} + \sum_{c'\in\mathcal{N}(v)\setminus c} L_{c'\to v}, \qquad L_v^{\text{app}} = L_v^{\text{ch}} + \sum_{c'\in\mathcal{N}(v)} L_{c'\to v}.$$

A useful identity for software: $L_{v\to c} = L_v^{\text{app}} - L_{c\to v}$ (compute the total once, then subtract).

8.3.2 Check-node rule

A check c knows that the XOR of its bits is 0, so bit v equals the XOR of the *other* bits: $c_v = \bigoplus_{v'\neq v} c_{v'}$. We need the LLR of an XOR of independent bits.

Advanced • Derivation of the tanh rule

For one bit with LLR L: $P(0) = \frac{e^{L/2}}{e^{L/2}+e^{-L/2}}$, $P(1) = \frac{e^{-L/2}}{e^{L/2}+e^{-L/2}}$, so

$$P(0) - P(1) = \tanh\frac{L}{2}.$$

For the XOR $b = b_1 \oplus b_2$ of two independent bits, $P(b=0) - P(b=1) = (P_1(0)P_2(0) + P_1(1)P_2(1)) - (P_1(0)P_2(1) + P_1(1)P_2(0)) = (P_1(0) - P_1(1))(P_2(0) - P_2(1))$. By induction the differences multiply for any number of bits. Converting back with $L = 2\tanh^{-1}(P(0)-P(1))$:

$$L_{c\to v} = 2\tanh^{-1}\Big(\prod_{v'\in\mathcal{N}(c)\setminus v} \tanh\frac{L_{v'\to c}}{2}\Big) = \boxplus_{v'\in\mathcal{N}(c)\setminus v} L_{v'\to c}.$$

The symbol $\boxplus$ ("box-plus") is the same operation as the f function of polar SC decoding [Part One, Ch. 6].

8.3.3 The ϕ-form

Products of many numbers close to ± 1 lose precision. Split sign and magnitude: with Gallager's function

$$\phi(x) = -\ln\tanh\frac{x}{2} = \ln\frac{e^x+1}{e^x-1}, \qquad x>0,$$

which is its own inverse ($\phi(\phi(x)) = x$), the check rule becomes

$$\operatorname{sign}(L_{c\to v}) = \prod_{v'\neq v}\operatorname{sign}(L_{v'\to c}), \qquad |L_{c\to v}| = \phi\Big(\sum_{v'\neq v}\phi(|L_{v'\to c}|)\Big).$$

Products became sums. ϕ is large for small x: an unreliable input dominates the sum and makes the output unreliable — a check is only as strong as its weakest bit. The library uses this form (lines 384–436): it computes the total $\sum\phi$ once per check and subtracts each edge's own term.

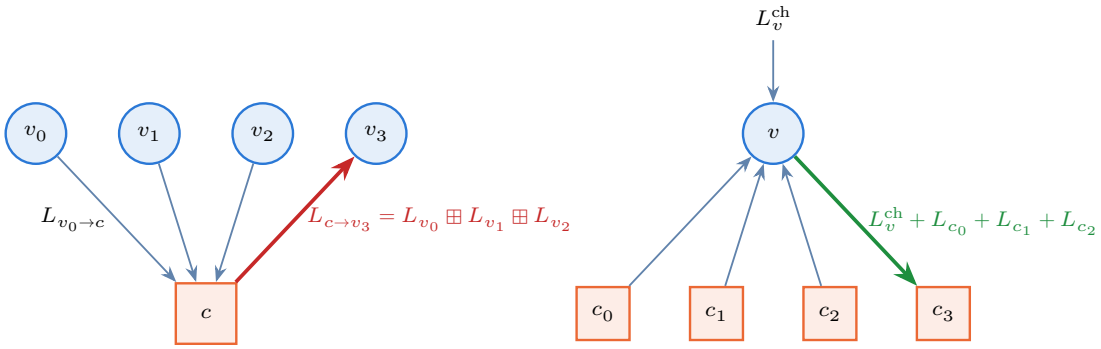


Figure 8.1. Left: check-node update (box-plus of the other inputs). Right: variable-node update (sum of the channel and the other inputs). The target edge is always left out.

8.4 The sum-product algorithm

Definition 8.1 • Flooding sum-product (SPA) decoder

1. **Initialise:** $L_{c \to v} = 0$ for all edges; $L_v^{\mathrm{app}} = L_v^{\mathrm{ch}}$.
2. **Variable-to-check:** for every edge, $L_{v \to c} = L_v^{\mathrm{app}} - L_{c \to v}$.
3. **Check-to-variable:** for every edge, $L_{c \to v} = \boxplus_{v' \neq v} L_{v' \to c}$ (tanh or ϕ form).
4. **A-posteriori LLR:** $L_v^{\mathrm{app}} = L_v^{\mathrm{ch}} + \sum_c L_{c \to v}$.
5. **Decide and test:** $\hat{c}_v = 1$ if $L_v^{\mathrm{app}} < 0$. If $\mathbf{H}\hat{\mathbf{c}}^T = \mathbf{0}$, stop; else go to 2, up to $I_{\max}$ iterations.

"Flooding" means that all checks are updated, then all variables, in each iteration.

Fig. 8.2 shows the flooding decoder as a hardware block diagram. The Tanner graph appears as an *interconnect* (edge permutation) between a bank of variable-node units and a bank of check-node units.

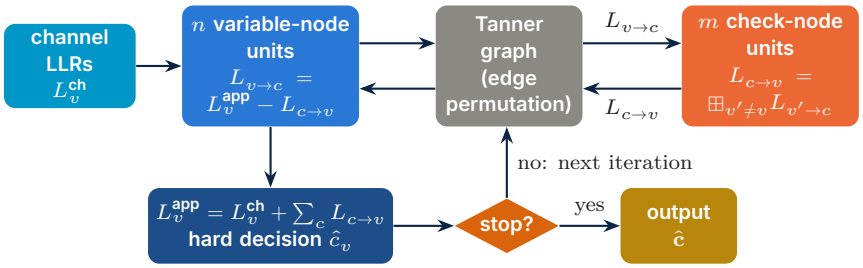


Figure 8.2. Block diagram. Flooding sum-product decoder: variable-node and check-node units exchange messages through the Tanner graph; it stops when $\mathbf{H}\hat{\mathbf{c}}^T = \mathbf{0}$ or after $I_{\max}$ iterations.

8.5 A complete numerical example

Worked Example 8.1 • Sum-product on the toy code

Sent $\mathbf{c} = (0,0,1,0,1,1)$, BPSK $\mathbf{x} = (+1,+1,-1,+1,-1,-1)$. With $\sigma^2 = 0.5$ the receiver gets

$$\mathbf{y} = (0.9,\ 1.2,\ -0.8,\ -0.3,\ -1.1,\ -0.7), \qquad \mathbf{L}^{\text{ch}} = 4\mathbf{y} = (3.6,\ 4.8,\ -3.2,\ -1.2,\ -4.4,\ -2.8).$$

The hard decision $(0,0,1,\mathbf{1},1,1)$ has bit 3 wrong. In iteration 1 every $L_{v\to c}$ equals the channel LLR.

Check messages, one by hand. Check $c_0 = \{v_0, v_1, v_3\}$ to v_0: $\tanh(4.8/2)\tanh(-1.2/2) = 0.98367 \cdot (-0.53705) = -0.52828$, so $L_{c_0\to v_0} = 2\tanh^{-1}(-0.52828) = -1.1755$. Check c_0 to v_3: $\tanh(1.8)\tanh(2.4) = 0.94681 \cdot 0.98367 = 0.93135$, $L_{c_0\to v_3} = +3.3369$. The check "tells" bit 3: your two partners are confidently 0, so you should be 0 too.

All check messages of iteration 1 (from the library):

	v_0	v_1	v_2	v_3	v_4	v_5
c_0	-1.1755	-1.1214		$+3.3369$		
c_1		$+2.9372$	-3.8871		-3.0164	
c_2	$+2.6168$				-2.4306	-3.2292
c_3			$+1.0342$	$+2.2895$		$+1.0853$

A-posteriori LLRs: e.g. $L_3^{\text{app}} = -1.2 + 3.3369 + 2.2895 = +4.4264$. All six:

$$\mathbf{L}^{\text{app}} = (5.0413,\ 6.6159,\ -6.0528,\ \mathbf{4.4264},\ -9.8470,\ -4.9440) \ \Rightarrow\ \hat{\mathbf{c}} = (0,0,1,0,1,1).$$

The syndrome is zero: the decoder stops after **one iteration** with the correct codeword. Had it continued, the next message from v_3 to c_0 would be $L_{v_3\to c_0} = -1.2 + 2.2895 = +1.0895$ (channel plus c_3 only).

Worked Example 8.2 • How close is BP to the exact MAP?

Summing over the 8 codewords gives the exact bit-MAP LLRs $(5.1924,\ 6.7845,\ -6.7392,\ 5.0063,\ -9.2271,\ -5.1919)$. After one iteration BP is close (same signs, magnitudes within about 0.7) but not equal, because the toy graph has 6-cycles. If we do *not* stop and run 20 iterations, the LLRs grow to about ± 60: the cycles feed every node's own evidence back to it, and BP becomes over-confident. The decisions stay correct, which is what matters for decoding.

8.6 BP is exact on trees

Advanced • Why

On a cycle-free graph, the sub-trees hanging from the different edges of a node use disjoint sets of channel samples, so the incoming messages are truly independent and Rules 1 and 2 are exact. By induction from the leaves, each message equals the exact LLR of its bit given the samples in its sub-tree, and L_v^{app} uses all samples: it is the bit-MAP LLR.

Worked Example 8.3 • Checking exactness numerically

The chain code with checks $\{0,1,2\}$, $\{2,3,4\}$, $\{4,5,6\}$ ($n = 7$) has no cycles. For 200 random LLR vectors we ran 10 SPA iterations and compared with brute-force MAP over its 16 codewords. The largest difference was 1.2×10^{-14} — rounding error only. This test is part of `test_ldpc.py`.

! Common mistake

Codes without cycles are weak (Etzion, Trachtenberg and Vardy showed that cycle-free codes of rate $\geq 1/2$ have $d_{\min} \leq 2$). Good LDPC codes *have* cycles; we only want them to be long, so the computation tree stays a tree for the first iterations (Chapter 4).

8.7 Practical issues

- **Saturation.** $\tanh(x/2)$ rounds to ± 1 for $|x| > 38$ in double precision, and $\tanh^{-1}(\pm 1) = \pm\infty$. The library clips the argument of ϕ to $[10^{-12}, 50]$, which limits message magnitudes to about 28.
- **Complexity.** Per iteration: one subtraction per edge (variable side), two ϕ evaluations and one subtraction per edge (check side), and one sum per variable. Total $O(E)$, i.e. linear in n.
- **Hardware** replaces ϕ by a small look-up table, or by the min-sum approximation of Chapter 9.
- **Early stopping** by the syndrome is free and saves most iterations at high E_b/N_0 (Chapter 13).

8.8 In Python

`ldpc.py` • lines 384--436

```
# ------------------------------------------------------------ 8. soft decoders (flooding)
def phi(x):
    """Gallager's function phi(x) = -ln tanh(x/2); it is its own inverse."""
    x = np.clip(x, 1e-12, 50.0)
    return -np.log(np.tanh(x / 2))


def check_update(code, v2c, rule="spa", alpha=0.75, beta=0.5, starts=None, seg=None):
    """Check-node messages from variable-to-check messages v2c (B, E).
    rule: 'spa' (sum-product), 'ms' (min-sum), 'nms' (normalised, x alpha),
    'oms' (offset, - beta). starts/seg describe the checks (default: all)."""
    st = code.cstart if starts is None else starts
    seg = code.cn if seg is None else seg                          # check of each edge
    neg = (v2c < 0).astype(np.int64)
    sign_all = np.add.reduceat(neg, st, axis=1) % 2               # parity of negatives
    sign = 1 - 2 * ((sign_all[:, seg] + neg) % 2)                  # exclude own sign
    mag = np.abs(v2c)
```

```
    if rule == "spa":
        p = phi(mag)
        out = phi(np.add.reduceat(p, st, axis=1)[:, seg] - p)
    else:
        m1 = np.minimum.reduceat(mag, st, axis=1)                  # smallest magnitude
        is1 = mag == m1[:, seg]
        n1 = np.add.reduceat(is1.astype(np.int64), st, axis=1)     # how many equal it
        m2 = np.minimum.reduceat(np.where(is1, np.inf, mag), st, axis=1)
        m2 = np.where(n1 >= 2, m1, m2)                             # second smallest
        out = np.where(is1, m2[:, seg], m1[:, seg])
        if rule == "nms":
            out = alpha * out
        elif rule == "oms":
            out = np.maximum(out - beta, 0.0)
    return sign * out


def bp_decode(code, llr, max_iter=50, rule="spa", alpha=0.75, beta=0.5, early_stop=True):
    """Flooding belief propagation. Returns hard decisions (B, n), the number of
    iterations used per frame and the final a-posteriori LLRs."""
    llr = np.atleast_2d(llr).astype(float)
    B = llr.shape[0]
    app, iters = llr.copy(), np.full(B, max_iter)
    act = np.arange(B)                                      # frames still decoding
    c2v = np.zeros((B, code.E))
    for it in range(1, max_iter + 1):
        v2c = app[act][:, code.vn] - c2v                    # extrinsic: remove own message
        c2v = check_update(code, v2c, rule, alpha, beta)
        app[act] = llr[act] + code.var_sum(c2v)             # a-posteriori LLRs
        if early_stop:
            ok = ~code.syndrome((app[act] < 0).astype(np.uint8)).any(axis=1)
            iters[act[ok]] = it
            act, c2v = act[~ok], c2v[~ok]                   # stop decoding valid codewords
            if act.size == 0:
                break
    return (app < 0).astype(np.uint8), iters, app
```

- `check_update` is vectorised over all edges of all frames. The parity of negative inputs per check (`sign_all`) and the total $\sum \phi$ per check are computed with `np.add.reduceat`; each edge then removes its own contribution — the extrinsic principle in two lines.
- `bp_decode` keeps only the frames that have not yet converged (`act`), so the work shrinks as frames finish.

8.9 Chapter summary

Key concepts

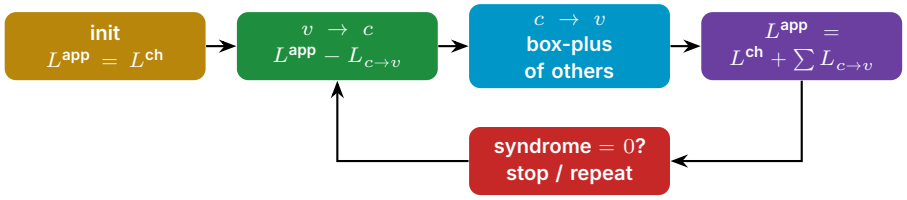
- BP approximates bit-MAP decoding by local message passing; it is exact on trees.
- Variable node: add channel LLR and the other incoming check messages.
- Check node: box-plus of the other incoming messages (tanh rule or ϕ form).

- Flooding schedule: all checks, then all variables; stop when the syndrome is zero.
- Cycles make BP over-confident but the decisions are usually still right.

Important formulas

Variable to check	$L_{v\to c} = L_v^{\mathrm{ch}} + \sum_{c'\neq c} L_{c'\to v} = L_v^{\mathrm{app}} - L_{c\to v}$
Check to variable	$\tanh\frac{L_{c\to v}}{2} = \prod_{v'\neq v}\tanh\frac{L_{v'\to c}}{2}$
ϕ-form	$\lvert L_{c\to v}\rvert = \phi\big(\sum_{v'\neq v}\phi(\lvert L_{v'\to c}\rvert)\big),\ \ \phi(x) = -\ln\tanh(x/2)$
Decision	$\hat{c}_v = 1 \iff L_v^{\mathrm{app}} < 0$

Visual recap • Chapter 8



Common student mistakes

! Common mistake

- Including the target edge's own message (no extrinsic subtraction): the decoder becomes over-confident at once and performs badly.
- Forgetting the channel LLR in the variable-to-check message.
- Multiplying the signs with a 0 LLR: a zero input makes the check output zero, which is correct (no information).
- Using $\tanh(L)$ instead of $\tanh(L/2)$.

Practice problems

Problem 8.1. A check of degree 4 receives $L_{v\to c} = (+2.0, -1.0, +3.0, +0.5)$. Compute the exact message to the last variable.

Solution $\tanh(1.0)\tanh(-0.5)\tanh(1.5) = 0.7616\cdot(-0.4621)\cdot 0.9051 = -0.3186$; $L = 2\tanh^{-1}(-0.3186) = -0.660$.

Problem 8.2. Same check: compute the message to the second variable using the ϕ form.

Solution Others: $2.0, 3.0, 0.5$. $\phi(2.0) = 0.2723$, $\phi(3.0) = 0.0997$, $\phi(0.5) = 1.4068$; sum $= 1.7788$; $\phi(1.7788) = 0.3409$. Signs: $+\ +\ +$, so $L = +0.341$. (Check with the tanh rule: $2\tanh^{-1}(0.7616\cdot 0.9051\cdot 0.2449) = 0.341$.)

Problem 8.3. A degree-3 variable node has $L^{\text{ch}} = -0.5$ and incoming $(+1.2, +0.4, -0.3)$. Find L^{app}, the decision and the message to the first check.

Solution $L^{\text{app}} = -0.5 + 1.3 = 0.8 \Rightarrow \hat{c} = 0$. To check 1: $0.8 - 1.2 = -0.4$.

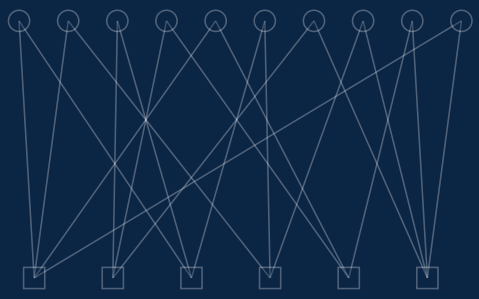
Self-assessment quiz

1. Why do the LLRs of independent observations add?
2. Why is ϕ its own inverse useful?
3. On which graphs is BP exact, and why?
4. What does a zero syndrome after one iteration tell us?

Answers: 1) Likelihoods multiply, logarithms add. 2) The same table/function is used twice. 3) Trees, because all incoming messages are independent. 4) The decisions form a codeword; we stop (it is very likely, but not certain, the sent one).

CHAPTER

9



Min-Sum and Layered Decoding

IN THIS CHAPTER YOU WILL LEARN

- the min-sum approximation and why it over-estimates;
- normalised and offset min-sum corrections;
- how to choose α and β, with real data;
- compressed check-node messages for hardware;
- layered (turbo-decoding) scheduling, step by step;
- why layered decoding converges about twice as fast;
- complexity and fixed-point notes.

9.1 The min-sum approximation

In the ϕ-form of the check rule, $\phi(x)$ is very large for small x. The sum $\sum \phi(|L_{v'\to c}|)$ is therefore dominated by the *smallest* input magnitude, and $\phi(\phi(x_{\min})) = x_{\min}$. Keeping only that term gives

$$L_{c\to v} \approx \Big(\prod_{v'\neq v} \operatorname{sign} L_{v'\to c}\Big) \cdot \min_{v'\neq v} |L_{v'\to c}| \qquad \text{(min-sum, MS)}.$$

Intuition

A check is only as reliable as its weakest bit. Min-sum takes this sentence literally: the output magnitude is the weakest input magnitude. It ignores that *several* weak inputs make the output even weaker, so it is always *too confident*: $|L^{\mathrm{MS}}_{c\to v}| \geq |L^{\mathrm{SPA}}_{c\to v}|$.

9.1.1 Only two magnitudes per check

For each check we need, for each edge, the minimum over the *other* edges. This is the smallest magnitude m_1 for every edge except the one that holds m_1, which receives the second smallest m_2. A check of degree d_c therefore needs to store only m_1, m_2, the index of m_1 and d_c sign bits — the **compressed message** format of hardware decoders. (If two inputs share the minimum, $m_2 = m_1$.)

Fig. 9.1 shows the check-node unit of a min-sum decoder built on this idea.

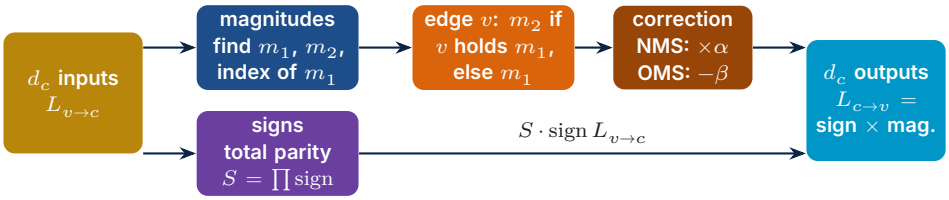


Figure 9.1. Block diagram. Min-sum check-node unit with the compressed message $(m_1, m_2, \text{index}, \text{signs})$.

Worked Example 9.1 • Min-sum on the toy code

Same input as Chapter 8: $\mathbf{L}^{\text{ch}} = (3.6, 4.8, -3.2, -1.2, -4.4, -2.8)$. Check $c_0 = \{v_0, v_1, v_3\}$ has input magnitudes $(3.6, 4.8, 1.2)$: $m_1 = 1.2$ (at v_3), $m_2 = 3.6$. Signs: $(+, +, -)$, total parity negative. So $L_{c_0\to v_0} = -1.2$, $L_{c_0\to v_1} = -1.2$, $L_{c_0\to v_3} = +3.6$. All check messages of iteration 1:

edge	c_0v_0	c_0v_1	c_0v_3	c_1v_1	c_1v_2	c_1v_4	c_2v_0	c_2v_4	c_2v_5	c_3v_2	c_3v_3	c_3v_5
SPA	−1.18	−1.12	3.34	2.94	−3.89	−3.02	2.62	−2.43	−3.23	1.03	2.29	1.09
MS	−1.2	−1.2	3.6	3.2	−4.4	−3.2	2.8	−2.8	−3.6	1.2	2.8	1.2

Every MS magnitude is larger than the SPA one. The decoder still finds the codeword in one iteration ($\mathbf{L}^{\text{app}} = (5.2, 6.8, -6.4, 5.2, -10.4, -5.2)$).

Min-sum ignores the noise level

If all channel LLRs are multiplied by a positive constant, every min-sum message is multiplied by the same constant and all decisions stay the same. So min-sum does not need σ^2: the decoder can work directly on y. The sum-product decoder does need correctly scaled LLRs.

9.2 Correcting min-sum

Two simple corrections bring min-sum close to sum-product:

$$\text{Normalised (NMS):} \quad |L_{c\to v}| = \alpha \cdot \min_{v'\neq v} |L_{v'\to c}|, \qquad 0 < \alpha < 1;$$

$$\text{Offset (OMS):} \quad |L_{c\to v}| = \max\Big(\min_{v'\neq v} |L_{v'\to c}| - \beta,\, 0\Big).$$

NMS keeps the scale invariance; OMS does not (it subtracts an absolute amount, so it assumes a known LLR scale). Both add one multiplication or one subtraction per check output — almost free.

Worked Example 9.2 • Corrected messages on the toy code

With $\alpha = 0.75$: $(-0.9, -0.9, 2.7, 2.4, -3.3, -2.4, 2.1, -2.1, -2.7, 0.9, 2.1, 0.9)$, giving $\mathbf{L}^{\text{app}} = (4.8, 6.3, -5.6, 3.6, -8.9, -4.6)$. With $\beta = 0.5$: $(-0.7, -0.7, 3.1, 2.7, -3.9, -2.7, 2.3, -2.3, -3.1, 0.7, 2.3, 0.7)$, giving $(5.2, 6.8, -6.4, 4.2, -9.4, -5.2)$. Both decode correctly in one iteration.

9.2.1 Choosing α and β

We decoded the same 2000 noisy frames of the PEG$(1008, 504)$ code at $E_b/N_0 = 2.0$ dB (at most 50 flooding iterations) with different parameters:

α (NMS)	0.5	0.625	0.7	0.75	0.8	0.875	1.0 (= MS)
FER	0.902	0.120	0.034	0.0215	**0.0140**	0.0205	0.181
average iterations	47.7	19.2	12.8	11.3	10.8	11.4	20.0
β (OMS)	0 (= MS)	0.25	**0.5**	0.75	1.0		
FER	0.181	0.028	**0.021**	0.064	0.358		

For comparison, the sum-product decoder had FER 0.019 on these frames. Plain min-sum loses a lot (FER ten times higher); NMS with $\alpha \approx 0.75$–0.8 and OMS with $\beta \approx 0.5$ recover almost all of it. Too small a factor is disastrous: with $\alpha = 0.5$ the messages are so weak that the decoder hardly moves.

Professional insight

The best α depends on the degree distribution, the code rate and the number of iterations; designers find it by density evolution or by simulation, as above. Values between 0.7 and 0.85 are common. Many hardware decoders use $\alpha = 0.75$ because it is $1 - \frac{1}{4}$: one shift and one subtraction.

9.3 Layered decoding

Intuition

In flooding, a check computed early in an iteration cannot help the other checks until the next iteration. It is like a classroom where every student writes an answer, and only then are all answers shared. Layered decoding shares at once: when a group of checks (a *layer*) has been processed, the bit beliefs are updated immediately, and the next layer already uses them. Good news travels faster.

Definition 9.1 • Row-layered decoding (turbo-decoding message passing)

Partition the checks into layers whose checks share no variable. Keep L_v^{app} and the stored $L_{c\to v}$. In each iteration, for each layer in turn and for each edge (c, v) of the layer:

1. $L_{v\to c} = L_v^{\text{app}} - L_{c\to v}^{\text{old}}$ (remove the old message of this check)
2. $L_{c\to v}^{\text{new}} =$ check update (SPA, MS, NMS or OMS) of the layer's $L_{v\to c}$
3. $L_v^{\text{app}} \leftarrow L_{v\to c} + L_{c\to v}^{\text{new}}$ (update the belief at once)

For a QC code each block row of Z checks is a natural layer: a circulant has one 1 per column, so the Z checks of a block row never share a variable (when the row has at most one non-zero block per block-column). 5G NR decoders are layered decoders working on one block row (or a few) at a time. For other codes the library builds layers by a greedy colouring of the checks (`make_layers`); the PEG$(1008, 504)$ code gets 8 layers.

Fig. 9.2 shows the data path of a layered decoder. One memory holds the beliefs L_v^{app}, another the check messages; one layer is processed per step, and for a QC code the Z checks of a block row

are processed in parallel through cyclic shifters.

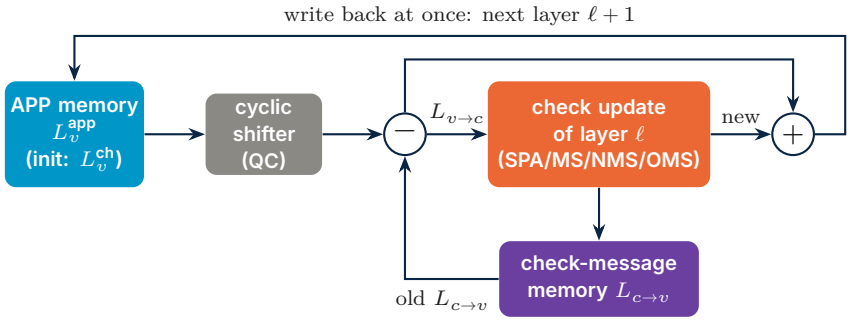


Figure 9.2. Block diagram. Row-layered decoder: subtract the old message, update the layer, add the new message and write the belief back before the next layer starts.

Worked Example 9.3 • Layered sum-product on the toy code

Each of the four checks is one layer (they all share variables). Start: $\mathbf{L}^{\text{app}} = \mathbf{L}^{\text{ch}}$.

layer	inputs $L_{v\to c}$	new $L_{c\to v}$	$\mathbf{L}^{\text{app}}$ after the layer
c_0: v_0, v_1, v_3	3.60, 4.80, −1.20	−1.18, −1.12, 3.34	(2.42, 3.68, −3.20, **2.14**, −4.40, −2.80)
c_1: v_1, v_2, v_4	3.68, −3.20, −4.40	2.94, −3.28, −2.72	(2.42, 6.62, −6.48, 2.14, −7.12, −2.80)
c_2: v_0, v_4, v_5	2.42, −7.12, −2.80	2.79, −1.91, −2.42	(5.21, 6.62, −6.48, 2.14, −9.03, −5.22)
c_3: v_2, v_3, v_5	−6.48, 2.14, −5.22	−2.09, 4.97, −2.12	(5.21, 6.62, −8.58, 7.10, −9.03, −7.34)

Bit 3 is already corrected after the *first* layer, and layer c_1 uses the updated belief 3.68 of v_1 instead of 4.80. The final values agree with `layered_decode`.

9.3.1 Faster convergence, measured

Same PEG code, same 2000 frames per point, NMS with $\alpha = 0.75$, at most 50 iterations:

	flooding		**layered (8 layers)**	
E_b/N_0	avg. iterations	FER	avg. iterations	FER
1.5 dB	25.8	0.278	19.9	0.265
2.0 dB	11.3	0.0215	6.6	0.018
2.5 dB	6.9	0.0010	4.0	0.0005

In the region where the code works, layered decoding needs about **40% fewer iterations** (often quoted as "about twice as fast") and has a slightly lower FER for the same iteration limit. It also needs less memory: only L^{app} and the check messages are stored, not separate variable-to-check messages.

! Common mistake

Layered decoding is *sequential* between layers. The hardware gains speed from convergence but loses parallelism across layers; the parallelism inside a layer (Z checks at once) is what QC structure provides. Very large Z and few layers is the sweet spot of 5G NR.

9.4 Complexity and implementation notes

Decoder	check-node work per edge	performance (vs SPA)
SPA (ϕ form)	two ϕ look-ups, add, subtract	reference
Min-sum	compare (find m_1, m_2), sign XOR	0.3–0.5 dB worse
NMS / OMS	MS + one multiply or subtract	within about 0.1 dB
Layered (any rule)	same per edge, fewer iterations	same or slightly better

- **Fixed point.** Hardware stores LLRs with a few bits (typically 5–8) and saturates. Min-sum variants are robust to coarse quantisation because they only compare values.
- **Early termination.** The syndrome test costs one XOR per edge; most frames at high E_b/N_0 stop after 2–5 iterations (Chapter 13).
- **Throughput** $\approx$ (bits per codeword) / (iterations $\times$ time per iteration): halving the iterations almost doubles the throughput.

9.5 In Python

ldpc.py • lines 441--487

```python
# ------------------------------------------------------------------ 9. layered decoding
def make_layers(code, Z=None):
    """Split the checks into layers whose checks share no variable. For a QC code
    pass Z: every block row of Z checks is one layer."""
    if Z is not None:
        groups = [np.arange(r, r + Z) for r in range(0, code.m, Z)]
    else:                                                # greedy colouring of checks
        used, groups = [], []
        for c in range(code.m):
            vs = set(code.vn[code.cstart[c]:code.cstart[c] + code.dc[c]])
            for g, u in zip(groups, used):
                if not (vs & u):
                    g.append(c); u |= vs; break
            else:
                groups.append([c]); used.append(set(vs))
        groups = [np.array(g) for g in groups]
    layers = []
    for g in groups:
        e = np.concatenate([np.arange(code.cstart[c], code.cstart[c] + code.dc[c]) for c in g])
        layers.append((e, np.r_[0, np.cumsum(code.dc[g])[:-1]], np.repeat(np.arange(len(g)), code.dc[g])))
    code.layers = layers
    return layers

```

```python

def layered_decode(code, llr, max_iter=50, rule="nms", alpha=0.75, beta=0.5):
    """Row-layered (turbo-decoding message passing) decoder: the a-posteriori
    LLRs are updated after every layer, so new information is used at once."""
    if code.layers is None:
        make_layers(code)
    app = np.atleast_2d(llr).astype(float).copy()
    B = app.shape[0]
    iters = np.full(B, max_iter)
    act = np.arange(B)
    a, c2v = app.copy(), np.zeros((B, code.E))          # working copies of active frames
    for it in range(1, max_iter + 1):
        for e, st, sg in code.layers:
            v2c = a[:, code.vn[e]] - c2v[:, e]              # remove the old message
            new = check_update(code, v2c, rule, alpha, beta, starts=st, seg=sg)
            a[:, code.vn[e]] = v2c + new                    # update APP immediately
            c2v[:, e] = new
        ok = ~code.syndrome((a < 0).astype(np.uint8)).any(axis=1)
        app[act] = a
        iters[act[ok]] = it
        act, a, c2v = act[~ok], a[~ok], c2v[~ok]
        if act.size == 0:
            break
    return (app < 0).astype(np.uint8), iters, app
```

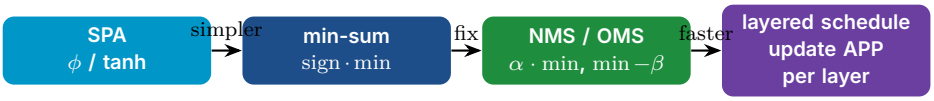
- `make_layers` either takes block rows of Z checks (QC codes) or builds layers greedily.
- In `layered_decode` the arrays `a` (beliefs) and `c2v` hold only the frames still decoding. The three lines inside the layer loop are exactly steps 1–3 of the definition.
- The check update is the same `check_update` as in flooding; `rule="nms"` with `alpha=0.75` is the default.

9.6 Chapter summary

Key concepts

- Min-sum: sign product $\times$ minimum magnitude; over-confident; scale-invariant.
- NMS ($\times\alpha$) and OMS ($-\beta$) remove most of the min-sum loss.
- Compressed check messages: m_1, m_2, index of m_1, signs.
- Layered decoding updates beliefs after each layer: about 40% fewer iterations.
- 5G NR decoders: layered NMS/OMS over block rows of a QC code.

Visual recap • Chapter 9



Common student mistakes

! Common mistake

- Taking the minimum over *all* inputs, including the target edge.
- Applying the offset β and allowing negative magnitudes (always clip at 0).
- In layered decoding, forgetting to subtract the check's *old* message before computing the new one.
- Putting two checks that share a variable into the same layer.

Practice problems

Problem 9.1. Inputs to a degree-5 check: $(+2.5, -0.8, +1.6, -3.0, +0.9)$. Find all five min-sum outputs.

Solution Magnitudes: $m_1 = 0.8$ (edge 2), $m_2 = 0.9$. Two negative inputs: total parity even $(+)$. Outputs: edge 1: $+\cdot$ sign of own $(+) \Rightarrow +0.8$; edge 2: -0.9; edge 3: $+0.8$; edge 4: -0.8; edge 5: $+0.8$. (Output sign = total parity $\times$ own sign.)

Problem 9.2. Apply NMS with $\alpha = 0.75$ and OMS with $\beta = 0.5$ to the outputs of the previous problem.

Solution NMS: $(0.6, -0.675, 0.6, -0.6, 0.6)$. OMS: $(0.3, -0.4, 0.3, -0.3, 0.3)$.

Problem 9.3. A QC code has a 6×24 base matrix with $Z = 96$. How many layers and how many checks per layer does a block-row layered decoder use?

Solution 6 layers of 96 checks each.

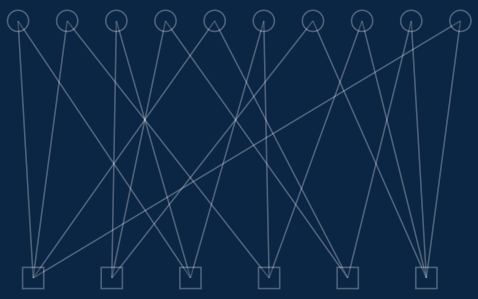
Self-assessment quiz

1. Why is min-sum always over-confident?
2. Why can a min-sum decoder work without knowing σ^2?
3. What is stored for each check in a compressed min-sum decoder?
4. Why does layered decoding converge faster?

Answers: 1) It ignores all inputs except the weakest one, but every extra input reduces the reliability. 2) Its decisions do not change under positive scaling. 3) m_1, m_2, the index of m_1 and the signs. 4) New beliefs are used within the same iteration.

CHAPTER

10



Density Evolution and EXIT Charts

IN THIS CHAPTER YOU WILL LEARN

- what "threshold" means for an LDPC ensemble;
- density evolution on the BEC, derived and computed;
- thresholds of regular codes and their gap to capacity;
- the stability condition for degree-2 nodes;
- designing irregular codes by linear programming;
- density evolution on the AWGN channel: Monte-Carlo and Gaussian approximation;
- EXIT charts and the decoding tunnel;
- error floors: stopping sets and trapping sets.

10.1 The idea: follow the messages, not the codewords

Intuition

Simulating one code tells us about that code. Richardson and Urbanke asked a bigger question: for a *randomly chosen* code with degree distributions (λ, ρ) and very large n, what happens to the messages, iteration by iteration? Three facts make this tractable:

1. **Concentration:** for large n almost all codes of the ensemble behave like the average.
2. **Tree-like neighbourhoods:** for large n the graph around a node has no short cycles, so the incoming messages are independent.
3. **Symmetry:** for symmetric channels and decoders the error probability does not depend on the codeword, so we may assume the all-zero codeword.

Then we only need to track the *probability distribution* ("density") of one message — this is **density evolution** (DE).

Definition 10.1 • Threshold

The **threshold** of an ensemble is the worst channel parameter (largest erasure probability ϵ^*, largest noise σ^*, smallest E_b/N_0) for which density evolution drives the message error probability to zero as the number of iterations grows. Long codes of the ensemble decode reliably below the threshold and fail above it.

10.2 Density evolution on the BEC

On the BEC a message is either correct or an erasure, so its "density" is a single number: the erasure probability. Let x_ℓ be the probability that a variable-to-check message is an erasure in iteration ℓ.

- **Check node of degree d:** the output is known only if all $d-1$ other inputs are known: $P(\text{erased}) = 1-(1-x_\ell)^{d-1}$. Averaged over the edge degrees: $y_\ell = 1-\rho(1-x_\ell)$.
- **Variable node of degree d:** the output is an erasure only if the channel bit *and* all $d-1$ other incoming messages are erased: $\epsilon\, y_\ell^{d-1}$. Averaged: $\epsilon\,\lambda(y_\ell)$.

$$x_0 = \epsilon, \qquad x_{\ell+1} = f(x_\ell) = \epsilon\,\lambda(1-\rho(1-x_\ell)), \qquad \epsilon^* = \sup\{\epsilon :\ f(x) < x \text{ for all } x \in (0,\epsilon]\}.$$

Worked Example 10.1 • The $(3,6)$ ensemble at $\epsilon = 0.40$

$\lambda(x) = x^2$, $\rho(x) = x^5$. First step: $x_1 = 0.4\,(1-(1-0.4)^5)^2 = 0.4\,(1-0.07776)^2 = 0.3402$. Continuing (library `de_bec`):

$$x_\ell = 0.400,\ 0.340,\ 0.306,\ 0.282,\ 0.262,\ 0.244,\ 0.227,\ 0.209,\ 0.191,\ 0.171,\ 0.148,\ \ldots$$

and $x_\ell < 10^{-20}$ after 19 iterations: decoding succeeds. At $\epsilon = 0.45$ the sequence stops at a *fixed point* $x \approx 0.355$: a large fraction of bits stays erased forever.

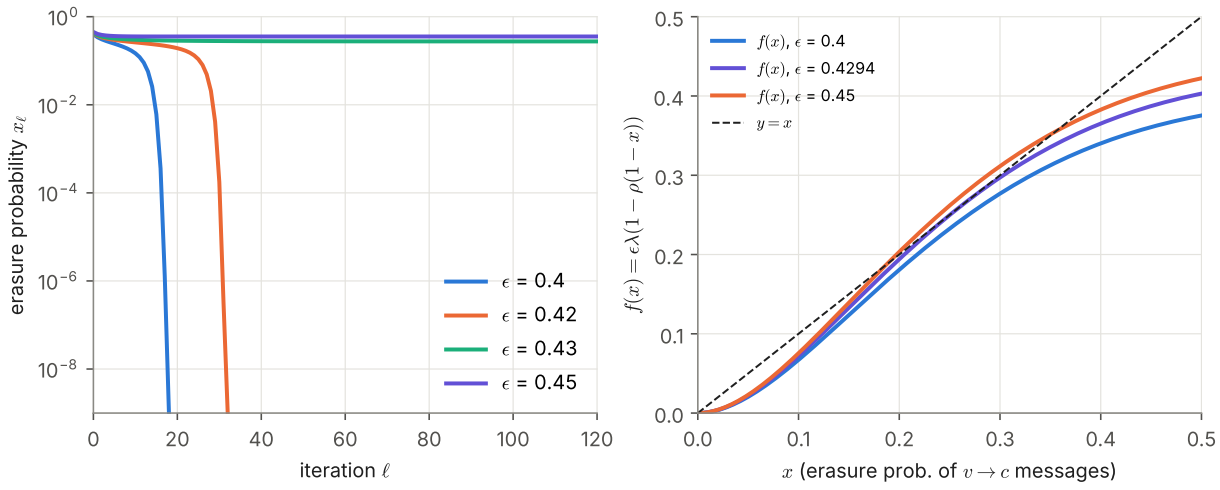


Figure 10.1. Left: erasure probability per iteration for the $(3,6)$ ensemble. Right: $f(x)$ against the line $y = x$. Below the threshold the curve stays under the line and the iteration slides to 0; at $\epsilon^* = 0.4294$ it touches the line.

10.2.1 Thresholds of regular ensembles

(d_v, d_c)	rate	BEC threshold ϵ^*	capacity limit $1-R$	gap	ratio $\epsilon^*/(1-R)$
$(3,4)$	0.250	0.6474	0.750	0.103	86.3%
$(4,6)$	0.333	0.5061	0.667	0.161	75.9%
$(3,5)$	0.400	0.5176	0.600	0.082	86.3%
$(3,6)$	0.500	**0.4294**	0.500	0.071	85.9%
$(4,8)$	0.500	0.3834	0.500	0.117	76.7%
$(5,10)$	0.500	0.3415	0.500	0.159	68.3%
$(3,9)$	0.667	0.2828	0.333	0.051	84.8%
$(3,12)$	0.750	0.2105	0.250	0.040	84.2%

For a fixed rate, $d_v = 3$ is the best regular choice: higher degrees make each check weaker (more inputs that must all be known). Regular codes stop about 14% short of capacity — irregular codes close this gap.

10.2.2 The stability condition

Near $x = 0$, $f(x) \approx \epsilon\,\lambda'(0)\,\rho'(1)\,x = \epsilon\lambda_2\rho'(1)\,x$. If this slope is larger than 1, small erasure probabilities grow again and decoding cannot finish. Hence

$$\epsilon\,\lambda_2\,\rho'(1) < 1 \qquad \text{(stability on the BEC)},$$
$$\lambda_2\,\rho'(1) < e^{1/(2\sigma^2)} \qquad \text{(stability on the BI-AWGN channel)}.$$

Degree-2 variable nodes are cheap and useful, but there must not be too many of them. Example: the $(2,4)$ ensemble has $\lambda_2 = 1$, $\rho'(1) = 3$, so its threshold can be at most $1/3$ (and it equals $1/3$).

10.3 Designing irregular codes

With ρ fixed, the condition $\epsilon\lambda(1-\rho(1-x)) < x$ is *linear* in the coefficients λ_i. So the best λ for a given rate and ϵ is a **linear program**: maximise $\sum_i \lambda_i/i$ (the rate) subject to $\sum_i \lambda_i = 1$, $\lambda_i \geq 0$, the DE condition on a grid of x, and stability. A bisection on ϵ then finds the largest threshold for rate $1/2$. Our script `design_bec.py` (scipy `linprog`) gave:

d_c	max. d_v	optimised $\lambda(x)$ (edge perspective)	threshold ϵ^*
6	3	x^2 (regular)	0.4294
6	6	$0.4188x + 0.1624x^2 + 0.4188x^5$	0.4757
7	10	$0.3224x + 0.2432x^2 + 0.4344x^9$	0.4855
9	30	$0.2552x + 0.1199x^2 + 0.1590x^4 + 0.0549x^5 + 0.4110x^{29}$	0.4876

(Each threshold was re-checked with `threshold_bec`.) Allowing higher degrees moves the threshold toward the capacity limit 0.5. Notice the pattern: many degree-2 nodes (as many as stability allows), some degree-3 nodes and a group of high-degree nodes — the "strong nodes" of Chapter 4. Sequences of such ensembles are known to approach capacity on the BEC arbitrarily closely.

10.4 Density evolution on the AWGN channel

On the AWGN channel a message is a real LLR, and its density is a whole function. Under the all-zero assumption the channel LLR is Gaussian, $L^{\text{ch}} \sim \mathcal{N}(2/\sigma^2,\, 4/\sigma^2)$. The variable-node rule adds independent LLRs (densities *convolve*); the check-node rule is a convolution in the (sign, ϕ) domain. Exact DE evaluates these convolutions numerically on quantised densities.

10.4.1 Monte-Carlo density evolution

A simple and honest alternative is to represent the density by a large *population* of samples: draw 10^5–10^6 channel LLRs, apply the check rule to random groups of $d_c - 1$ samples and the variable rule to random groups of $d_v - 1$ check samples plus a fresh channel sample, and repeat. `de_mc` does this with 2×10^5 samples.

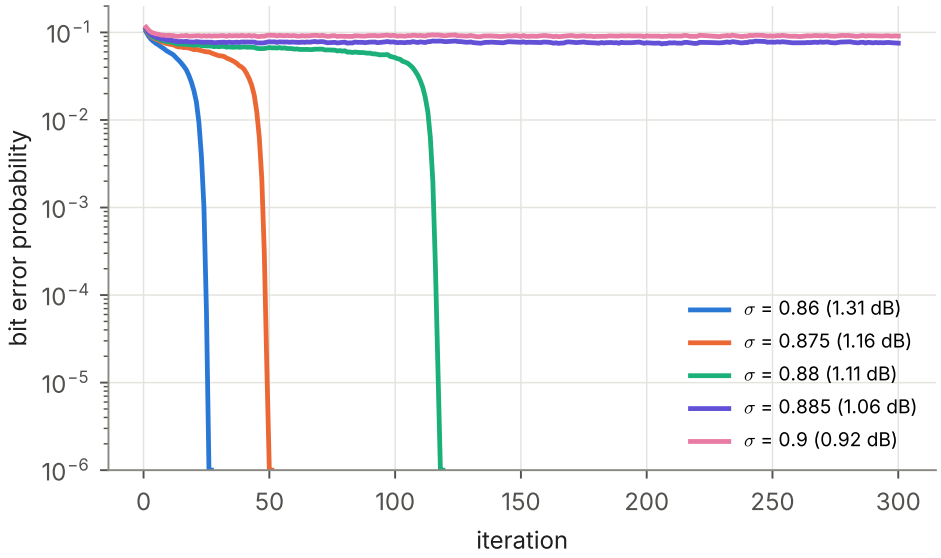


Figure 10.2. Monte-Carlo density evolution for the $(3, 6)$ ensemble with sum-product decoding: bit error probability of the variable-to-check messages per iteration.

Worked Example 10.2 • Locating the $(3, 6)$ AWGN threshold

In Fig. 10.2 the error probability goes to zero for $\sigma = 0.880$ ($E_b/N_0 = 1.11$ dB) after 119 iterations, but stays near 0.075 for $\sigma = 0.885$ (1.06 dB). So $\sigma^* \in (0.880, 0.885)$, in agreement with the value $\sigma^* \approx 0.881$ (1.10 dB) reported by Richardson and Urbanke. Near the threshold the number of iterations grows very quickly: real decoders with 50 iterations need a little more E_b/N_0.

10.4.2 Gaussian approximation (GA)

Chung, Richardson and Urbanke (2001) observed that the messages are close to Gaussian and *consistent* (variance = 2× mean). Then one number — the mean — describes a density. With $\varphi(m) = 1 - \mathbb{E}[\tanh(u/2)]$, $u \sim \mathcal{N}(m, 2m)$:

$$m_v^{(\ell)} = \frac{2}{\sigma^2} + (d_v - 1)\, m_u^{(\ell-1)}, \qquad m_u^{(\ell)} = \varphi^{-1}\Big(1 - \big[1 - \varphi(m_v^{(\ell)})\big]^{d_c - 1}\Big).$$

The threshold is the largest σ for which $m_u \to \infty$. (Irregular codes average these expressions over λ and ρ.) `de_ga` evaluates φ by Gauss–Hermite quadrature.

ensemble	rate	GA threshold σ^*	GA threshold E_b/N_0	BPSK Shannon limit
$(3, 6)$	$1/2$	0.8719	1.19 dB	0.19 dB
$(4, 8)$	$1/2$	0.8295	1.62 dB	0.19 dB
$(3, 4)$	$1/4$	1.2491	1.08 dB	−0.79 dB

For $(3, 6)$ the GA is about 0.08 dB pessimistic compared with the Monte-Carlo result: a good tool for design, where only *differences* between ensembles matter. The $(3, 6)$ code is still 0.9 dB from the Shannon limit; well-designed irregular codes come within a few hundredths of a dB for very long blocks.

10.5 EXIT charts

An **EXIT chart** (extrinsic information transfer, ten Brink) tracks the *mutual information* $I \in [0, 1]$ between a message and its bit instead of the full density. Each node type becomes a curve:

- variable-node decoder (VND): $I_{E,V}$ as a function of the a-priori $I_{A,V}$ coming from the checks;
- check-node decoder (CND): $I_{E,C}$ as a function of $I_{A,C} = I_{E,V}$.

Plot the VND curve and the *inverse* CND curve in the same box. Decoding is a staircase between them; it reaches $I = 1$ (success) only if there is an open **tunnel**.

$$\text{BEC:}\quad I_{E,V} = 1 - \epsilon\,(1 - I_A)^{d_v - 1}, \qquad I_{E,C} = I_A^{\,d_c - 1};$$

$$\text{AWGN:}\quad I_{E,V} = J\Big(\sqrt{(d_v - 1)\, J^{-1}(I_A)^2 + \sigma_{\text{ch}}^2}\Big), \quad I_{E,C} \approx 1 - J\Big(\sqrt{d_c - 1}\; J^{-1}(1 - I_A)\Big),$$

with $\sigma_{\text{ch}}^2 = 4/\sigma^2 = 8R\,E_b/N_0$ and $J(s) = 1 - \mathbb{E}\left[\log_2(1 + e^{-u})\right]$, $u \sim \mathcal{N}(s^2/2, s^2)$.

Professional insight

EXIT charts turn code design into *curve fitting*: choose degree distributions so that the VND curve lies just above the inverse CND curve everywhere. On the BEC, the area under each curve is tied to the code rate (the *area theorem*), which explains why a narrow tunnel means a rate close to capacity and a slow, many-iteration decoder.

Fig. 10.4 shows the model behind the EXIT chart: the decoder is seen as two component decoders that pass mutual information back and forth, exactly like the two curves in Fig. 10.3.

10.6 What density evolution does not see: error floors

DE describes infinitely long, cycle-free codes. Real codes of finite length show two effects:

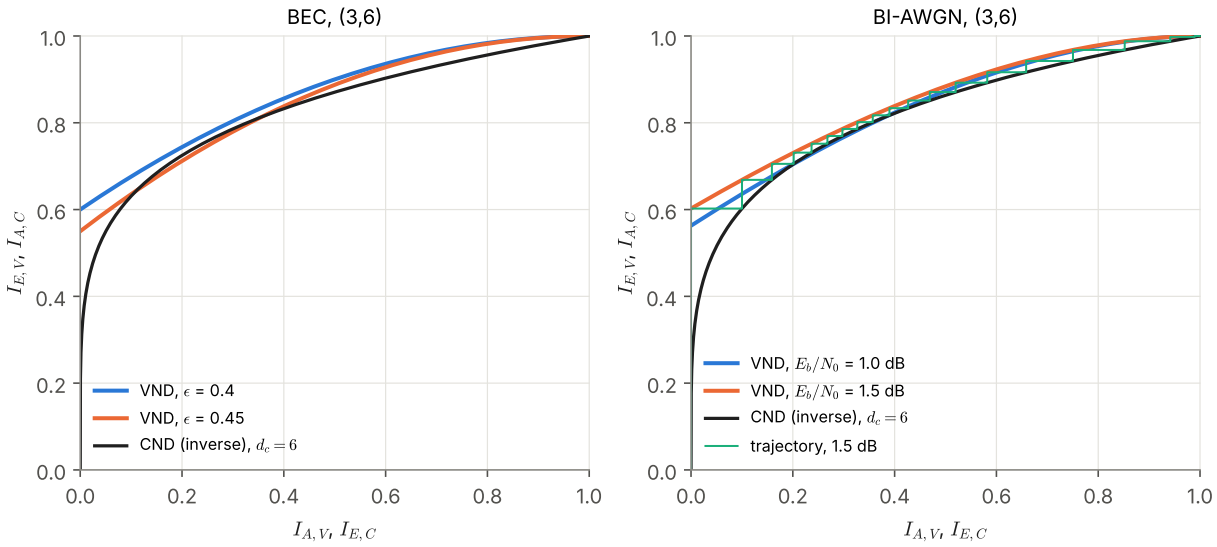


Figure 10.3. EXIT charts of the $(3, 6)$ ensemble. Left (BEC): the tunnel is open at $\epsilon = 0.40$ and closed at 0.45. Right (AWGN): at 1.5 dB a narrow tunnel is open; the green staircase is the predicted decoding trajectory (19 steps).

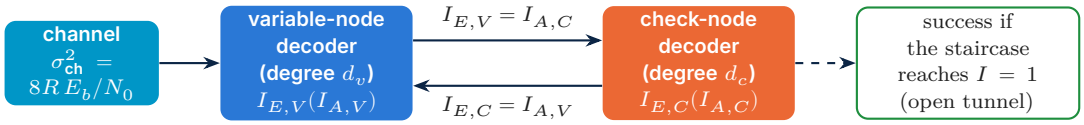


Figure 10.4. **Block diagram.** The iterative decoder as two component decoders exchanging extrinsic information.

- **Finite-length waterfall.** The curve is less steep and shifted to the right of the threshold; the shift shrinks roughly like $1/\sqrt{n}$ (we saw this on the BEC in Chapter 7, and we will see it for 5G codes in Chapter 13).
- **Error floor.** At high SNR, rare noise patterns concentrated on small harmful sub-graphs defeat the decoder.

Definition 10.2 • Trapping set

An (a, b) **trapping set** is a set of a variable nodes whose induced sub-graph has exactly b check nodes of odd degree. If those a bits are wrong, only b checks complain, and if b is small the decoder may never leave the wrong state. Stopping sets play this role on the BEC; codewords are the special case $b = 0$.

How designers lower the floor

Avoid small cycles that combine into small trapping sets (PEG, careful QC shifts); avoid too many degree-2 nodes in cycles; use an outer code (BCH in DVB-S2, CRC in 5G) to catch residual errors; use post-processing or decoder tweaks (e.g. offsets, quantisation) that break trapping-set oscillations.

10.7 In Python

The DE and EXIT functions are in lines 492–593 of `ldpc.py`:

```
from ldpc import de_bec, threshold_bec, de_ga, threshold_ga, de_mc, J, J_inv
print(threshold_bec({3: 1.0}, {6: 1.0}))       # 0.4294
print(threshold_ga({3: 1.0}, {6: 1.0}))        # 0.8719 (sigma)
pe = de_mc(3, 6, 0.88, iters=300, pop=200000)   # error probability per iteration
```

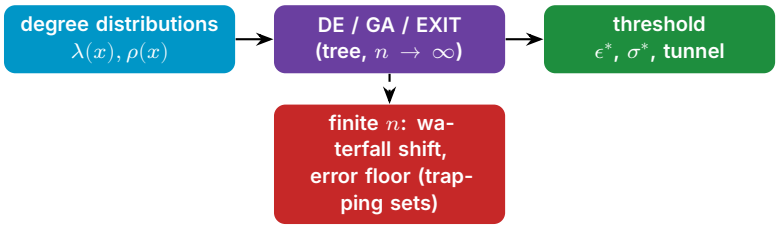
Degree distributions are dictionaries {degree: fraction of edges}.

10.8 Chapter summary

Key concepts

- DE tracks the message density of an infinitely long, tree-like ensemble; it predicts a sharp threshold.
- BEC: $x_{\ell+1} = \epsilon\lambda(1-\rho(1-x_\ell))$; $(3,6)$ threshold 0.4294.
- Stability: $\epsilon\lambda_2\rho'(1) < 1$ limits the number of degree-2 nodes.
- Irregular λ found by linear programming approaches capacity (0.4876 with $d_{v,\max} = 30$).
- AWGN: Monte-Carlo DE gives $(3,6)$ threshold ≈ 1.1 dB; GA gives 1.19 dB.
- EXIT charts: decoding succeeds iff the tunnel between VND and inverse CND curves is open.
- Error floors come from small trapping/stopping sets, which DE does not capture.

Visual recap • Chapter 10



Common student mistakes

! Common mistake

- Using node-perspective fractions in the DE recursion: DE needs the *edge* perspective λ, ρ.
- Thinking that the threshold is the performance of a particular finite code: it is a limit for $n \to \infty$.
- Stopping the DE iteration too early near the threshold, where convergence is extremely slow (hundreds of iterations).
- Expecting DE to predict the error floor.

Practice problems

Problem 10.1. Compute x_1 and x_2 for the $(3, 6)$ ensemble at $\epsilon = 0.45$.

Solution $x_1 = 0.45\,(1 - 0.55^5)^2 = 0.45\,(0.94967)^2 = 0.4058$. $x_2 = 0.45\,(1 - 0.5942^5)^2 = 0.45\,(1 - 0.0741)^2 = 0.3858$. The library gives the same values.

Problem 10.2. Find the largest λ_2 allowed by stability for $\rho(x) = x^6$ and target $\epsilon^* = 0.48$.

Solution $\rho'(1) = 6$; $\lambda_2 < 1/(0.48 \cdot 6) = 0.347$.

Problem 10.3. Show that the design rate of $\lambda(x) = 0.3224x + 0.2432x^2 + 0.4344x^9$, $\rho(x) = x^6$ is $1/2$.

Solution $\int \lambda = 0.1612 + 0.0811 + 0.0434 = 0.2857$; $\int \rho = 1/7 = 0.1429$; $R = 1 - 0.1429/0.2857 = 0.500$.

Problem 10.4. On the BEC EXIT chart of $(3, 6)$, at which ϵ does the VND curve first touch the inverse CND curve?

Solution At the threshold $\epsilon^* = 0.4294$; for larger ϵ the tunnel is closed.

Self-assessment quiz

1. Why may DE assume that the all-zero codeword was sent?
2. What happens to x_ℓ above the BEC threshold?
3. Why does the GA need only one parameter per density?
4. Name two finite-length effects not predicted by DE.

Answers: 1) Channel and decoder are symmetric, so the error probability is the same for every codeword. 2) It converges to a non-zero fixed point. 3) Consistent Gaussian densities have variance = 2×mean. 4) The shifted, less steep waterfall and the error floor.

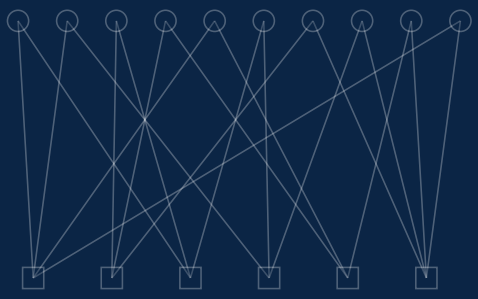
PART IV

Professional Level: 5G and Practical Implementation

How 3GPP NR uses LDPC codes, a complete tested Python implementation, and engineering analysis of real simulation results, including a fair comparison with polar codes.

CHAPTER

11



LDPC Codes in 5G NR

IN THIS CHAPTER YOU WILL LEARN

- which NR channels use LDPC codes;
- the transport-block processing chain of TS 38.212;
- the two base graphs and when each is used;
- lifting sizes, set index and shift values;
- code-block segmentation and filler bits;
- puncturing, circular-buffer rate matching and redundancy versions;
- HARQ with incremental redundancy;
- worked parameter examples computed with the library.

11.1 Where NR uses LDPC codes

In 5G NR the transport channels that carry user data — **DL-SCH** (downlink shared channel, sent on PDSCH), **UL-SCH** (uplink shared channel, sent on PUSCH) and **PCH** (paging) — are protected by LDPC codes. Control information (DCI, UCI with more than 11 bits) and the broadcast channel use polar codes [Part One, Ch. 10]. The split follows the job: data blocks are large (up to 8448 bits per code block, many code blocks per transport block) and need very high throughput, HARQ and fine rate adaptation; control blocks are short and need the best short-length performance.

11.2 The processing chain

1. **Transport-block CRC.** A payload bits get a CRC of 24 bits (CRC24A) if $A > 3824$, otherwise 16 bits (CRC16): $B = A + L$ bits.
2. **Base graph selection** (next section) from A and the target code rate R.
3. **Segmentation.** If B is larger than the maximum code-block size K_{cb} (8448 for BG1, 3840 for BG2), the block is split into $C = \lceil B/(K_{cb} - 24) \rceil$ code blocks, each with its own 24-bit CRC (CRC24B). Each code block then carries K' bits (data plus CRC).

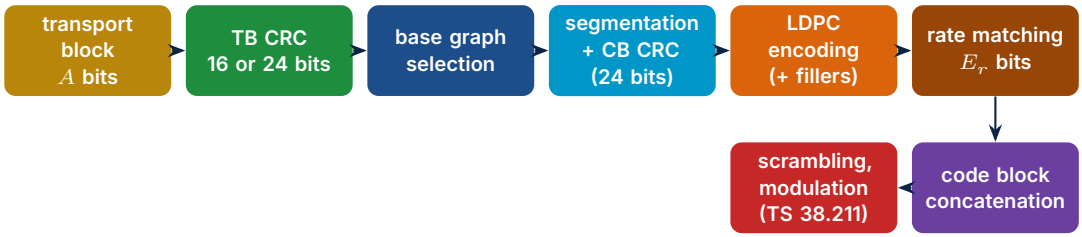


Figure 11.1. LDPC-coded transport-block processing in TS 38.212 (Section 7.2), simplified.

4. **LDPC encoding** of each code block, after filling it up to $K = 22Z$ (BG1) or $K = 10Z$ (BG2) bits with known *filler* bits.
5. **Rate matching**: bit selection from a circular buffer and bit interleaving; E_r bits per code block.
6. **Concatenation** of the code blocks, then scrambling and modulation (TS 38.211).

11.3 The two base graphs

	Base graph 1 (BG1)	**Base graph 2 (BG2)**
Size $m_b \times n_b$	46×68	42×52
Non-zero entries (edges of the base graph)	316	197
Systematic columns k_b	22	10 (only 6, 8 or 9 used for small K)
Core (dual-diagonal)	rows 0–3, columns 22–25	rows 0–3, columns 10–13
Extension (degree-1 parity)	42 columns	38 columns
Mother code rate (after puncturing $2Z$)	$22/66 = 1/3$	$10/50 = 1/5$
Highest rate with the core only	$22/24 \approx 0.92$	$10/12 \approx 0.83$
Maximum K per code block	8448	3840
Used for	large blocks, high rates	small blocks, low rates

Definition 11.1 • Base graph selection (TS 38.212, 7.2.2)

Use **BG2** if $A \leq 292$, or if $A \leq 3824$ and $R \leq 0.67$, or if $R \leq 0.25$; otherwise use **BG1**.

Puncturing of the first two columns

The first $2Z$ systematic bits (base columns 0 and 1, the two highest-degree columns) are encoded but *never transmitted*. The decoder starts them with LLR 0 and they are recovered through their many checks. This puncturing of high-degree nodes improves the threshold of the protograph — a design trick found by density evolution.

11.4 Lifting

Definition 11.2 • Lifting sizes and shift values

The lifting size Z is taken from 51 values $Z = a \cdot 2^j \leq 384$, $a \in \{2, 3, 5, 7, 9, 11, 13, 15\}$, grouped into eight sets by a (set index $i_{LS} = 0, \ldots, 7$). For each base graph the tables of TS 38.212 give one value V_{ij} per non-zero entry and per set index; the actual circulant shift is

$$P_{ij} = V_{ij} \bmod Z.$$

Z is the smallest value with $k_b \cdot Z \geq K'$, where $k_b = 22$ for BG1 and, for BG2, $k_b = 10$ if $K' > 640$, 9 if $K' > 560$, 8 if $K' > 192$, else 6.

i_{LS}	lifting sizes Z
0	2, 4, 8, 16, 32, 64, 128, 256
1	3, 6, 12, 24, 48, 96, 192, 384
2	5, 10, 20, 40, 80, 160, 320
3	7, 14, 28, 56, 112, 224
4	9, 18, 36, 72, 144, 288
5	11, 22, 44, 88, 176, 352
6	13, 26, 52, 104, 208
7	15, 30, 60, 120, 240

One set of tables per i_{LS} (instead of one per Z) keeps the specification compact; the reduction modulo Z makes the same V_{ij} usable for all Z of the set.

Fig. 11.2 collects the parameter rules of the last two sections in one flow chart: from the block size and the code rate to the shift values that the encoder and decoder use.

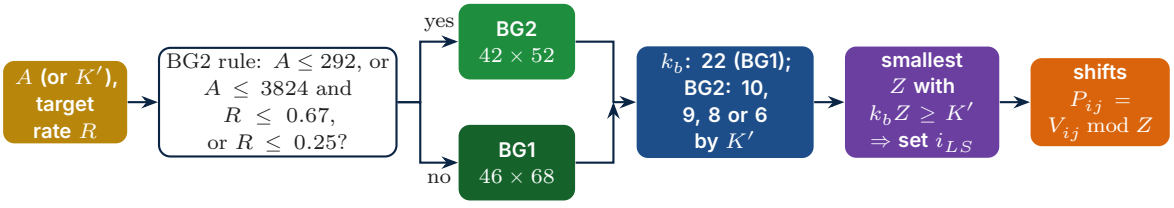


Figure 11.2. Block diagram. Choosing the base graph, the lifting size and the shift values in 5G NR.

11.5 Rate matching and redundancy versions

After encoding, the codeword without the $2Z$ punctured bits ($N = 66Z$ or $50Z$ bits) is written into a **circular buffer** of length N_{cb} ($N_{cb} = N$, or smaller when the receiver's soft buffer is limited). The transmitter reads E bits starting at position k_0 and wraps around if needed; filler bits are skipped. Four **redundancy versions** (RV) define four starting points:

	rv = 0	rv = 1	rv = 2	rv = 3
BG1: k_0	0	$\lfloor \frac{17N_{cb}}{66Z} \rfloor Z$	$\lfloor \frac{33N_{cb}}{66Z} \rfloor Z$	$\lfloor \frac{56N_{cb}}{66Z} \rfloor Z$
BG2: k_0	0	$\lfloor \frac{13N_{cb}}{50Z} \rfloor Z$	$\lfloor \frac{25N_{cb}}{50Z} \rfloor Z$	$\lfloor \frac{43N_{cb}}{50Z} \rfloor Z$

With rv = 0 the first transmission starts with the systematic bits, then the core parity, then as many extension parity bits as E allows. A low rate simply reads further into the buffer; a rate above the mother rate stops early. The selected bits are then interleaved row-by-column over the Q_m bits of each modulation symbol.

Fig. 11.3 shows the circular buffer and the four starting points for BG1 with $N_{cb} = N$.

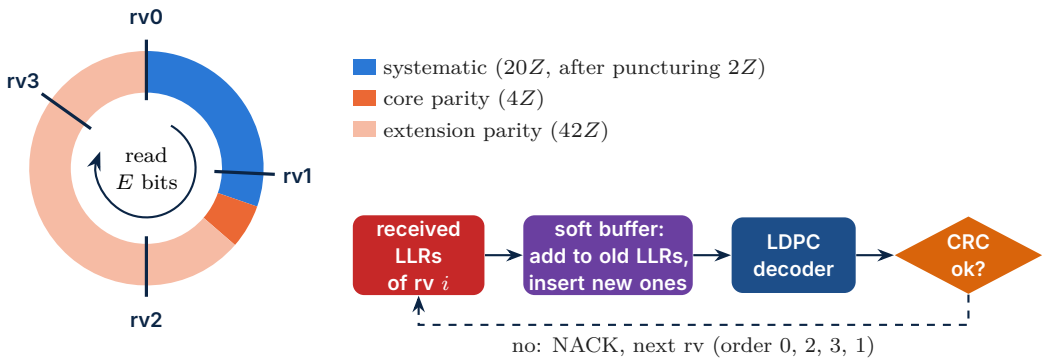


Figure 11.3. Block diagram. Left: circular buffer of a BG1 codeword with the starting points of the four redundancy versions. Right: the HARQ receiver combines the LLRs of all transmissions.

Professional insight

HARQ with incremental redundancy. If the CRC fails, the receiver keeps the soft LLRs and asks for a retransmission. The transmitter sends another RV — typically in the order $0, 2, 3, 1$ — which contains mostly *new* parity bits. The receiver adds the LLRs of bits received twice and inserts the new ones: the effective code rate drops with each transmission. The extension part of the base graph (degree-1 parity columns) makes this natural: every extra parity bit is one extra check row of a lower-rate code. This is the main reason the NR design is called "rate-compatible".

11.6 Worked parameter examples

The function `NRCode(K, E)` of the library applies the rules above for one code block (K = data plus CRC bits, E = transmitted bits, rv = 0, no bit interleaving). It keeps only the base-graph rows that touch transmitted bits, exactly as a practical decoder would.

K	E	R	BG	k_b	Z	i_{LS}	K_{enc}	fillers	base rows used	edges
120	360	0.333	2	6	20	2	200	80	14	1720
256	512	0.500	2	8	32	0	320	64	10	2144
1024	2048	0.500	2	10	104	6	1040	16	12	8008
1024	1152	0.889	1	22	48	1	1056	32	5	3792
3840	19200	0.200	2	10	384	1	3840	0	42	75 648
4096	8192	0.500	1	22	192	1	4224	128	24	40 320
8448	25344	0.333	1	22	384	1	8448	0	46	121 344

Here K is the number of data and CRC bits of the code block (called K' in TS 38.212) and $K_{\text{enc}} = 22Z$ (BG1) or $10Z$ (BG2) is the encoder input length (called K in TS 38.212); the difference is filled with filler bits. The value k_b only decides Z.

Worked Example 11.1 • $K = 1024$, $R = 1/2$, step by step

1. $K = 1024 > 292$, $K \leq 3824$ and $R = 0.5 \leq 0.67$: **BG2**.
2. $K > 640 \Rightarrow k_b = 10$. Smallest Z in the table with $10Z \geq 1024$: $Z = 104$ (set $i_{LS} = 6$, $104 = 13 \cdot 8$).
3. Encoder input $10 \cdot 104 = 1040$ bits: 1024 data/CRC bits and 16 filler bits (zeros, not transmitted).
4. Full codeword $52Z = 5408$ bits; the first $2Z = 208$ are punctured; $E = 2048$ bits are read from the buffer starting with the systematic part, so base columns up to 21 are needed and only base rows 0–11 take part in decoding.
5. The decoder gets LLR 0 for the 208 punctured bits and for untransmitted parity, and a very large positive LLR for the 16 filler bits (known zeros).

11.7 Decoding NR LDPC codes in practice

- **Algorithm:** layered normalised or offset min-sum, one block row (Z checks) per layer; the base graph is processed in row order, so the core rows (best-protected bits) come first.
- **Iterations:** a maximum of the order of 10–25 is common in hardware, with early termination by syndrome or CRC.
- **Punctured and filler bits:** start at LLR 0 and $+\infty$ (a large number), respectively.
- **HARQ combining:** add the LLRs of repeated bits before decoding (the library function `llr_in` does this when E is larger than the buffer).
- **Throughput** scales with Z: a decoder with 384 parallel check processors handles the largest lifting at full parallelism.

11.8 In Python

ldpc.py • lines 202--278

```
202 # ------------------------------------------------------------- 4. 5G NR base graphs
203 NR_LIFTING_SETS = [[2, 4, 8, 16, 32, 64, 128, 256], [3, 6, 12, 24, 48, 96, 192, 384],
204                    [5, 10, 20, 40, 80, 160, 320], [7, 14, 28, 56, 112, 224],
```

```
                    [9, 18, 36, 72, 144, 288], [11, 22, 44, 88, 176, 352],
                    [13, 26, 52, 104, 208], [15, 30, 60, 120, 240]]   # TS 38.212 Table
    5.3.2-1
DATA = Path(__file__).resolve().parent.parent / "data"


def nr_base_graph(bg, ils):
    """Base matrix V (entries -1 or V_ij) of BG1 (46 x 68) or BG2 (42 x 52) for
    lifting-set index ils, read from TS 38.212 Tables 5.3.2-2/3 (csv)."""
    shape = (46, 68) if bg == 1 else (42, 52)
    V = -np.ones(shape, dtype=int)
    row = 0
    for line in open(DATA / f"5G_bg{bg}.csv").read().splitlines()[2:]:
        f = line.split(";")
        if f[0].strip():
            row = int(f[0])
        V[row, int(f[1])] = int(f[2 + ils])
    return V


def nr_select(K, R, bg=None):
    """Base graph and lifting size for K information bits (incl. CRC) and rate R
    (TS 38.212, Sections 7.2.2 and 5.2.2)."""
    if bg is None:
        bg = 2 if (K <= 292 or (K <= 3824 and R <= 0.67) or R <= 0.25) else 1
    if bg == 1:
        kb = 22
    else:
        kb = 10 if K > 640 else 9 if K > 560 else 8 if K > 192 else 6
    Z = min(z for s in NR_LIFTING_SETS for z in s if kb * z >= K)
    ils = next(i for i, s in enumerate(NR_LIFTING_SETS) if Z in s)
    return bg, Z, ils


class NRCode:
    """A 5G NR LDPC code: lifted base graph, encoder and simple rate matching
    (redundancy version 0, no interleaving). Only the rows of the base graph that
    are needed for the E transmitted bits are kept (the other parity bits are not
    sent, so their checks carry no information)."""

    def __init__(self, K, E, bg=None):
        self.bg, Z, ils = nr_select(K, K / E, bg)
        self.K, self.E, self.Z, self.ils = K, E, Z, ils
        self.V = nr_base_graph(self.bg, ils)
        self.P = np.where(self.V >= 0, self.V % Z, -1)      # shift values P = V mod Z
        self.kb = 22 if self.bg == 1 else 10                 # information columns
        self.Kp = self.kb * Z                                # K' incl. filler bits
        self.filler = np.arange(K, self.Kp)                  # known zero bits
        nfull = self.V.shape[1] * Z                          # 68Z or 52Z
        tx = np.setdiff1d(np.arange(2 * Z, nfull), self.filler)   # first 2Z punctured
        self.Ncb = len(tx)
        tx = tx[:E] if E <= len(tx) else tx                   # bits that are ever sent
        ncol = max(self.kb + 4, -(-(tx.max() + 1) // Z))      # base columns in use
        self.code = qc_lift(self.P[:ncol - self.kb, :ncol], Z)
```

```
        self.code.name = f"NR BG{self.bg} K={K} E={E} Z={Z}"
        self.n = ncol * Z
        self.enc = DualEncoder(self.code, self.Kp, 4 * Z)
        self.tx_pos0 = tx
        self.tx_pos = np.resize(tx, E)                          # circular buffer, rv = 0

    def encode(self, u):
        u = np.atleast_2d(u)
        s = np.zeros((u.shape[0], self.Kp), dtype=np.uint8)
        s[:, :self.K] = u                                       # filler bits stay 0
        c = self.enc.encode(s)
        return c, c[:, self.tx_pos]                             # full codeword, sent bits

    def llr_in(self, llr_tx):
        """Map received LLRs of the E sent bits to the full-length decoder input."""
        L = np.zeros((llr_tx.shape[0], self.n))
        for p in range(0, self.E, len(self.tx_pos0)):           # repeated bits add up
            q = min(p + len(self.tx_pos0), self.E)
            L[:, self.tx_pos[p:q]] += llr_tx[:, p:q]
        L[:, self.filler] = 1e3                                 # filler bits are known zeros
        return L
```

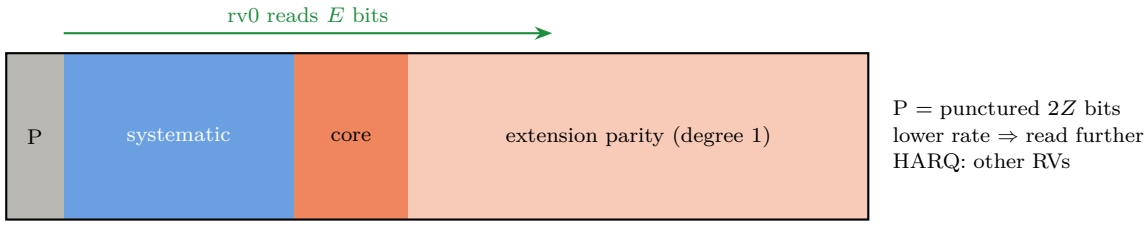
- `nr_base_graph` reads the TS 38.212 tables (stored as CSV files) for one set index.
- `nr_select` implements the base-graph rule and the choice of k_b, Z and i_{LS}.
- `NRCode` lifts the needed part of the base graph with `qc_lift`, builds the `DualEncoder`, and stores the transmitted positions `tx_pos` of the circular buffer.

11.9 Chapter summary

Key concepts

- NR data (DL-SCH, UL-SCH, PCH) uses LDPC; control and broadcast use polar codes.
- Two QC base graphs: BG1 (46×68, large blocks and high rates) and BG2 (42×52, small blocks and low rates).
- 51 lifting sizes in 8 sets; shift $P_{ij} = V_{ij} \bmod Z$.
- The first $2Z$ systematic bits are punctured; filler bits are known zeros.
- Circular-buffer rate matching with four redundancy versions supports HARQ incremental redundancy.

Visual recap • Chapter 11



Common student mistakes

! Common mistake

- Using the table values V_{ij} directly as shifts: always reduce modulo Z.
- Transmitting the first $2Z$ bits, or forgetting that the decoder must still include them (with LLR 0).
- Treating filler bits as unknown: they are known zeros and must get a large positive LLR.
- Choosing the base graph from the code-block size alone: the rule uses A and the target rate R.

Practice problems

Problem 11.1. Which base graph and lifting size are used for $K = 500$ bits at $R = 0.4$?

Solution $K \leq 3824$ and $R \leq 0.67$: BG2. $192 < K \leq 560$: $k_b = 8$. Smallest Z with $8Z \geq 500$: $Z = 64$ (set 0). Encoder input $10 \cdot 64 = 640$ bits, so 140 filler bits.

Problem 11.2. What is the codeword length after puncturing for BG1 with $Z = 384$, and the mother rate?

Solution $66 \cdot 384 = 25\,344$ bits; rate $8448/25344 = 1/3$.

Problem 11.3. For BG2 with $Z = 104$ and $N_{cb} = N$, find the RV2 starting position.

Solution $N = 50Z$, so $k_0 = \lfloor 25 \cdot 50Z/(50Z) \rfloor Z = 25Z = 2600$.

Problem 11.4. A transport block has $A = 10\,000$ bits, $R = 0.8$. How many code blocks are needed?

Solution $A > 3824$: CRC24A, $B = 10\,024$. $R > 0.67$ and $A > 3824$: BG1, $K_{cb} = 8448$. $C = \lceil 10\,024/(8448 - 24) \rceil = 2$.

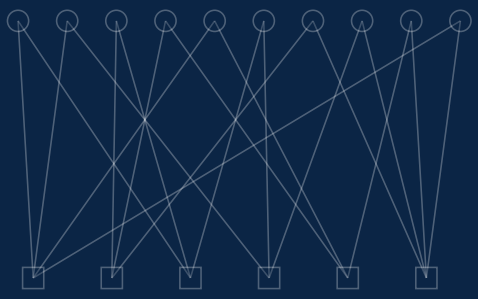
Self-assessment quiz

1. Why are the two first columns of the base graphs punctured?
2. What is the role of the set index i_{LS}?
3. Why does the extension part make HARQ easy?
4. Which base graph would you expect for a 100-bit block at rate 1/2?

Answers: 1) They are high-degree nodes; puncturing them improves the threshold and the rate. 2) It selects which table of shift values is used. 3) Each extra parity bit has its own check (degree-1 column), so lower rates are obtained by adding rows. 4) BG2 ($A \leq 292$).

CHAPTER

12



Python Implementation

IN THIS CHAPTER YOU WILL LEARN

- how the software of this book is organised;
- the edge-list data structure of a Tanner graph;
- vectorised message passing with `np.add.reduceat`;
- the module, block by block, with line numbers;
- a demonstration with real printed output;
- the Monte-Carlo simulation script;
- how every part of the code was tested.

12.1 Organisation of the software

`ldpc.py`	the library (NumPy only, about 600 lines): GF(2) algebra, code object, constructions, 5G NR codes, encoders, channels, all decoders, density evolution, simulation
`examples.py`	every number of the worked examples (Chapters 2–11)
`demo.py`	the demonstration of this chapter
`test_ldpc.py`	ten unit tests (Section 12.6)
`run_sims.py`	the Monte-Carlo runs of Chapter 13
`compare_polar.py`	the LDPC-versus-polar runs of Chapter 13 (uses `polar.py` of Part One)
`make_figs.py`, `plot_sims.py`	all figures
`design_bec.py`	the linear-programming design of Chapter 10
`project.py`	the final project of Chapter 16
`data/5G_bg1.csv`, `5G_bg2.csv`	base-graph tables of TS 38.212

The complete source of `ldpc.py` is printed in Appendix B with line numbers; the listings in this chapter use the same numbers. Appendix C gives the MATLAB version.

Fig. 12.1 shows how the files depend on each other.

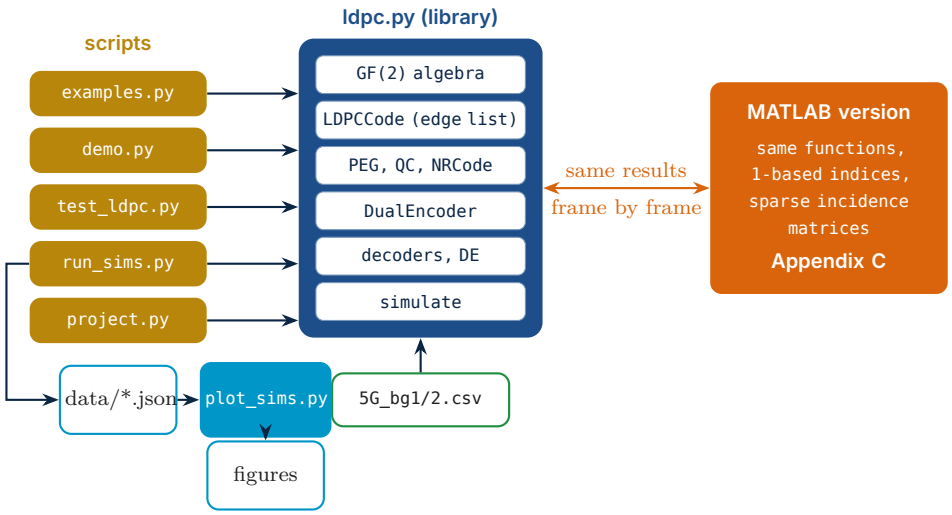


Figure 12.1. Block diagram. Structure of the book's software and its MATLAB counterpart.

12.2 The data structure: an edge list

Intuition

A Tanner graph is a list of edges. If every edge has a number e, all messages of one iteration fit in one array of length E (two arrays for the two directions). A check-node update is then a "group-by check" operation on that array, and a variable-node update is a "group-by variable" operation. NumPy can do both without Python loops.

edge e	0	1	2	3	4	5	6	7	8	9	10	11
`cn[e]`	0	0	0	1	1	1	2	2	2	3	3	3
`vn[e]`	0	1	3	1	2	4	0	4	5	2	3	5
`vperm`	0	6	1	3	4	9	2	10	5	7	8	11

For the toy code: edges are sorted by check, so check j owns the slice starting at `cstart[j]` $= (0, 3, 6, 9)$. `vperm` lists the edges sorted by variable (variable 0 owns edges 0 and 6, variable 1 owns 1 and 3, …), with starts `vstart` $= (0, 2, 4, 6, 8, 10)$. Then, for a batch of B frames stored as a (B, E) array `x`:

- per-check sums: `np.add.reduceat(x, cstart, axis=1)` $\to (B, m)$;
- per-variable sums: `np.add.reduceat(x[:, vperm], vstart, axis=1)` $\to (B, n)$;
- per-check minima: `np.minimum.reduceat(...)`, the core of min-sum.

12.3 The library, block by block

12.3.1 The code object (lines 60–103)

ldpc.py • lines 60--103

```
60  # ------------------------------------------------------------- 2. the code object
61  class LDPCCode:
62      """An LDPC code stored as the edge list of its Tanner graph.
63      Edge e joins check node cn[e] and variable node vn[e]; edges are sorted
64      by check node (then variable node)."""
65
66      def __init__(self, m, n, cn, vn, name="LDPC"):
67          order = np.lexsort((vn, cn))
68          self.m, self.n, self.name = m, n, name
69          self.cn, self.vn = np.asarray(cn)[order], np.asarray(vn)[order]
70          self.E = len(self.cn)
71          self.dc = np.bincount(self.cn, minlength=m)          # check degrees
72          self.dv = np.bincount(self.vn, minlength=n)          # variable degrees
73          self.cstart = np.r_[0, np.cumsum(self.dc)[:-1]]     # first edge of each check
74          self.vperm = np.lexsort((self.cn, self.vn))         # edges sorted by variable
75          self.vstart = np.r_[0, np.cumsum(self.dv)[:-1]]
76          self.layers = None
77
78      @classmethod
79      def from_H(cls, H, name="LDPC"):
```

```
80          cn, vn = np.nonzero(np.asarray(H) % 2)
81          return cls(H.shape[0], H.shape[1], cn, vn, name)
82
83      def H(self):
84          """Dense parity-check matrix (use only for small codes)."""
85          H = np.zeros((self.m, self.n), dtype=np.uint8)
86          H[self.cn, self.vn] = 1
87          return H
88
89      def design_rate(self):
90          return 1 - self.m / self.n
91
92      def check_sum(self, x):
93          """Per-check sums of edge values x (shape (B, E)) -> (B, m)."""
94          return np.add.reduceat(x, self.cstart, axis=1)
95
96      def var_sum(self, x):
97          """Per-variable sums of edge values x (shape (B, E)) -> (B, n)."""
98          return np.add.reduceat(x[:, self.vperm], self.vstart, axis=1)
99
100     def syndrome(self, c):
101         """Syndrome s = H c^T for a batch of words c (shape (B, n))."""
102         c = np.atleast_2d(c)
103         return self.check_sum(c[:, self.vn].astype(np.int32)) % 2
```

`from_H` builds the edge list from a dense matrix; QC and 5G codes are built directly as edge lists, so even the $17\,664 \times 26\,112$ matrix of BG1 with $Z = 384$ is never stored densely. `syndrome` gathers the hard bits on the edges and adds them per check.

12.3.2 Channels (lines 324–338)

ldpc.py • lines 324--338

```
324 # ------------------------------------------------------------------ 6. channels
325 def bpsk(c):
326     return 1.0 - 2.0 * c
327
328
329 def sigma_from_ebn0(ebn0_db, R):
330     """Noise standard deviation for BPSK (Es = 1) at the given Eb/N0 and rate."""
331     return np.sqrt(1.0 / (2 * R * 10 ** (ebn0_db / 10)))
332
333
334 def awgn_llr(c, ebn0_db, R, rng):
335     """Send codewords c through BPSK + AWGN and return channel LLRs 2y/sigma^2."""
336     sigma = sigma_from_ebn0(ebn0_db, R)
337     y = bpsk(c) + sigma * rng.standard_normal(c.shape)
338     return 2 * y / sigma ** 2
```

12.3.3 The other blocks

block	lines	explained in
`gf2`	lines 11–55	Chapter 3 (RREF, rank, $\mathbf{G}$ from $\mathbf{H}$)
`construct`	lines 108–197	Chapter 5 (Gallager, PEG, QC lifting, girth)
`nr5g`	lines 202–278	Chapter 11 (base graphs, lifting, rate matching)
`encode`	lines 283–319	Chapter 6 (dense $\mathbf{G}$, dual-diagonal encoder)
`hard`	lines 343–379	Chapter 7 (bit flipping, peeling)
`bp`	lines 384–436	Chapter 8 (SPA, min-sum family)
`layered`	lines 441–487	Chapter 9 (layers, layered decoding)
`de`	lines 492–593	Chapter 10 (DE, GA, Monte-Carlo DE, EXIT)
`sim`	lines 598–622	this chapter

12.3.4 Monte-Carlo simulation (lines 598–622)

ldpc.py • lines 598--622

```
598 # ------------------------------------------------------------- 11. Monte-Carlo
        simulation
599 def simulate(encode, decode, k, R, ebn0_list, info=None, max_frames=100000,
600                min_errors=100, batch=1000, seed=2026, stop_fer=0.0):
601     """BER/FER versus Eb/N0. encode(u) -> (codeword, sent bits);
602     decode(llr of sent bits) -> (decoded codeword, iterations); info = positions
603     of the message bits inside the decoded codeword. The sweep stops after the
604     first point with FER < stop_fer."""
605     rng = np.random.default_rng(seed)
606     res = {"ebn0": list(ebn0_list), "ber": [], "fer": [], "frames": [], "avg_iter": []}
607     for eb in ebn0_list:
608         bit_err = fr_err = frames = it_sum = 0
609         while frames < max_frames and fr_err < min_errors:
610             u = rng.integers(0, 2, (batch, k), dtype=np.uint8)
611             c, x = encode(u)
612             llr = awgn_llr(x, eb, R, rng)
613             c_hat, iters = decode(llr)
614             u_hat = c_hat[:, info] if info is not None else c_hat[:, :k]
615             e = (u_hat != u).sum(axis=1)
616             bit_err += e.sum(); fr_err += (e > 0).sum(); frames += batch; it_sum += iters.
        sum()
617         res["ber"].append(bit_err / (frames * k)); res["fer"].append(fr_err / frames)
618         res["frames"].append(frames); res["avg_iter"].append(it_sum / frames)
619         if fr_err / frames < stop_fer:                    # low enough: stop the sweep
620             res["ebn0"] = res["ebn0"][:len(res["fer"])]
621             break
622     return res
```

The function is independent of the code family: it receives an `encode` function (returning the full codeword and the transmitted bits) and a `decode` function (returning decisions and iteration counts). Errors are counted on the *information* bits. A run at one E_b/N_0 stops after `min_errors` frame errors or `max_frames` frames, and the sweep stops after the first point below `stop_fer`.

12.4 Demonstration with outputs

demo.py

```python
"""Demonstrations for Chapter 12 (run: python3 demo.py)."""
import numpy as np
from ldpc import *

rng = np.random.default_rng(12)
# 1. a PEG code and its encoder
code = peg(1008, 504, 3, seed=1)
G, info = generator_from_H(code.H())
print(f"{code.name}: n={code.n} m={code.m} edges={code.E} girth={girth(code)} k={G.shape[0]}")

# 2. one noisy frame: unsatisfied checks after each iteration
fr = np.random.default_rng(1)                          # one frame at Eb/N0 = 2 dB
u = fr.integers(0, 2, (1, 504)); c = encode_G(G, u).astype(np.uint8)
L = awgn_llr(c, 2.0, 0.5, fr)
print("hard-decision bit errors before decoding:", int(((L < 0) != c).sum()),
      " unsatisfied checks:", int(code.syndrome((L < 0).astype(np.uint8)).sum()))
make_layers(code)
for name, f in [("flooding SPA", lambda t: bp_decode(code, L, t, "spa", early_stop=False)[0]),
                ("flooding NMS", lambda t: bp_decode(code, L, t, "nms", early_stop=False)[0]),
                ("layered NMS ", lambda t: layered_decode(code, L, t, "nms")[0])]:
    trace = []
    for t in range(1, 21):
        x = f(t)
        trace.append(int(code.syndrome(x).sum()))
        if trace[-1] == 0:
            break
    print(f"{name}: unsatisfied checks per iteration {trace}  bit errors at end {int((x != c).sum())}")

# 3. a 5G NR code block
nr = NRCode(1024, 2048); make_layers(nr.code, nr.Z)
u = rng.integers(0, 2, (1000, 1024)); cfull, x = nr.encode(u)
for eb in [1.0, 1.5, 2.0]:
    Lr = awgn_llr(x, eb, 0.5, rng)
    xh, it, _ = layered_decode(nr.code, nr.llr_in(Lr), 50, "nms")
    fer = (xh[:, :1024] != u).any(1).mean()
    print(f"{nr.code.name}, Eb/N0 = {eb} dB: FER = {fer:.3f}, average iterations = {it.mean():.1f}")
```

OUTPUT (real run)

```
PEG(1008,504): n=1008 m=504 edges=3024 girth=8 k=504
hard-decision bit errors before decoding: 107  unsatisfied checks: 193
flooding SPA: unsatisfied checks per iteration [89, 56, 48, 49, 34, 23, 19, 16, 15, 6, 0]  bit errors at end 0
flooding NMS: unsatisfied checks per iteration [91, 59, 56, 52, 41, 23, 24, 21, 12, 9, 3, 0]  bit errors at end 0
```

```
layered NMS : unsatisfied checks per iteration [61, 40, 28, 25, 12, 3, 0]  bit errors at end 0
NR BG2 K=1024 E=2048 Z=104, Eb/N0 = 1.0 dB: FER = 0.257, average iterations = 26.1
NR BG2 K=1024 E=2048 Z=104, Eb/N0 = 1.5 dB: FER = 0.004, average iterations = 9.5
NR BG2 K=1024 E=2048 Z=104, Eb/N0 = 2.0 dB: FER = 0.000, average iterations = 6.3
```

- The PEG code has girth 8 and full rank, so $k = 504$.
- One frame at $E_b/N_0 = 2$ dB arrives with 107 wrong hard decisions and 193 unsatisfied checks. Flooding SPA needs 11 iterations, flooding NMS 12 and layered NMS only 7. The number of unsatisfied checks does not always fall monotonically: the decoder can move "sideways" before it locks onto the codeword.
- The 5G NR code with $K = 1024$, $R = 1/2$ (BG2, $Z = 104$) goes from FER 0.26 at 1.0 dB to 0.004 at 1.5 dB and no error in 1000 frames at 2.0 dB, with 6.3 iterations on average.

12.5 The simulation script

run_sims.py

```python
"""Monte-Carlo simulations used for the BER/FER figures of the book."""
import json, sys, time, numpy as np
from ldpc import *

def jsonify(r):
    return {k: [float(x) for x in v] if isinstance(v, list) else v for k, v in r.items()}

def run(tag, **kw):
    t = time.time()
    r = simulate(**kw)
    print(tag, f"{time.time() - t:.0f}s", ["%.2e" % f for f in r["fer"]], flush=True)
    r = jsonify(r)
    out[tag.split()[0]] = r
    json.dump(out, open(f"../data/sim_{group}.json", "w"), indent=1)    # save after every
    curve
    return r

def peg_setup():
    code = peg(1008, 504, 3, seed=1)
    G, info = generator_from_H(code.H())
    make_layers(code)
    enc = lambda u: (lambda c: (c, c))(encode_G(G, u).astype(np.uint8))
    return code, G, info, enc

def nr_setup(K, E):
    nr = NRCode(K, E)
    make_layers(nr.code, nr.Z)
    return nr

E = lambda a, b, s=0.25: list(np.round(np.arange(a, b + 1e-9, s), 2))
group = sys.argv[1]
out = {}
common = dict(max_frames=40000, min_errors=50, seed=2026, stop_fer=2e-4)
```

```
if group == "A":          # decoders on PEG(1008, 504)
    code, G, info, enc = peg_setup()
    k, R = 504, 0.5
    for name, rule, lay in [("SPA", "spa", False), ("NMS", "nms", False), ("OMS", "oms", False),
                            ("MS", "ms", False), ("LNMS", "nms", True), ("LSPA", "spa", True)]:
        dec = (lambda L, r=rule: layered_decode(code, L, 50, r)[:2]) if lay else \
              (lambda L, r=rule: bp_decode(code, L, 50, r)[:2])
        hi = 3.0 if rule != "ms" else 3.75
        run(name, encode=enc, decode=dec, k=k, R=R, ebn0_list=E(0.5, hi), info=info,
                        batch=1000, **common)
    dec = lambda L: bit_flip(code, (L < 0).astype(np.uint8), 50)
    run("BF", encode=enc, decode=dec, k=k, R=R, ebn0_list=E(3.0, 7.0, 0.5), info=info,
                    batch=2000, **common)
elif group == "B":        # maximum number of iterations, flooding NMS
    code, G, info, enc = peg_setup()
    for it in [5, 10, 20, 50, 100]:
        dec = lambda L, it=it: bp_decode(code, L, it, "nms")[:2]
        run(f"I{it}", encode=enc, decode=dec, k=504, R=0.5, ebn0_list=E(0.5, 3.0),
                            info=info, batch=1000, **common)
elif group == "C":        # 5G NR, R = 1/2, block lengths
    for K in [128, 256, 512, 1024, 2048, 4096]:
        nr = nr_setup(K, 2 * K)
        dec = lambda L, nr=nr: layered_decode(nr.code, nr.llr_in(L), 50, "nms")[:2]
        hi = {128: 4.0, 256: 3.5, 512: 3.0, 1024: 2.5, 2048: 2.25, 4096: 2.0}[K]
        run(f"K{K} BG{nr.bg} Z{nr.Z}", encode=nr.encode, decode=dec, k=K, R=0.5,
                            ebn0_list=E(0.0, hi), info=np.arange(K), batch=max(200, 100000 // (2 * K)),
                            **(common | ({"max_frames": 20000} if K >= 2048 else {})))
        out[f"K{K}"].update(bg=nr.bg, Z=nr.Z)
elif group == "D":        # 5G NR, K = 1024, code rates
    for R in [1 / 5, 1 / 3, 1 / 2, 2 / 3, 3 / 4, 5 / 6, 8 / 9]:
        Ecw = int(round(1024 / R))
        nr = nr_setup(1024, Ecw)
        dec = lambda L, nr=nr: layered_decode(nr.code, nr.llr_in(L), 50, "nms")[:2]
        lo = {0.2: -1.0, 1/3: -0.5}.get(R, 0.0)
        run(f"R{R:.3f} BG{nr.bg} Z{nr.Z}", encode=nr.encode, decode=dec, k=1024,
                                R=1024 / Ecw, ebn0_list=E(lo, lo + 5.0), info=np.arange(1024),
                                batch=200, **common)
        out[f"R{R:.3f}"].update(bg=nr.bg, Z=nr.Z, E=Ecw)
json.dump(out, open(f"../data/sim_{group}.json", "w"), indent=1)
```

Each group writes a JSON file with Eb/N0, BER, FER, the number of frames and the average number of iterations; `plot_sims.py` turns them into the figures of Chapter 13. With fixed seeds, the runs are exactly reproducible.

12.6 How the code was tested

OUTPUT (real run)

```
PASS GF(2): rank, inverse and generator matrix
PASS Code object: degrees, H and syndromes
PASS Girth and 4-cycles
PASS 5G NR tables, parameter selection and encoder
PASS Check-node update equals the brute-force formulas
PASS Sum-product is exact (bit-MAP) on a cycle-free graph
PASS Decoders: noiseless frames, high-SNR frames, layered = flooding decisions
PASS Peeling and bit flipping
PASS Density evolution thresholds
PASS Monte-Carlo simulation runs and decodes at high SNR

10 tests passed
```

GF(2)	ranks, $\mathbf{A}\mathbf{A}^{-1} = \mathbf{I}$, $\mathbf{H}\mathbf{G}^T = \mathbf{0}$ and identity at the information positions
Code object	$\mathbf{H}$ round trip, degrees, zero syndrome for all 8 toy codewords
Girth	toy 6, QC examples 8 and 4 (with exactly 5 four-cycles), chain code ∞
5G NR	316 and 197 entries in every set index; parameter selection; $\mathbf{H}\mathbf{c}^T = \mathbf{0}$ for eight (K, E) pairs
Check update	SPA equals the tanh product, MS equals sign×min, NMS equals α×MS, on random inputs
Exactness	SPA equals brute-force bit-MAP on a cycle-free code (error $< 10^{-9}$)
Decoders	noiseless frames decoded in one iteration; all frames correct at 4 dB
Peeling, BF	peeling never wrong; BF corrects every single error of a girth-≥ 6, $d_v = 3$ code
DE	BEC thresholds 0.4294 and 0.6474; GA threshold in $(0.870, 0.874)$
Simulation	a short 5G run at 4 dB has no frame error

Speed

The library is written for clarity, not for maximum speed, but vectorisation over frames and edges makes it practical: the simulations of Chapter 13 (millions of decoded frames) ran on a two-core virtual machine in a few hours. For more speed, compile the check update with Numba, use `float32`, or move to a GPU array library; the structure of the code does not change.

12.7 Chapter summary

Key concepts

- A Tanner graph is stored as edge arrays `cn`, `vn` sorted by check, plus `vperm` for the variable view.
- Message passing = group-by operations: `reduceat` over check slices and variable slices.
- Decoders work on batches of frames and drop frames as soon as their syndrome is zero.
- The simulation function is generic: any encoder and decoder can be plugged in.
- Ten unit tests check algebra, structure, decoders, DE and the 5G construction.

Common student mistakes

! Common mistake

- Building a dense $\mathbf{H}$ for a large QC code: memory explodes ($17\,664 \times 26\,112$ bytes $= 461$ MB for BG1, $Z = 384$).
- Python loops over edges or checks: 100–1000 times slower than the vectorised version.
- Counting errors on the whole codeword instead of the information bits.
- Reusing the same noise for different decoders without saying so (fair for comparison, but say it).

Practice problems

Problem 12.1. For the toy code, what are `cstart` and `vstart`, and which edges belong to variable 4?

Solution `cstart`$= (0, 3, 6, 9)$, `vstart`$= (0, 2, 4, 6, 8, 10)$; variable 4 owns `vperm[8:10]`$= (5, 7)$, i.e. edges (c_1, v_4) and (c_2, v_4).

Problem 12.2. Write one NumPy line that returns the number of unsatisfied checks of each frame from hard decisions `x`.

Solution `code.syndrome(x).sum(axis=1)`, or explicitly `(np.add.reduceat(x[:, code.vn], code.cstart, axis=1) % 2).sum(1)`.

Problem 12.3. Modify the call in `run_sims.py` to simulate offset min-sum with $\beta = 0.4$ and 20 iterations.

Solution `dec = lambda L: bp_decode(code, L, 20, "oms", beta=0.4)[:2]`.

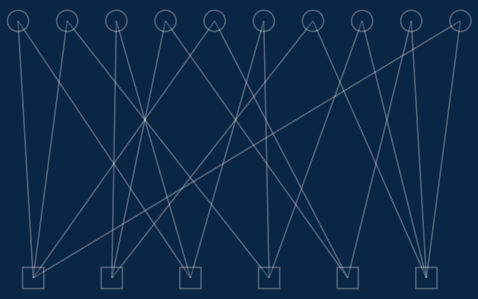
Self-assessment quiz

1. Why are edges sorted by check node?
2. What does `vperm` do?
3. Why does `bp_decode` keep the array `act`?
4. Which test proves that the sum-product rules are implemented correctly?

Answers: 1) So that each check's edges are a contiguous slice for `reduceat`. 2) It reorders edges by variable for the variable-node sums. 3) To process only frames that have not converged. 4) The exactness test on the cycle-free code (together with the brute-force check-update test).

CHAPTER

13



Performance Analysis and Comparison with Polar Codes

IN THIS CHAPTER YOU WILL LEARN

- how the simulations were run and how to read them;
- sum-product, min-sum family and bit flipping compared;
- average iterations and the iteration limit;
- the effect of block length (5G NR, $K = 128$ to 4096);
- the effect of code rate (1/5 to 8/9) and the gap to capacity;
- a fair comparison of LDPC and polar codes;
- design trade-offs and current research.

13.1 Simulation set-up

Codes	PEG(1008, 504) of Chapter 5 (girth 8); 5G NR codes built by `NRCode` (rv0, BPSK)
Channel	BPSK over AWGN, $\sigma^2 = 1/(2R\,E_b/N_0)$, R = information rate
Decoders	flooding SPA, MS, NMS ($\alpha = 0.75$), OMS ($\beta = 0.5$); layered NMS and SPA; bit flipping; 50 iterations unless stated; syndrome-based early stopping
Statistics	per point: until 50 frame errors or 40 000 frames (20 000 for $K \geq 2048$); a sweep stops after the first point with FER $< 2 \times 10^{-4}$; only points with at least 10 frame errors are plotted
Reproducibility	fixed seeds; random information bits in every frame (not only the all-zero codeword)

Fig. 13.1 shows the simulation loop with these rules.

! Common mistake

A point with 10 errors has a relative uncertainty of roughly $\pm 30\%$ (one standard deviation is about $1/\sqrt{10}$). Differences of 0.05 dB between curves are therefore not significant; differences of several tenths of a dB are.

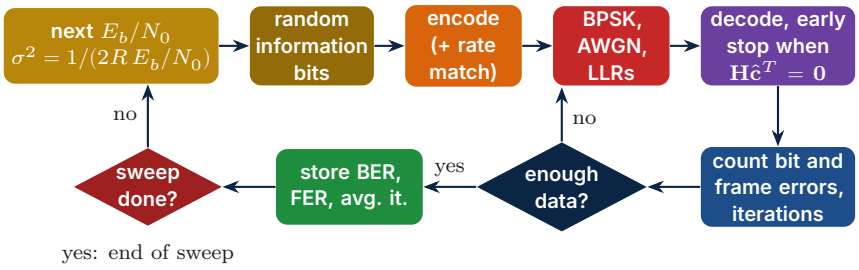


Figure 13.1. Block diagram. Monte-Carlo loop of Chapter 13. "Enough data" = 50 frame errors or the frame limit; the sweep is done after the first point with FER $< 2 \times 10^{-4}$.

13.2 Comparing decoders

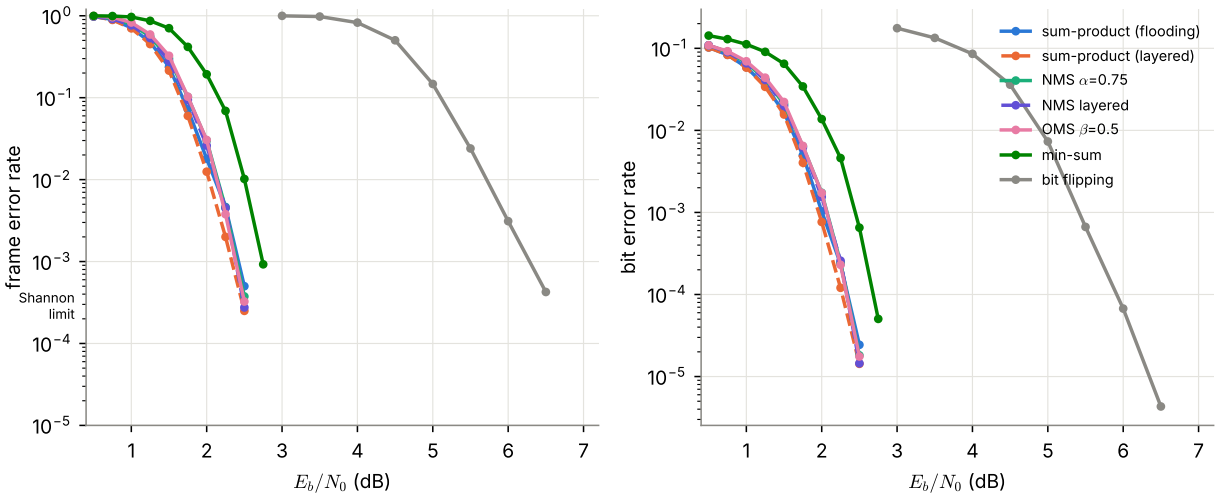


Figure 13.2. PEG(1008, 504) code, $R = 1/2$, at most 50 iterations. Left: frame error rate; right: bit error rate. Dashed: layered schedule. Dotted vertical line: Shannon limit for BPSK at $R = 1/2$ (0.19 dB).

decoder (max. 50 iterations)	FER 10^{-2} at	FER 10^{-3} at	vs. SPA at 10^{-3}	avg. it. at 2.5 dB
sum-product, flooding	2.11 dB	2.42 dB	reference	6.4
sum-product, layered	2.03 dB	2.33 dB	−0.09 dB	3.7
NMS ($\alpha = 0.75$), flooding	2.15 dB	2.40 dB	−0.02 dB	6.8
NMS ($\alpha = 0.75$), layered	2.14 dB	2.38 dB	−0.04 dB	3.9
OMS ($\beta = 0.5$), flooding	2.13 dB	2.39 dB	−0.03 dB	6.8
min-sum, flooding	2.50 dB	2.74 dB	+0.32 dB	8.4
bit flipping (hard)	5.71 dB	6.29 dB	+3.87 dB	(2.8 at 6.5 dB)

(Crossings by log-linear interpolation between simulated points.) Four lessons:

1. **The corrected min-sum decoders are as good as SPA.** NMS and OMS are within a few hundredths of a dB of flooding SPA — inside the statistical uncertainty. At this length, with 50 iterations, they even look marginally better at 10^{-3}; this is noise, not a real gain.
2. **Plain min-sum loses about** 0.3–0.4 **dB.** Its over-confidence costs a factor of about ten in FER at 2.5 dB.

3. **The layered schedule helps a little in FER and a lot in iterations.** Layered SPA is the best curve (0.09 dB better than flooding SPA at 10^{-3}), because with a 50-iteration limit it finishes more of the slowly converging frames.
4. **Hard decisions are expensive.** Bit flipping needs almost 4 dB more than SPA (Chapter 7 explained 1.6 dB of this as the information loss of the hard decision; the rest is the crude decoding rule).

At FER 10^{-3} the best decoder is still about 2.1 dB from the Shannon limit (0.19 dB): this is a short code ($n = 1008$) and a regular-like degree distribution, whose infinite-length threshold is already about 1.1 dB (Chapter 10).

13.3 Iterations

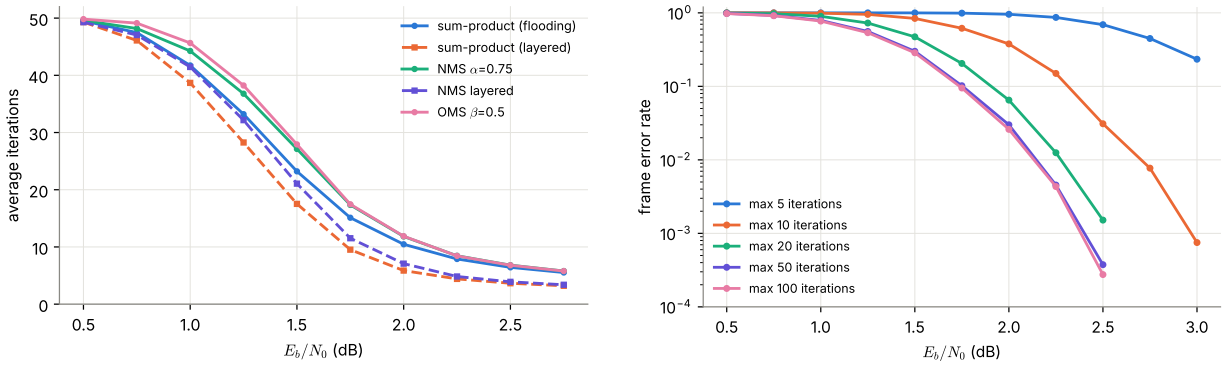


Figure 13.3. PEG(1008, 504). Left: average number of iterations with early stopping. Right: flooding NMS with different iteration limits.

Average iterations. With early stopping, the average falls quickly once the code works: at 2.5 dB, flooding SPA needs 6.4 iterations, flooding NMS 6.8, layered NMS 3.9 and layered SPA 3.7. Layered decoding thus saves about 40% of the iterations in the useful region, as predicted in Chapter 9. Near the waterfall (1.0–1.5 dB) most frames use the whole budget of 50 iterations.

Iteration limit. The right panel shows flooding NMS with $I_{\max} = 5, 10, 20, 50, 100$. At FER 10^{-2}:

$I_{\max}$	5	10	20	50	100
E_b/N_0 at FER 10^{-2}	> 3.0 dB	2.70 dB	2.28 dB	2.15 dB	2.13 dB

Going from 10 to 20 iterations gains 0.4 dB, from 20 to 50 only 0.13 dB, and from 50 to 100 almost nothing. Because early stopping keeps the *average* small, a large maximum costs only latency in the rare difficult frames. (The curves for 50 iterations and for the NMS decoder of Fig. 13.2 are identical, because they use the same seed — a useful consistency check.)

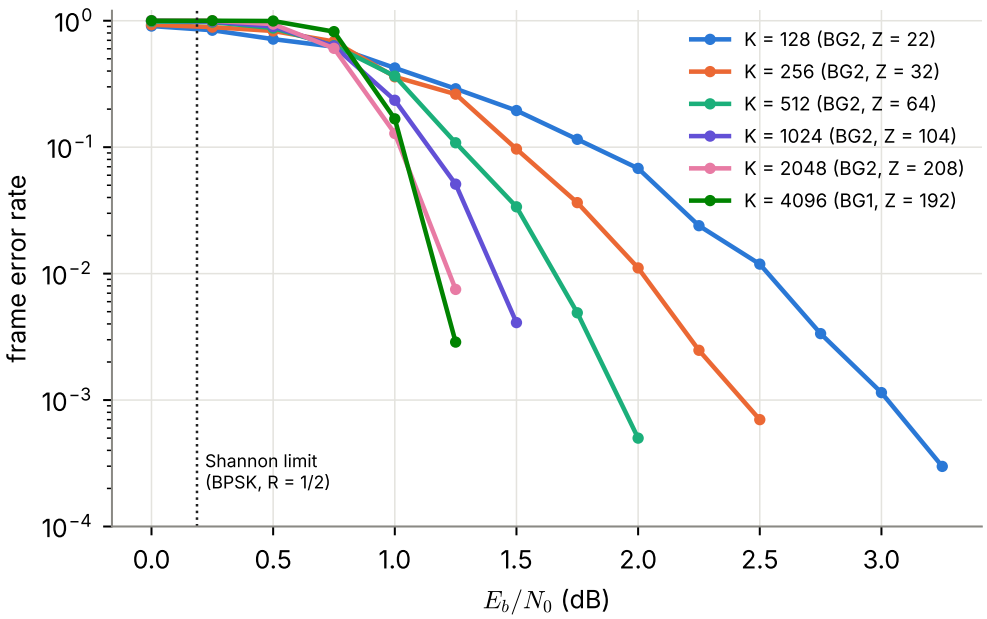


Figure 13.4. 5G NR codes at $R = 1/2$ (rv0, BPSK), layered NMS, 50 iterations.

13.4 Block length

K (information bits)	128	256	512	1024	2048	4096
base graph, Z	BG2, 22	BG2, 32	BG2, 64	BG2, 104	BG2, 208	BG1, 192
E_b/N_0 at FER 10^{-2} (dB)	2.53	2.02	1.66	1.41	1.22	1.17
E_b/N_0 at FER 10^{-3} (dB)	3.03	2.43	1.92	≈ 1.6	< 1.5	≈ 1.3
gap to 0.19 dB at FER 10^{-2}	2.34	1.83	1.47	1.22	1.03	0.98

- **Each doubling of K gains 0.2–0.5 dB** at FER 10^{-2}; the gain shrinks as K grows. From $K = 128$ to $K = 4096$ the required E_b/N_0 falls by 1.4 dB.
- **The waterfall becomes steeper.** For $K = 4096$ the FER drops from 0.17 at 1.0 dB to 0.003 at 1.25 dB, and no frame failed at 1.5 dB (20 000 frames). For $K = 128$ the same factor of 50 needs about 1.25 dB. This is the finite-length scaling of Chapter 10: long codes concentrate around the ensemble threshold.
- **Still about 1 dB from the Shannon limit** at $K = 4096$ and FER 10^{-2}. Part of the gap is the BPSK/finite-length penalty that any code has; part is the practical decoder (layered NMS, 50 iterations).
- At $K = 4096$ the base-graph rule switches to BG1 ($K > 3824$). The cross-over is smooth: BG1 at this size is as good as the BG2 trend predicts.
- Values marked $\approx$ or $<$ rely on points with fewer than 10 errors.

13.5 Code rate

All seven codes carry $K = 1024$ information bits; only E changes (5120 bits at $R = 1/5$, 1152 bits at $R = 8/9$). The base-graph rule selects BG2 up to $R = 2/3$ and BG1 above.

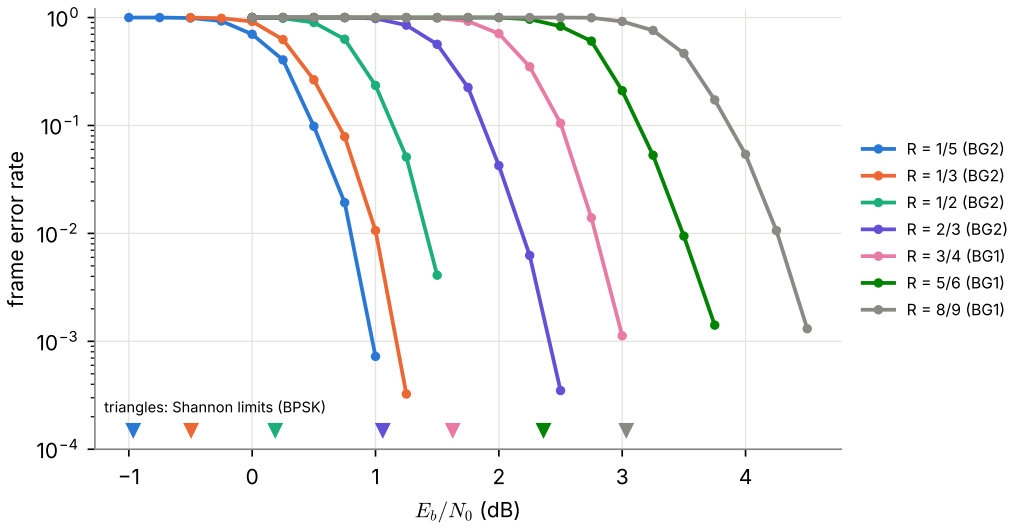


Figure 13.5. 5G NR codes with $K = 1024$ at seven rates, layered NMS, 50 iterations. Triangles: Shannon limits for BPSK.

rate R	1/5	1/3	1/2	2/3	3/4	5/6	8/9
base graph, Z	BG2, 104	BG2, 104	BG2, 104	BG2, 104	BG1, 48	BG1, 48	BG1, 48
E_b/N_0 at FER 10^{-2} (dB)	0.80	1.00	1.41	2.19	2.78	3.49	4.26
Shannon limit, BPSK (dB)	−0.96	−0.50	0.19	1.06	1.63	2.36	3.03
gap (dB)	1.76	1.50	1.22	1.13	1.16	1.13	1.22

- From $R = 1/2$ to $R = 8/9$ the gap to the Shannon limit is nearly constant, about 1.1–1.2 dB at FER 10^{-2} for $K = 1024$: the rate-compatible design works well over a wide range.
- At low rates the gap grows (1.5 dB at 1/3, 1.8 dB at 1/5). The extra parity bits come from degree-1 extension columns, each protected by a single check; they add energy but little "structure". Low-rate operation is also where repetition-like coding and HARQ combining are used in practice.
- High rates need more E_b/N_0 simply because the Shannon limit itself rises steeply (Chapter 2).

13.6 LDPC versus polar codes

Fig. 13.6 shows how the comparison is made fair: both schemes carry the same number of information bits in the same number of coded bits, see the same channel model and are measured with the same E_b/N_0 definition.

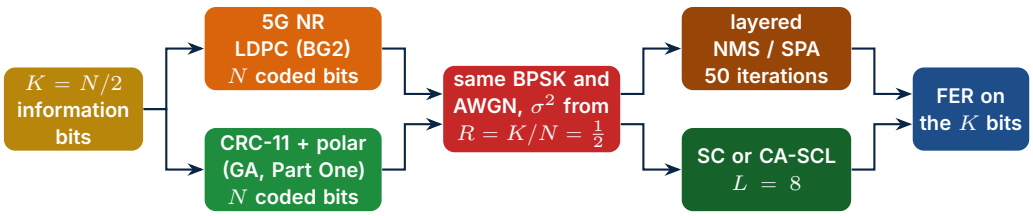


Figure 13.6. Block diagram. Set-up of the LDPC versus polar comparison. The CRC bits of the polar scheme are counted as overhead, so both schemes have the same information rate.

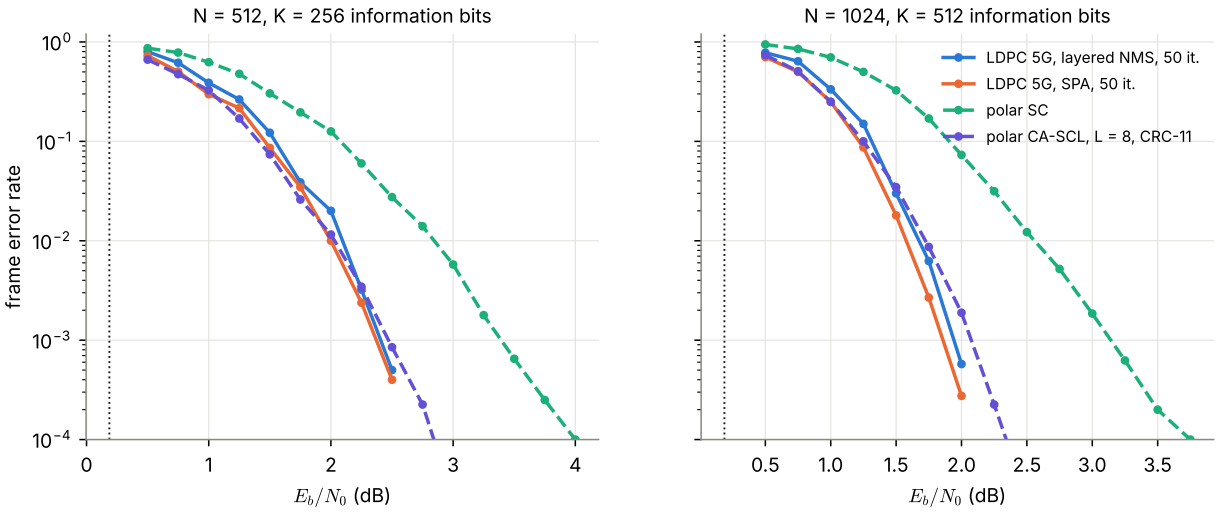


Figure 13.7. Equal length and rate: 5G NR LDPC codes (BG2) and polar codes of Part One (GA construction at 2 dB, natural order) with SC and CRC-aided list decoding. The 11 CRC bits of the list decoder are counted as overhead: all schemes carry $N/2$ information bits in N coded bits.

	$N = 512$, $K = 256$		$N = 1024$, $K = 512$	
E_b/N_0 needed for FER	10^{-2}	10^{-3}	10^{-2}	10^{-3}
LDPC (5G BG2), SPA, 50 it.	2.00 dB	2.37 dB	1.58 dB	1.86 dB
LDPC (5G BG2), layered NMS, 50 it.	2.09 dB	2.41 dB	1.68 dB	1.94 dB
polar, CA-SCL $L = 8$, CRC-11	2.03 dB	2.47 dB	1.72 dB	2.07 dB
polar, SC	2.85 dB	3.39 dB	2.56 dB	3.14 dB

- **At $N = 512$ the two families are equal** within about 0.1 dB: at FER 10^{-2} CRC-aided list decoding with $L = 8$ is 0.03 dB behind LDPC with SPA and 0.06 dB ahead of layered NMS; at 10^{-3} it is 0.06–0.10 dB behind both. These differences are close to the statistical uncertainty.
- **At $N = 1024$ LDPC pulls ahead** by about 0.1–0.2 dB at FER 10^{-3}, and the gap grows with length: polar codes need longer lists to keep up, LDPC codes benefit directly from the larger graph.
- **Plain SC is not competitive**: about 0.8–1.0 dB behind. This is why 5G uses CA-SCL for polar codes [Part One, Ch. 8].
- These results agree with the 5G decision: polar codes for short control messages (tens to a few hundred bits, where CA-SCL is at least as good and needs no iterations), LDPC codes for data (hundreds to thousands of bits, high throughput).

What this comparison does not say

A list size of $L = 32$ or a better polar construction would move the polar curves left; a better LDPC decoder schedule or more iterations would move the LDPC curves left. Hardware cost, latency and HARQ support are different questions. Always state length, rate, decoder, list size or iteration limit, and the CRC accounting.

13.7 Complexity, latency and throughput

	LDPC (layered NMS)	Polar (CA-SCL, list L)
Work per frame	$\approx 2E$ messages per iteration $\times$ iterations (fewer at high SNR)	$\approx L \cdot \frac{N}{2} \log_2 N$ f/g operations + sorting
Parallelism	Z checks per layer; all frames independent	sequential over bits; parallel over the list
Latency	iterations $\times$ layers	$\approx N$ (plus list management); fast decoders shorten it
Early stopping	yes (syndrome)	no (CRC only at the end)
Rate flexibility	lifting + circular buffer	puncturing / shortening / repetition
HARQ-IR	natural (extension parity)	possible, less natural

13.8 Practical design trade-offs

- **Decoder choice.** Layered NMS or OMS is the default: within about 0.1 dB of SPA, fewer iterations, simple hardware. Use SPA in software when every tenth of a dB counts.
- **Iteration limit.** Choose it at the target FER: going from 20 to 50 iterations helps only near the waterfall, and early stopping keeps the *average* low.
- **Block length.** Longer blocks gain steeply up to a few thousand bits; segmentation into code blocks of at most 8448 bits limits the gain in 5G.
- **Code rate.** For $K = 1024$ the gap to the Shannon limit is nearly constant (about 1.1–1.2 dB) from $R = 1/2$ to $8/9$ and grows at low rates, where many parity bits come from degree-1 extension columns.
- **Floors.** If the system needs FER $\ll 10^{-5}$, check the error floor with long simulations or hardware emulation; DE does not predict it.

13.9 Current research directions

- **Spatially coupled LDPC codes.** Coupling a chain of protographs makes BP reach the (much better) MAP threshold: "threshold saturation" (Kudekar, Richardson and Urbanke, 2011). Candidates for optical and 6G links.
- **Non-binary LDPC codes** over GF(q) improve short-block performance at a higher decoding cost.
- **Learned decoders.** Unfolding BP into a neural network and training weights ("neural min-sum") generalises the normalisation factor α to one weight per edge and iteration.
- **Quantum LDPC codes.** Sparse stabiliser codes with good rate and distance are a leading route to fault-tolerant quantum computing; their decoders build on BP with post-processing.

- **Low-latency and high-throughput decoders** for 6G (hundreds of Gbit/s), including unrolled and partially parallel architectures.

13.10 Chapter summary

Key concepts

- NMS and OMS are within a few hundredths of a dB of SPA; plain min-sum loses about 0.3 dB; bit flipping almost 4 dB.
- Layered decoding saves about 40% of the iterations and is slightly better at a fixed iteration limit.
- 20 iterations lose about 0.13 dB against 50; more than 50 hardly helps.
- 5G NR LDPC at $R = 1/2$: from 2.53 dB ($K = 128$) to 1.17 dB ($K = 4096$) at FER 10^{-2}.
- For $K = 1024$ the gap to the BPSK limit is about 1.1–1.2 dB for $R \geq 1/2$ and larger at low rates.
- LDPC and CA-SCL polar codes are equal at $N = 512$; LDPC is 0.1–0.2 dB better at $N = 1024$.

Common student mistakes

! Common mistake

- Comparing codes of different rates in E_s/N_0 instead of E_b/N_0.
- Trusting points with two or three errors.
- Comparing a list decoder with a CRC to an LDPC code without counting the CRC bits in the rate.
- Reporting the FER without the decoder, the schedule and the iteration limit.

Practice problems

Problem 13.1. Using the table of block lengths, estimate the E_b/N_0 saved by doubling K from 512 to 1024 bits, and the energy saving in percent.

Solution $1.66 - 1.41 = 0.25$ dB, i.e. $10^{0.025} = 1.059$: about 6% less energy per bit.

Problem 13.2. A link must reach FER 10^{-2} with $K = 1024$ at $E_b/N_0 = 2.5$ dB. Which of the simulated rates can be used?

Solution Rates up to 2/3 (2.19 dB); 3/4 needs 2.78 dB.

Problem 13.3. With layered NMS the PEG code needs 3.9 iterations on average at 2.5 dB. If one iteration takes 1 μs, what is the average decoding throughput for 504 information bits? What sets the worst case?

Solution $504/3.9\,\mu\text{s} \approx 129$ Mbit/s on average. The worst case is set by the iteration limit (50 μs per frame, 10 Mbit/s), so hardware needs buffering or a smaller limit.

Problem 13.4. At FER 10^{-2} the PEG code with flooding SPA needs 2.11 dB and the BPSK limit is 0.19 dB. Which part of the 1.9 dB gap would disappear for a very long $(3, 6)$ code, according to Chapter 10?

Solution The $(3, 6)$ threshold is about 1.1 dB, so roughly 1.0 dB of the gap is a finite-length effect; the remaining 0.9 dB comes from the regular degree distribution and needs irregular design.

Self-assessment quiz

1. Why does early stopping reduce the average number of iterations most at high E_b/N_0?
2. Why can a longer code have a steeper waterfall?
3. What must be counted to compare a CRC-aided polar code fairly with an LDPC code?
4. Name one research direction that closes the gap between BP and MAP decoding.

Answers: 1) Most frames are corrected in a few iterations and stop as soon as the syndrome is zero. 2) Its performance concentrates around the ensemble threshold (finite-length scaling). 3) The CRC bits as part of the overhead (same information rate), plus the decoder complexity. 4) Spatial coupling (threshold saturation).

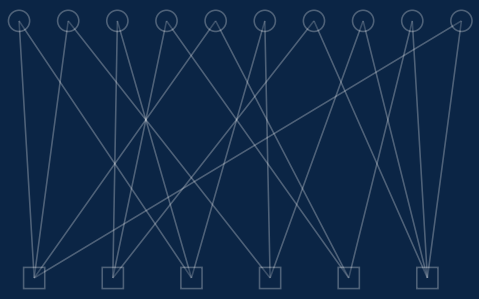
PART V

Expert Practice

Fully solved problems, interview and examination questions, and an end-to-end 5G NR LDPC link as a final project.

CHAPTER

14



Fully Solved Problems

IN THIS CHAPTER YOU WILL LEARN

Twenty-four problems, from basic to advanced. Try each one with paper and pencil before reading the solution. Every numerical answer was checked with the library (script `solved.py`). Unless stated otherwise: BPSK $0 \mapsto +1$, LLR $L = 2y/\sigma^2$, the toy matrix $\mathbf{H}$ of (1.1), and edges listed check by check.

14.1 Basic level

Problem 14.1. (Rate, noise and LLR) An LDPC code has $n = 1200$, $k = 600$ and is used with BPSK at $E_b/N_0 = 1.5$ dB. Find R, E_s/N_0, σ and the LLR of the sample $y = -0.4$.

Solution $R = 0.5$. $E_b/N_0 = 10^{0.15} = 1.4125$, so $E_s/N_0 = 0.5 \cdot 1.4125 = 0.7063$. $\sigma^2 = 1/(2 \cdot 0.7063) = 0.7079$, $\sigma = 0.8414$. $L = 2(-0.4)/0.7079 = -1.130$: a weak vote for bit 1.

Problem 14.2. (Codeword test) Is $\mathbf{c} = (1, 1, 1, 0, 0, 1)$ a codeword of the toy code?

Solution Check 0: $c_0 \oplus c_1 \oplus c_3 = 1 \oplus 1 \oplus 0 = 0$; check 1: $c_1 \oplus c_2 \oplus c_4 = 1 \oplus 1 \oplus 0 = 0$; check 2: $c_0 \oplus c_4 \oplus c_5 = 1 \oplus 0 \oplus 1 = 0$; check 3: $c_2 \oplus c_3 \oplus c_5 = 1 \oplus 0 \oplus 1 = 0$. Yes.

Problem 14.3. (Degrees, rank and cycles)

$$\mathbf{H}_8 = \begin{bmatrix} 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 \end{bmatrix}.$$

Find the degrees, the design rate, the true rate and the number of 4-cycles.

Solution Every column has weight 2 and every row weight 4: $(2, 4)$-regular, $E = 16$, design rate $1 - 2/4 = 0.5$. Rows 0 and 1 add to 11111111, and so do rows 2 and 3: one row is redundant, rank 3, $k = 5$, true rate $5/8 = 0.625$. Rows 0 and 2 share columns 0 and 2, rows

0 and 3 share 1 and 3, rows 1 and 2 share 4 and 6, rows 1 and 3 share 5 and 7: exactly 4 four-cycles (each pair overlaps in 2 positions, $\binom{2}{2} = 1$ cycle each), girth 4.

Problem 14.4. (Edge perspective) An ensemble has $L(x) = 0.5x^2 + 0.3x^3 + 0.2x^6$ and all check nodes of degree 6. Find $\lambda(x)$, $\rho(x)$ and the design rate.

Solution $\sum iL_i = 1 + 0.9 + 1.2 = 3.1$ edges per variable node. $\lambda_2 = 1/3.1 = 0.3226$, $\lambda_3 = 0.9/3.1 = 0.2903$, $\lambda_6 = 1.2/3.1 = 0.3871$; $\lambda(x) = 0.3226x + 0.2903x^2 + 0.3871x^5$, $\rho(x) = x^5$. $\int \lambda = 1/3.1 = 0.3226$, $\int \rho = 1/6$, $R = 1 - 0.1667/0.3226 = 0.483$.

Problem 14.5. (Syndrome and bit flipping) The toy code receives $\mathbf{r} = (1, 1, 0, 1, 1, 0)$. Find the syndrome and run bit flipping.

Solution $\mathbf{s} = (1 \oplus 1 \oplus 1,\ 1 \oplus 0 \oplus 1,\ 1 \oplus 1 \oplus 0,\ 0 \oplus 1 \oplus 0) = (1, 0, 0, 1)$. Failed checks per bit: $(1, 1, 1, 2, 0, 1)$; only bit 3 has the maximum 2. Flip it: $(1, 1, 0, 0, 1, 0) = 110010$, a codeword, after one iteration.

Problem 14.6. (Peeling) The toy code receives $(1, ?, ?, 0, ?, 1)$ over a BEC. Decode.

Solution Round 1: $c_0 = \{0, 1, 3\}$ has one erasure: $c_1 = 1 \oplus 0 = 1$. $c_2 = \{0, 4, 5\}$: $c_4 = 1 \oplus 1 = 0$. $c_3 = \{2, 3, 5\}$: $c_2 = 0 \oplus 1 = 1$. (c_1 had three erasures and was not used.) Result $(1, 1, 1, 0, 0, 1)$ in one round.

14.2 Intermediate level

Problem 14.7. (Check-node outputs) A degree-4 check receives $(1.5, -2.0, 0.8, 3.0)$. Compute all four outputs with SPA and with min-sum.

Solution tanh of halves: $(0.6351, -0.7616, 0.3799, 0.9051)$.
To edge 0: $2\tanh^{-1}\left((-0.7616)(0.3799)(0.9051)\right) = 2\tanh^{-1}(-0.2619) = -0.536$; MS: -0.8.
To edge 1: $2\tanh^{-1}(0.6351 \cdot 0.3799 \cdot 0.9051) = +0.444$; MS: $+0.8$.
To edge 2: $2\tanh^{-1}(0.6351 \cdot (-0.7616) \cdot 0.9051) = -0.939$; MS: -1.5.
To edge 3: $2\tanh^{-1}(0.6351 \cdot (-0.7616) \cdot 0.3799) = -0.372$; MS: -0.8.
Every MS magnitude is larger (over-confidence).

Problem 14.8. (Variable-node outputs) A degree-3 variable node has $L^{\text{ch}} = 0.6$ and incoming check messages $(-1.1, 0.4, 2.0)$. Find L^{app}, the decision and the three outgoing messages.

Solution $L^{\text{app}} = 0.6 - 1.1 + 0.4 + 2.0 = 1.9 \Rightarrow \hat{c} = 0$. Outgoing: $1.9 - (-1.1) = 3.0$, $1.9 - 0.4 = 1.5$, $1.9 - 2.0 = -0.1$. The third check is told "probably 1", because the other evidence is weak without its own strong vote.

Problem 14.9. (One sum-product iteration) The toy code receives the LLRs below. Run one SPA iteration.

$$\mathbf{L} = (-0.5,\ 2.5,\ -1.5,\ 1.0,\ -2.0,\ 1.5)$$

Solution Hard decision $(1, 0, 1, 0, 1, 0)$, syndrome $(1, 0, 0, 1)$: not a codeword. Check messages,

grouped by check:

$$\begin{aligned} c_0 \to (v_0, v_1, v_3): &\ (0.828, -0.227, -0.422), \\ c_1 \to (v_1, v_2, v_4): &\ (1.056, -1.537, -1.205), \\ c_2 \to (v_0, v_4, v_5): &\ (-1.056, -0.314, 0.378), \\ c_3 \to (v_2, v_3, v_5): &\ (0.605, -0.855, -0.605). \end{aligned}$$

$\mathbf{L}^{\text{app}} = (-0.727,\ 3.328,\ -2.432,\ -0.277,\ -3.519,\ 1.273)$, decision $(1,0,1,1,1,0) = 101110$: a codeword (syndrome zero), found in one iteration. Bit 3 changed sign: its two checks both said "1".

Problem 14.10. (Normalised min-sum) Repeat the previous problem with NMS, $\alpha = 0.75$.

Solution Check messages 0.75×MS, grouped by check as above:

$$\begin{aligned} &(0.75, -0.375, -0.375), \quad (1.125, -1.5, -1.125), \\ &(-1.125, -0.375, 0.375), \quad (0.75, -1.125, -0.75). \end{aligned}$$

$\mathbf{L}^{\text{app}} = (-0.875, 3.25, -2.25, -0.5, -3.5, 1.125)$, the same codeword 101110 in one iteration.

Problem 14.11. (A bad QC base matrix) Lift $\mathbf{V} = \begin{bmatrix} 0 & 2 & 4 \\ 1 & 3 & 5 \end{bmatrix}$ with $Z = 6$. Is there a 4-cycle?

Solution For columns (i, j): $s_{0i} - s_{1i} + s_{1j} - s_{0j}$: $(0,1)$: $0-1+3-2 = 0$; $(0,2)$: $0-1+5-4 = 0$; $(1,2)$: $2-3+5-4 = 0$. All three column pairs close 4-cycles, each lifted $Z = 6$ times: 18 four-cycles (confirmed by `count_4cycles`). Rows that differ by the same constant in every column are a classic mistake.

Problem 14.12. (5G parameters) Find the base graph, Z, i_{LS} and the number of filler bits for $K = 200$ data+CRC bits and $E = 600$.

Solution $K \le 292$: BG2. $192 < K \le 560$: $k_b = 8$; smallest Z with $8Z \ge 200$ is $Z = 26$ (set $i_{LS} = 6$). Encoder input $10 \cdot 26 = 260$ bits, so 60 filler bits. Transmitted bits come from base columns up to 27, so the decoder uses 18 base rows.

Problem 14.13. (Dual-diagonal encoding) A staircase parity part gives $\lambda = (0, 1, 1, 0, 1, 1)$. Find the parity bits.

Solution $p_0 = 0$, $p_1 = 0 \oplus 1 = 1$, $p_2 = 1 \oplus 1 = 0$, $p_3 = 0 \oplus 0 = 0$, $p_4 = 0 \oplus 1 = 1$, $p_5 = 1 \oplus 1 = 0$: $\mathbf{p} = (0, 1, 0, 0, 1, 0)$.

Problem 14.14. (Gallager rank) A $(4, 8)$ Gallager matrix has $n = 400$. Give m and bounds on the rank and the true rate.

Solution $m = 4 \cdot 400/8 = 200$. The four bands each sum to the all-ones row, so at least 3 rows are dependent: rank ≤ 197, $k \ge 203$, $R \ge 0.5075$ (design rate 0.5).

Problem 14.15. (Layered schedule) Why can the two checks $c_0 = \{v_0, v_1, v_3\}$ and $c_2 = \{v_0, v_4, v_5\}$ not be in one layer? Which toy checks can share a layer?

Solution They share v_0, so updating both from the same $L_{v_0}^{\text{app}}$ would lose one update. In the toy code every pair of checks shares exactly one variable (rows overlap in one position), so no two checks can share a layer: 4 layers.

14.3 Advanced level

Problem 14.16. (BEC density evolution) For the $(4, 8)$ ensemble at $\epsilon = 0.35$ compute x_1, x_2. Will decoding succeed?

Solution $x_1 = 0.35\,(1 - 0.65^7)^3 = 0.35\,(0.9510)^3 = 0.3010$; $x_2 = 0.35\,(1 - 0.6990^7)^3 = 0.2712$. Since $0.35 < \epsilon^* = 0.3834$, the sequence goes to 0 (the library reaches 3×10^{-29}).

Problem 14.17. (Stability) With $\rho(x) = x^7$, what is the largest λ_2 for a threshold of 0.47?

Solution $\rho'(1) = 7$, so $\lambda_2 < 1/(0.47 \cdot 7) = 0.304$.

Problem 14.18. (Optimised ensemble) Verify that $\lambda(x) = 0.4188x + 0.1624x^2 + 0.4188x^5$ with $\rho(x) = x^5$ has rate $1/2$ and satisfies stability at $\epsilon = 0.4757$.

Solution $\int \lambda = 0.2094 + 0.0541 + 0.0698 = 0.3333$; $\int \rho = 1/6$; $R = 1 - 0.1667/0.3333 = 0.5$. Stability: $0.4757 \cdot 0.4188 \cdot 5 = 0.996 < 1$ — just satisfied: the optimiser used as many degree-2 nodes as allowed.

Problem 14.19. (Gaussian approximation, first step) $(3, 6)$ ensemble, $\sigma = 0.9$. Compute the channel mean m_0, $\varphi(m_0)$, the mean of the first check messages and the next variable mean.

Solution $m_0 = 2/0.81 = 2.469$. Quadrature gives $\varphi(2.469) = 0.3822$. Check: $m_u = \varphi^{-1}(1 - (1 - 0.3822)^5) = \varphi^{-1}(0.9100) = 0.197$. Variables: $m_v = 2.469 + 2(0.197) = 2.863$. The mean grows, but slowly: $\sigma = 0.9$ is above the GA threshold 0.872, and the iteration stalls at a fixed point.

Problem 14.20. (EXIT on the BEC) $(3, 6)$, $\epsilon = 0.4$. A-priori information $I_A = 0.5$ enters a variable node. What leaves it, and what does the check node then return?

Solution $I_{E,V} = 1 - 0.4\,(1 - 0.5)^2 = 0.9$; then $I_{E,C} = 0.9^5 = 0.590 > 0.5$: the information grew in one round trip, so the tunnel is open at this point.

Problem 14.21. (Error floor estimate) A code on the BEC has exactly 3 stopping sets of size 2 and no other small ones. Estimate the FER floor at $\epsilon = 0.05$.

Solution Each is fully erased with probability $\epsilon^2 = 0.0025$; union bound $3 \cdot 0.0025 = 0.0075$. The floor is about 7.5×10^{-3}, independent of how good the rest of the code is.

Problem 14.22. (HARQ effective rate) $K = 1024$ bits are first sent with $E = 1152$ bits ($R = 0.889$, BG1). The retransmission sends 1152 new parity bits. What is the effective rate after combining?

Solution $1024/(1152+1152) = 0.444$. Note that the base graph chosen at the first transmission (BG1) stays the same; the extension rows supply the new parity.

Problem 14.23. (Segmentation) A transport block has $A = 19\,992$ bits and $R = 0.5$. Find the TB CRC, the base graph, the number of code blocks C, the bits per block K', Z and the filler bits.

Solution $A > 3824$: CRC24A, $B = 20\,016$. $A > 3824$ and $R > 0.25$: BG1, $K_{cb} = 8448$. $C = \lceil 20\,016/8424 \rceil = 3$. With a 24-bit CRC per block, $B' = 20\,016 + 72 = 20\,088$ and $K' = B'/3 = 6696$. Z: smallest with $22Z \geq 6696$ is $Z = 320$ ($i_{LS} = 2$). Encoder input $22 \cdot 320 = 7040$, so 344 filler bits per block.

Problem 14.24. (Decoder throughput, a design estimate) A layered decoder processes one block row of BG1 with $Z = 384$ per 2 clock cycles at 1 GHz and runs 10 iterations over all 46 rows ($R = 1/3$, $K = 8448$). Estimate the information throughput.

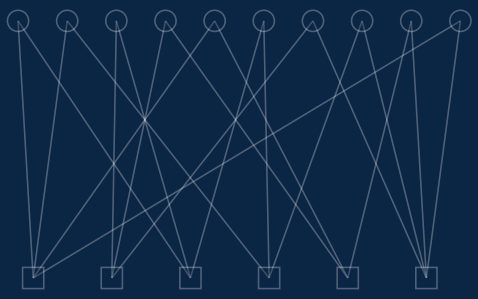
Solution Cycles per codeword: $10 \cdot 46 \cdot 2 = 920$, i.e. 0.92 μs. Throughput $\approx 8448/0.92\,\mu\text{s} = 9.2$ Gbit/s. At rate 8/9 only 5 rows are active and the same hardware gives about $10 \cdot 5 \cdot 2 = 100$ cycles per block: much higher throughput. (These are illustrative assumptions, not a specific chip.)

14.4 Summary of methods

Rate	$k = n - \operatorname{rank} \mathbf{H}$; design rate $1 - \int \rho / \int \lambda$
Cycles	4-cycle $\iff$ two rows share two columns; QC: $s_{ai} - s_{bi} + s_{bj} - s_{aj} \equiv 0 \pmod Z$
Decoding by hand	check: box-plus of the others; variable: $L^{\text{app}} - L_{c \to v}$; stop at zero syndrome
Thresholds	iterate DE / GA / EXIT; bisection on the channel parameter
5G NR	BG rule from (A, R); k_b and Z from K; fillers $= 22Z$ or $10Z$ minus K'

CHAPTER

15



Interview and Examination Questions

IN THIS CHAPTER YOU WILL LEARN

Thirty-two questions with model answers, in four levels: beginner, intermediate, advanced and engineering interview. The answers are written the way an examiner would like to read them: short, precise, with the key formula or number.

15.1 Beginner level

Problem 15.1. What is an LDPC code, in one sentence?

Solution A linear block code defined by a sparse parity-check matrix $\mathbf{H}$: every codeword satisfies $\mathbf{H}\mathbf{c}^T = \mathbf{0}$, and each row and column of $\mathbf{H}$ has only a few ones.

Problem 15.2. What do the variable nodes and check nodes of a Tanner graph represent?

Solution Variable nodes are the code bits (columns of $\mathbf{H}$); check nodes are the parity equations (rows of $\mathbf{H}$); an edge is a one in $\mathbf{H}$.

Problem 15.3. What is a $(3, 6)$-regular code, and what is its design rate?

Solution Every bit is in 3 checks and every check has 6 bits; design rate $1 - 3/6 = 1/2$.

Problem 15.4. Why is the true rate sometimes higher than $1 - m/n$?

Solution Because some rows of $\mathbf{H}$ can be linearly dependent: $k = n - \operatorname{rank}_2 \mathbf{H} \geq n - m$.

Problem 15.5. What is the syndrome, and what does a zero syndrome mean?

Solution $\mathbf{s} = \mathbf{H}\hat{\mathbf{c}}^T$; zero means $\hat{\mathbf{c}}$ is a codeword (not necessarily the transmitted one).

Problem 15.6. Who invented LDPC codes, and who rediscovered them?

Solution R. G. Gallager (1962); rediscovered by MacKay and Neal in 1995–1996.

Problem 15.7. Which parts of 5G NR use LDPC codes?

Solution The data transport channels DL-SCH and UL-SCH (on PDSCH and PUSCH) and the paging channel PCH; control and broadcast use polar codes.

Problem 15.8. Write the BPSK/AWGN channel LLR.

Solution $L = 2y/\sigma^2$ with $0 \mapsto +1$; positive values favour bit 0.

15.2 Intermediate level

Problem 15.9. State the variable-node and the check-node update of the sum-product algorithm.

Solution $L_{v\to c} = L_v^{\mathrm{ch}} + \sum_{c'\neq c} L_{c'\to v}$; $\tanh\frac{L_{c\to v}}{2} = \prod_{v'\neq v}\tanh\frac{L_{v'\to c}}{2}$.

Problem 15.10. What is extrinsic information and why must it be used?

Solution The message on an edge excludes what was received on that edge. Otherwise a node's own belief comes back and is counted twice, making the decoder over-confident and wrong.

Problem 15.11. Derive the check rule for two bits in one line.

Solution $P(b_1 \oplus b_2 = 0) - P(b_1 \oplus b_2 = 1) = (P_1(0) - P_1(1))(P_2(0) - P_2(1))$ and $P(0) - P(1) = \tanh(L/2)$.

Problem 15.12. What is the min-sum approximation and its main weakness?

Solution $L_{c\to v} \approx \prod \mathrm{sign} \cdot \min |L|$ over the other inputs; it over-estimates the reliability. Normalisation ($\times\alpha$, $\alpha \approx 0.75$–0.8) or an offset ($-\beta$, $\beta \approx 0.5$) corrects it.

Problem 15.13. What is a 4-cycle and how do you detect it in $\mathbf{H}$?

Solution A closed path $v_a - c_j - v_b - c_l - v_a$; it exists iff two rows share ones in two columns. Count with the overlaps of $\mathbf{H}\mathbf{H}^T$.

Problem 15.14. What is a stopping set?

Solution A set of variable nodes such that every neighbouring check is connected to it at least twice. The peeling decoder fails exactly when the erasures contain one.

Problem 15.15. Why is a generator matrix rarely used to encode long LDPC codes?

Solution It is dense: $O(kn)$ operations and storage. Structured parity parts (dual-diagonal, ALT) give linear-time encoding.

Problem 15.16. What is a quasi-cyclic LDPC code?

Solution A code whose $\mathbf{H}$ consists of $Z \times Z$ circulant permutation (or zero) blocks given by a small base matrix of shifts; storage and hardware scale with the base matrix, and one base matrix serves many lengths.

15.3 Advanced level

Problem 15.17. What does density evolution compute, and under which assumptions?

Solution The distribution of the messages per iteration for the ensemble (λ, ρ) as $n \to \infty$, assuming tree-like neighbourhoods, concentration around the ensemble average and channel/decoder symmetry (all-zero codeword). It gives the threshold.

Problem 15.18. Give the BEC density-evolution recursion and the $(3, 6)$ threshold.

Solution $x_{\ell+1} = \epsilon\lambda(1 - \rho(1 - x_\ell))$; $\epsilon^* = 0.4294$ (capacity limit 0.5).

Problem 15.19. What is the stability condition and why does it limit degree-2 nodes?

Solution Near zero error, $x_{\ell+1} \approx \epsilon\lambda_2\rho'(1)x_\ell$; convergence needs $\epsilon\lambda_2\rho'(1) < 1$ (BEC) or $\lambda_2\rho'(1) < e^{1/(2\sigma^2)}$ (AWGN). Too many degree-2 nodes make the slope larger than one.

Problem 15.20. Why are irregular codes better than regular ones?

Solution High-degree variable nodes converge fast and help the low-degree ones (a "wave"); optimised mixtures move the threshold close to capacity, e.g. from 0.4294 to 0.4876 on the BEC at rate 1/2 in Chapter 10.

Problem 15.21. Explain an EXIT chart in three sentences.

Solution It plots the extrinsic mutual information of the variable-node decoder against its a-priori input, together with the inverse curve of the check-node decoder. Decoding is a staircase between the curves. It converges iff there is an open tunnel up to $I = 1$.

Problem 15.22. What causes an error floor, and why does DE not predict it?

Solution Small harmful sub-graphs (stopping sets, trapping sets, low-weight codewords) that rare noise patterns hit. DE assumes infinite length and no cycles, so these finite structures are invisible to it.

Problem 15.23. How does layered decoding differ from flooding, and what is gained?

Solution Beliefs are updated after each layer of non-overlapping checks, so later layers use new information in the same iteration. It needs about 40% fewer iterations (Chapter 9) and less memory.

Problem 15.24. Why is BP exact on a cycle-free graph but not on a graph with cycles?

Solution On a tree all incoming messages depend on disjoint sets of channel samples, so they are independent; cycles create dependence and double counting.

15.4 Engineering interview level

Problem 15.25. Why did 3GPP choose LDPC for 5G data channels?

Solution Very high throughput through parallel layered decoding, excellent performance for medium and long blocks, flexible lengths and rates by lifting and circular-buffer rate matching, and natural HARQ incremental redundancy (degree-1 extension parity).

Problem 15.26. What are BG1 and BG2, and how is one chosen?

Solution Two QC base graphs, 46×68 (BG1, large blocks, rates $1/3$ to about 0.92) and 42×52 (BG2, small blocks, rates down to $1/5$). BG2 if $A \leq 292$, or $A \leq 3824$ and $R \leq 0.67$, or $R \leq 0.25$; otherwise BG1.

Problem 15.27. Why are the first $2Z$ systematic bits punctured in NR?

Solution They belong to the two highest-degree columns. Not sending them raises the rate at almost no cost and was found to improve the threshold of the protograph; the decoder recovers them through their many checks.

Problem 15.28. How would you choose the normalisation factor of an NMS decoder?

Solution By density evolution for the target degree distribution, or by simulation of the real code and iteration limit at the target FER; we found $\alpha = 0.75$–0.8 best for a $(3, \cdot)$ PEG code at 2 dB. Prefer values that are cheap in hardware, such as 0.75.

Problem 15.29. A decoder works in floating point but fails in fixed point. What do you check?

Solution The LLR scaling and saturation (clipping at too small a value), the bit width of the APP memory (it must hold sums of several messages), rounding in the normalisation, and sign handling of zero.

Problem 15.30. How do you measure an error floor at 10^{-9} FER?

Solution Not by plain Monte-Carlo on a CPU (it needs about 10^{11} frames). Use FPGA or GPU emulation, importance sampling around the dominant trapping sets, or analytical estimates from enumerated trapping sets.

Problem 15.31. Compare LDPC and polar codes for a 500-bit block at rate $1/2$.

Solution At this length CA-SCL polar decoding ($L = 8$) and 5G LDPC BP perform about the same: in Chapter 13, at $N = 512$, $R = 1/2$, they are within 0.1 dB of each other. LDPC decoders are more parallel and support HARQ-IR more naturally; polar decoders need no iterations. Always state length, rate, decoder and iterations or list size.

Problem 15.32. How would you speed up the Python simulator of this book by 10–100 times?

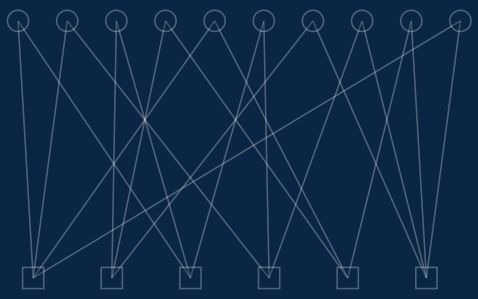
Solution Compile the check-node update (Numba or C), use `float32` and larger batches, run frames in parallel processes, or port to a GPU array library; keep the edge-list structure.

15.5 How to prepare

- Re-derive the two update rules and the BEC density-evolution recursion without notes.
- Decode the toy code of Chapter 8 by hand, with SPA and min-sum.
- Memorise a few anchor numbers: $(3, 6)$ thresholds 0.4294 (BEC) and ≈ 1.1 dB (AWGN); Shannon limit 0.19 dB at $R = 1/2$; BG1 46×68, BG2 42×52, $Z \leq 384$, $K \leq 8448$.
- Run `demo.py` and change one parameter at a time.

CHAPTER

16



Final Project: A 5G NR LDPC Link with HARQ

IN THIS CHAPTER YOU WILL LEARN

- a complete transport-block link in about 150 lines;
- the 5G CRC polynomials and a fast linear CRC;
- code-block segmentation and filler bits in practice;
- circular-buffer rate matching with redundancy versions;
- HARQ incremental redundancy with soft combining;
- real results and how to read them;
- extensions for students and researchers.

16.1 Project specification

Build a simulator that sends 5G NR transport blocks over BPSK/AWGN and measures the transport-block error rate (TBER) after the first transmission and after one HARQ retransmission:

1. take A payload bits and a target code rate R;
2. attach the transport-block CRC (CRC24A or CRC16);
3. select the base graph (Chapter 11) and segment into C code blocks with CRC24B when needed;
4. encode each code block with the 5G LDPC encoder (filler bits included);
5. rate-match with redundancy version 0 (first transmission) and 2 (retransmission);
6. send BPSK over AWGN, compute LLRs, combine the LLRs of both transmissions in a soft buffer;
7. decode each code block with layered NMS (25 iterations), check the CB and TB CRCs.

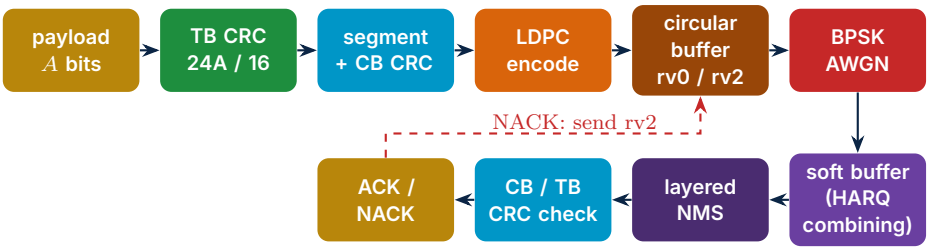


Figure 16.1. Architecture of the final project.

16.2 Design decisions

- **Fast CRC.** A CRC is linear: $\mathrm{crc}(\mathbf{m}) = \mathbf{mP}$. Row i of $\mathbf{P}$ is $x^{r+k-1-i} \bmod g(x)$, computed by repeated multiplication by x in $O(kr)$ operations. The program checks it against plain long division for all three polynomials (CRC24A: $D^{24} + D^{23} + D^{18} + D^{17} + D^{14} + D^{11} + D^{10} + D^7 + D^6 + D^5 + D^4 + D^3 + D + 1$, CRC24B: $D^{24} + D^{23} + D^6 + D^5 + D + 1$, CRC16: $D^{16} + D^{12} + D^5 + 1$).
- **Mother code with all rows.** A retransmission sends parity bits that the first transmission did not use, so the decoder keeps the whole base graph (all 46 or 42 block rows); untransmitted bits simply have LLR 0.
- **Circular buffer.** $N_{cb} = N = 66Z$ or $50Z$, starting positions $k_0 = \{0, 17, 33, 56\}Z$ (BG1) or $\{0, 13, 25, 43\}Z$ (BG2), filler bits skipped.
- **Energy.** Every transmission uses the same E_b/N_0 (defined with the first-transmission rate R), so the retransmission doubles the received energy — as in a real HARQ process.

16.3 The complete program

project.py

```python
"""Final project: a 5G NR LDPC transport-block link with HARQ (Chapter 16).

Transport block -> TB CRC -> base graph selection -> code-block segmentation
(+ CB CRC) -> LDPC encoding -> rate matching (RV) -> BPSK/AWGN -> LLR ->
de-rate-matching (+ HARQ combining) -> layered NMS decoding -> CRC checks.
"""
import json, numpy as np
from ldpc import *

# ------------------------------------------------------------------ CRCs (TS 38.212, 5.1)
CRC_POLY = {"CRC24A": [24, 23, 18, 17, 14, 11, 10, 7, 6, 5, 4, 3, 1, 0],
            "CRC24B": [24, 23, 6, 5, 1, 0],
            "CRC16":  [16, 12, 5, 0]}


class CRC:
    """crc(m) = m P (mod 2). Row i of P is x^(r + k-1-i) mod g(x), built by repeated
    multiplication by x, so the set-up costs only O(k r)."""

    def __init__(self, name, k):
```

```
        exps = CRC_POLY[name]
        self.r, self.k = exps[0], k
        g = np.zeros(self.r + 1, np.uint8)
        g[[self.r - e for e in exps]] = 1                # g[0] = coefficient of x^r
        self.P = np.zeros((k, self.r), np.uint8)
        cur = g[1:].copy()                                # x^r mod g
        for i in range(k - 1, -1, -1):
            self.P[i] = cur
            top = cur[0]
            cur = np.r_[cur[1:], 0]                       # multiply by x
            if top:
                cur ^= g[1:]

    def attach(self, m):
        return np.concatenate([m, (m.astype(np.int64) @ self.P) % 2], axis=1).astype(np.
    uint8)

    def check(self, w):
        return ((w[:, :self.k].astype(np.int64) @ self.P) % 2 == w[:, self.k:]).all(axis=1)


def crc_long_division(m, name):
    """Reference CRC by polynomial long division (used to test the class above)."""
    exps = CRC_POLY[name]; r = exps[0]
    g = np.zeros(r + 1, np.uint8); g[[r - e for e in exps]] = 1
    reg = np.r_[np.array(m, np.uint8), np.zeros(r, np.uint8)]
    for i in range(len(m)):
        if reg[i]:
            reg[i:i + r + 1] ^= g
    return reg[-r:]


# ---------------------------------------------------------------- the transport-block link
class TBLink:
    def __init__(self, A, R):
        self.A, self.R = A, R
        self.tbcrc = CRC("CRC24A" if A > 3824 else "CRC16", A)
        B = A + self.tbcrc.r
        self.bg = 2 if (A <= 292 or (A <= 3824 and R <= 0.67) or R <= 0.25) else 1
        Kcb = 8448 if self.bg == 1 else 3840
        self.C = 1 if B <= Kcb else int(np.ceil(B / (Kcb - 24)))
        self.L_cb = 24 if self.C > 1 else 0               # CB CRC only when segmented
        assert B % self.C == 0, "choose A so that B is a multiple of C"
        self.Kp = B // self.C + self.L_cb                 # bits per code block (K')
        self.cbcrc = CRC("CRC24B", self.Kp - 24) if self.C > 1 else None
        self.G = int(round(A / R))                        # coded bits of the transport
    block
        self.Er = self.G // self.C                        # coded bits per code block
        self.nr = NRCode(self.Kp, 10 ** 6, bg=self.bg)    # mother code, all rows (HARQ)
        make_layers(self.nr.code, self.nr.Z)
        Z = self.nr.Z
        self.N = (66 if self.bg == 1 else 50) * Z         # circular buffer length N_cb =
    N
        buf = np.arange(2 * Z, 2 * Z + self.N)            # d_k is code bit 2Z + k
```

```
        self.buffer = buf[~np.isin(buf, self.nr.filler)]   # filler bits are skipped
        f = (0, 17, 33, 56) if self.bg == 1 else (0, 13, 25, 43)
        self.k0 = {rv: f[rv] * Z for rv in range(4)}       # floor(f N_cb / N) Z with N_cb
    = N

    def positions(self, rv):
        """Code-bit positions sent with redundancy version rv."""
        start = np.searchsorted(self.buffer, 2 * self.nr.Z + self.k0[rv])
        return self.buffer[(start + np.arange(self.Er)) % len(self.buffer)]

    def encode(self, a):
        b = self.tbcrc.attach(a)
        n_data = self.Kp - self.L_cb
        cws = []
        for r in range(self.C):
            seg = b[:, r * n_data:(r + 1) * n_data]
            cws.append(self.nr.encode(self.cbcrc.attach(seg) if self.cbcrc else seg)[0])
        return cws

    def decode(self, llrs, max_iter=25):
        """llrs: one full-length LLR array per code block. Returns TB-ok flags,
        decoded payload and average iterations per code block."""
        data, ok_cb, its = [], [], 0
        for L in llrs:
            L = L.copy(); L[:, self.nr.filler] = 1e3        # known filler bits
            x, it, _ = layered_decode(self.nr.code, L, max_iter, "nms")
            its = its + it
            cb = x[:, :self.Kp]
            ok_cb.append(self.cbcrc.check(cb) if self.cbcrc else np.ones(len(cb), bool))
            data.append(cb[:, :self.Kp - self.L_cb])
        b = np.concatenate(data, axis=1)
        ok = self.tbcrc.check(b) & np.all(ok_cb, axis=0)
        return ok, b[:, :self.A], its / self.C


def run(A, R, ebn0_list, frames=1000, batch=100, seed=7, rvs=(0, 2)):
    link = TBLink(A, R); rng = np.random.default_rng(seed)
    res = {"ebn0": list(ebn0_list), "tber1": [], "tber2": [], "ber1": [], "it1": []}
    for eb in ebn0_list:
        e1 = e2 = b1 = it1 = n = 0
        while n < frames:
            a = rng.integers(0, 2, (batch, A), dtype=np.uint8)
            cws = link.encode(a)
            soft = [np.zeros((batch, link.nr.n)) for _ in cws]       # HARQ soft buffers
            for t, rv in enumerate(rvs):
                pos = link.positions(rv)
                for i, c in enumerate(cws):
                    np.add.at(soft[i], (slice(None), pos), awgn_llr(c[:, pos], eb, R, rng))
                ok, ahat, its = link.decode(soft)
                if t == 0:
                    e1 += (~ok).sum(); b1 += (ahat != a).sum(); it1 += its.sum()
                else:
                    e2 += (~ok).sum()
            n += batch
```

```
        res["tber1"].append(e1 / n); res["tber2"].append(e2 / n)
        res["ber1"].append(b1 / (n * A)); res["it1"].append(it1 / n)
        print(f"A={A:5d} R={R:.3f} Eb/N0={eb:4.2f} dB: TB error 1st tx {e1 / n:.4f}, "
              f"after rv0+rv2 {e2 / n:.4f}, BER {b1 / (n * A):.2e}, avg it {it1 / n:.1f}",
    flush=True)
    return res


if __name__ == "__main__":
    rng = np.random.default_rng(1)
    for name, k in [("CRC24A", 50), ("CRC24B", 30), ("CRC16", 40)]:
        m = rng.integers(0, 2, (1, k), dtype=np.uint8)
        assert (CRC(name, k).attach(m)[0, k:] == crc_long_division(m[0], name)).all()
    print("CRC24A, CRC24B and CRC16 agree with polynomial long division")
    for A, R in [(12000, 2 / 3), (1000, 1 / 2)]:
        link = TBLink(A, R)
        print(f"A = {A}, R = {R:.3f}: TB CRC {link.tbcrc.r} bits, BG{link.bg}, C = {link.C}
     code block(s) of "
              f"K' = {link.Kp} bits, Z = {link.nr.Z} (iLS {link.nr.ils}), fillers {link.nr.
    Kp - link.Kp}, "
              f"G = {link.G}, E_r = {link.Er}, k0 = {list(link.k0.values())}")
        cws = link.encode(rng.integers(0, 2, (2, A), dtype=np.uint8))
        print("   H c^T = 0 for every code block:", all(not link.nr.code.syndrome(c).any()
    for c in cws),
              "| rv0 and rv2 overlap in", len(np.intersect1d(link.positions(0), link.
    positions(2))), "positions")
    out = {"A1000": run(1000, 1 / 2, [0.5, 0.75, 1.0, 1.25, 1.5, 1.75, 2.0], frames=2000,
    batch=200),
           "A12000": run(12000, 2 / 3, [1.5, 1.75, 2.0, 2.25, 2.5], frames=400, batch=50),
           "A1000_low": run(1000, 1 / 2, [-3.25, -3.0, -2.75, -2.5, -2.25, -2.0], frames
    =1000, batch=200)}
    json.dump(out, open("../data/project.json", "w"), indent=1)
```

16.4 Results

OUTPUT (real run)

```
CRC24A, CRC24B and CRC16 agree with polynomial long division
A = 12000, R = 0.667: TB CRC 24 bits, BG1, C = 2 code block(s) of K' = 6036 bits, Z = 288 (iLS 4),
     fillers 300, G = 18000, E_r = 9000, k0 = [0, 4896, 9504, 16128]
   H c^T = 0 for every code block: True | rv0 and rv2 overlap in 0 positions
A = 1000, R = 0.500: TB CRC 16 bits, BG2, C = 1 code block(s) of K' = 1016 bits, Z = 104 (iLS 6),
     fillers 24, G = 2000, E_r = 2000, k0 = [0, 1352, 2600, 4472]
   H c^T = 0 for every code block: True | rv0 and rv2 overlap in 0 positions
A= 1000 R=0.500 Eb/N0=0.50 dB: TB error 1st tx 0.9810, after rv0+rv2 0.0000, BER 1.71e-01, avg it 24.9
A= 1000 R=0.500 Eb/N0=0.75 dB: TB error 1st tx 0.8315, after rv0+rv2 0.0000, BER 1.22e-01, avg it 23.9
A= 1000 R=0.500 Eb/N0=1.00 dB: TB error 1st tx 0.4915, after rv0+rv2 0.0000, BER 6.04e-02, avg it 20.7
A= 1000 R=0.500 Eb/N0=1.25 dB: TB error 1st tx 0.1495, after rv0+rv2 0.0000, BER 1.66e-02, avg it 15.0
A= 1000 R=0.500 Eb/N0=1.50 dB: TB error 1st tx 0.0220, after rv0+rv2 0.0000, BER 2.12e-03, avg it 10.6
A= 1000 R=0.500 Eb/N0=1.75 dB: TB error 1st tx 0.0010, after rv0+rv2 0.0000, BER 1.43e-04, avg it 8.2
A= 1000 R=0.500 Eb/N0=2.00 dB: TB error 1st tx 0.0000, after rv0+rv2 0.0000, BER 0.00e+00, avg it 6.7
A=12000 R=0.667 Eb/N0=1.50 dB: TB error 1st tx 1.0000, after rv0+rv2 0.0000, BER 7.63e-02, avg it 24.9
A=12000 R=0.667 Eb/N0=1.75 dB: TB error 1st tx 0.6400, after rv0+rv2 0.0000, BER 1.76e-02, avg it 21.2
```

```
A=12000 R=0.667 Eb/N0=2.00 dB: TB error 1st tx 0.0075, after rv0+rv2 0.0000, BER 1.98e-05, avg it 12.6
A=12000 R=0.667 Eb/N0=2.25 dB: TB error 1st tx 0.0000, after rv0+rv2 0.0000, BER 0.00e+00, avg it 9.1
A=12000 R=0.667 Eb/N0=2.50 dB: TB error 1st tx 0.0000, after rv0+rv2 0.0000, BER 0.00e+00, avg it 7.3
A= 1000 R=0.500 Eb/N0=-3.25 dB: TB error 1st tx 0.9990, after rv0+rv2 0.9700, BER 3.05e-01, avg it
     25.0
A= 1000 R=0.500 Eb/N0=-3.00 dB: TB error 1st tx 1.0000, after rv0+rv2 0.8130, BER 2.99e-01, avg it
     25.0
A= 1000 R=0.500 Eb/N0=-2.75 dB: TB error 1st tx 1.0000, after rv0+rv2 0.4220, BER 2.93e-01, avg it
     25.0
A= 1000 R=0.500 Eb/N0=-2.50 dB: TB error 1st tx 1.0000, after rv0+rv2 0.0960, BER 2.88e-01, avg it
     25.0
A= 1000 R=0.500 Eb/N0=-2.25 dB: TB error 1st tx 1.0000, after rv0+rv2 0.0150, BER 2.83e-01, avg it
     25.0
A= 1000 R=0.500 Eb/N0=-2.00 dB: TB error 1st tx 1.0000, after rv0+rv2 0.0000, BER 2.77e-01, avg it
     25.0
```

- **Set-up checks.** For $A = 12\,000$, $R = 2/3$: CRC24A, BG1 (since $A > 3824$ and $R > 0.25$), two code blocks of $K' = 6036$ bits, $Z = 288$ ($i_{LS} = 4$) and 300 filler bits per block; $E_r = 9000$ bits per block. For $A = 1000$, $R = 1/2$: CRC16, BG2, one code block of 1016 bits, $Z = 104$, 24 filler bits. All codewords satisfy $\mathbf{H}\mathbf{c}^T = \mathbf{0}$, and rv0 and rv2 share no position: the retransmission is pure incremental redundancy.
- **First transmission, $A = 1000$, $R = 1/2$.** The TBER falls from 0.49 at 1.0 dB to 0.022 at 1.5 dB and 0.001 at 1.75 dB, with 10.6 and 8.2 iterations on average (25 at most). This agrees with the single-code-block results of Chapter 13 (the extra 16 CRC bits cost very little).
- **First transmission, $A = 12\,000$, $R = 2/3$.** The waterfall is much steeper: TBER 0.64 at 1.75 dB but 0.0075 at 2.0 dB and no error at 2.25 dB. Two long code blocks of 6036 bits behave like one very long code, and the TB fails if *either* block fails.
- **After HARQ.** With rv2 added, no transport block failed at any of these E_b/N_0 values: the combined code has rate 1/4 (or 1/3) and twice the energy. To see the HARQ waterfall we ran $A = 1000$ down to -3.25 dB per transmission (Fig. 16.2): after combining, the TBER is 0.42 at -2.75 dB, 0.096 at -2.5 dB, 0.015 at -2.25 dB and 0 at -2.0 dB. One retransmission moves the operating point by about 3.8 dB.
- **An undetected error.** At -3.25 dB one first transmission (out of 1000) passed the CRC16 check with 294 wrong bits. A failed LDPC decoder outputs an almost random word, and a 16-bit CRC accepts a random word with probability 2^{-16}; with 1000 such frames this happens with probability about 1.5% — rare, but it happened. This is why 5G uses CRC24A for large transport blocks and an extra CRC per code block, and why decoders often stop early only when the syndrome *and* the CRC agree.

16.5 Extensions for students and researchers

1. Add QPSK and 16-QAM with max-log LLRs, and the bit interleaver of TS 38.212 (Section 5.4.2.2).
2. Implement limited-buffer rate matching ($N_{cb} < N$) and all four redundancy versions with the order $0, 2, 3, 1$.

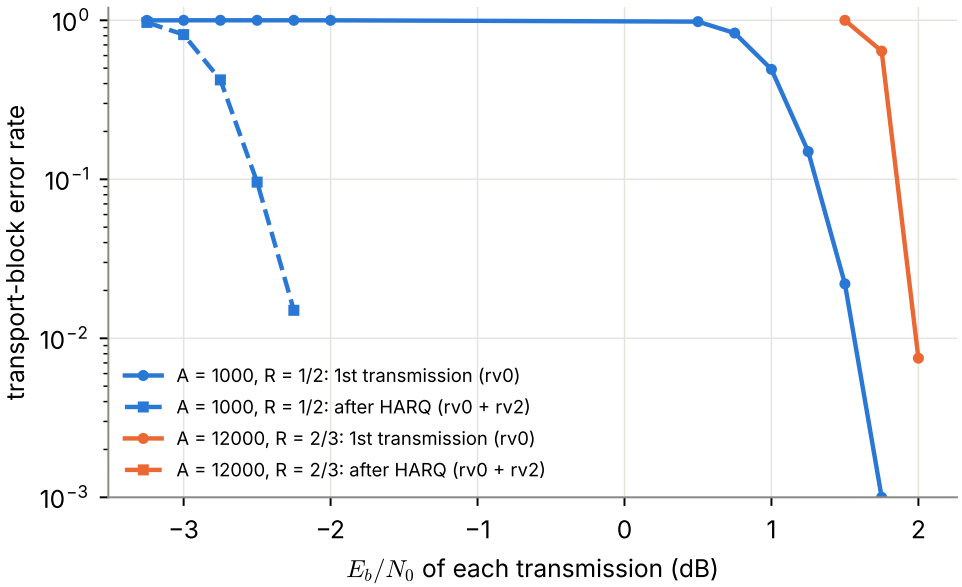


Figure 16.2. Transport-block error rate of the final project: first transmission and after combining rv0 and rv2.

3. Replace layered NMS by layered OMS and by layered SPA; compare TBER and average iterations.
4. Add a fading channel (Rayleigh, independent per symbol) and measure the throughput $= (1 - \text{TBER}) \cdot A$ per transmission.
5. Quantise the LLRs and messages to 6 bits and find the loss.
6. Use `design_bec.py` ideas to design your own protograph, lift it with `qc_lift`, and compare it with BG2.
7. Port the decoder to a GPU array library and measure the speed-up.

16.6 Project checklist

Correctness	CRCs match long division; every codeword satisfies $\mathbf{H}\mathbf{c}^T = \mathbf{0}$; noiseless frames decode
Reproducibility	fixed seeds; parameters printed at the start of every run
Statistics	at least 50–100 errors per point, or say clearly that a point is uncertain
Fairness	compare at the same information rate, same E_b/N_0 definition, same iteration limit
Reporting	TBER and BER, average iterations, decoder and schedule, number of frames

16.7 Final words

LDPC codes are a beautiful example of an idea that waited thirty years for computers to catch up. The ingredients are simple — parity checks, a sparse graph, two local rules — but together they come within a fraction of a decibel of Shannon's limit, and they now carry most of the world's wireless and broadcast data. With Part One (polar codes) and this book (LDPC codes) you know both families of 5G channel codes from the first principle to the working program. The next step is yours: change a parameter, break something, measure, and explain.

Appendix A — Mathematical Prerequisites

This appendix adds to Appendix A of Part One (logarithms, decibels, probability, the Gaussian distribution and the Q-function) the algebra that is special to LDPC codes.

A.1 Arithmetic in GF(2)

$\oplus$	0	1
0	0	1
1	1	0

$\cdot$	0	1
0	0	0
1	0	1

Addition is XOR and multiplication is AND. Every element is its own negative ($a \oplus a = 0$), so subtraction equals addition and "−" signs can be dropped in all GF(2) formulas. A vector $\mathbf{x}$ satisfies the parity check $\mathbf{h}$ when $\mathbf{h} \cdot \mathbf{x}^T = \bigoplus_i h_i x_i = 0$, i.e. when the number of positions with $h_i = x_i = 1$ is even.

A.2 Linear algebra over GF(2)

- **Row operations:** swap two rows; add (XOR) one row to another. There is no scaling (the only non-zero scalar is 1).
- **Rank:** the number of non-zero rows in reduced row echelon form. $\dim(\text{null space}) = n - \text{rank}$.
- **Inverse:** reduce $[\mathbf{A} \mid \mathbf{I}]$ to $[\mathbf{I} \mid \mathbf{A}^{-1}]$; it exists iff $\text{rank}\,\mathbf{A} = n$.
- **Dual code:** the rows of $\mathbf{H}$ span the dual code $\mathcal{C}^{\perp}$; $\mathbf{G}\mathbf{H}^T = \mathbf{0}$.
- **Warning:** a vector can be orthogonal to itself ($\mathbf{x} \cdot \mathbf{x} = 0$ for every even-weight $\mathbf{x}$), so GF(2) has no "length" and no orthogonal projection.

A.3 Permutation and circulant matrices

A **permutation matrix** has exactly one 1 in every row and every column. The **cyclic shift** $\mathbf{P}$ ($Z \times Z$, ones at $(r, (r+1) \bmod Z)$) generates all circulant permutation matrices $\mathbf{P}^s$, with

$$\mathbf{P}^a \mathbf{P}^b = \mathbf{P}^{(a+b) \bmod Z}, \qquad (\mathbf{P}^s)^{-1} = (\mathbf{P}^s)^T = \mathbf{P}^{(Z-s) \bmod Z}, \qquad (\mathbf{P}^s \mathbf{x}^T)_r = x_{(r+s) \bmod Z}.$$

These rules give the 4-cycle condition of QC codes and the encoding trick of Chapter 6: a closed path through blocks $\mathbf{P}^{s_1}, (\mathbf{P}^{s_2})^{-1}, \mathbf{P}^{s_3}, \ldots$ returns to its start iff the alternating sum of the shifts is 0 mod Z.

A.4 Sparse matrices

A sparse matrix is stored as the list of positions of its non-zero entries (coordinate format), or row by row (CSR). For an LDPC code the coordinate list *is* the edge list of the Tanner graph. Products like $\mathbf{H}\mathbf{x}^T$ cost one operation per non-zero entry.

A.5 Hyperbolic functions used in decoding

$$\tanh x = \frac{e^x - e^{-x}}{e^x + e^{-x}}, \qquad \tanh^{-1} y = \frac{1}{2}\ln\frac{1+y}{1-y}, \qquad \phi(x) = -\ln\tanh\frac{x}{2} = \ln\frac{e^x+1}{e^x-1}.$$

x	0.1	0.25	0.5	1	2	3	5	10
$\tanh(x/2)$	0.0500	0.1244	0.2449	0.4621	0.7616	0.9051	0.9866	0.99991
$\phi(x)$	2.9966	2.0846	1.4068	0.7719	0.2723	0.0997	0.0135	0.0001

ϕ is decreasing, convex, $\phi(\phi(x)) = x$, $\phi(x) \approx 2e^{-x}$ for large x and $\phi(x) \approx \ln(2/x)$ for small x.

A.6 Polynomials of degree distributions

For $\lambda(x) = \sum_i \lambda_i x^{i-1}$: $\lambda(1) = 1$, $\int_0^1 \lambda(x)\,dx = \sum_i \lambda_i/i$, $\lambda'(0) = \lambda_2$, $\lambda'(1) = \sum_i (i-1)\lambda_i$. The same holds for ρ.

A.7 Mutual information of a Gaussian LLR

If $L \sim \mathcal{N}(\mu, 2\mu)$ is the LLR of a bit (consistent Gaussian), the mutual information between the bit and L is $J(\sigma) = 1 - \mathbb{E}\left[\log_2(1 + e^{-L})\right]$ with $\sigma^2 = 2\mu$. J increases from 0 to 1; there is no closed form, and the library evaluates it by Gauss–Hermite quadrature.

Appendix B — Python Source Code

This appendix prints the complete library `ldpc.py` and the most important scripts. The line numbers are the ones used in the chapters. All files (and the scripts not printed here: `examples.py`, `solved.py`, `make_figs.py`, `plot_sims.py`, `make_pyref.py`) are in the code archive that accompanies the book.

Requirements

Python 3.9 or newer and NumPy for the library; SciPy for `design_bec.py` and `make_pyref.py`; Matplotlib for the figures. The files `data/5G_bg1.csv` and `data/5G_bg2.csv` must be in the folder `../data` relative to `ldpc.py` (as in the archive). Run `python3 test_ldpc.py` first: all ten tests must pass.

B.1 The library `ldpc.py`

ldpc.py • lines 1--9

```
"""ldpc.py -- a small, complete LDPC library for the book
"LDPC Codes: From Zero to Expert" (Part Two).  Only NumPy is needed.

Conventions (same as Part One): bits are 0/1 integers, BPSK maps 0 -> +1 and
1 -> -1, and an LLR is L = ln P(bit=0|y) / P(bit=1|y), so L > 0 favours 0.
"""
import numpy as np
from pathlib import Path
```

ldpc.py • lines 10--58

```
# ==== [gf2]
# ------------------------------------------------------------------ 1. GF(2) algebra
def gf2_rref(A):
    """Reduced row echelon form over GF(2). Returns (R, pivot_columns)."""
    R = np.array(A, dtype=np.uint8) % 2
    m, n = R.shape
    pivots, row = [], 0
    for col in range(n):
        if row == m:
            break
        hits = np.nonzero(R[row:, col])[0]
        if hits.size == 0:
            continue
```

```
        p = row + hits[0]
        R[[row, p]] = R[[p, row]]                    # swap the pivot row up
        others = np.nonzero(R[:, col])[0]
        others = others[others != row]
        R[others] ^= R[row]                          # clear the column
        pivots.append(col)
        row += 1
    return R[:row], pivots


def gf2_rank(A):
    return len(gf2_rref(A)[1])


def gf2_inv(A):
    """Inverse of a square, invertible GF(2) matrix."""
    n = A.shape[0]
    R, piv = gf2_rref(np.hstack([A % 2, np.eye(n, dtype=np.uint8)]))
    if piv[:n] != list(range(n)):
        raise ValueError("matrix is singular over GF(2)")
    return R[:, n:]


def generator_from_H(H):
    """Systematic-style generator matrix G (k x n) with H G^T = 0.
    Returns (G, info_pos): codeword bits at info_pos are the message bits."""
    R, piv = gf2_rref(H)
    n = H.shape[1]
    free = [j for j in range(n) if j not in set(piv)]
    G = np.zeros((len(free), n), dtype=np.uint8)
    G[:, free] = np.eye(len(free), dtype=np.uint8)
    G[:, piv] = R[:, free].T                         # c[piv_i] = sum_j R[i, free_j] c[
    free_j]
    return G, np.array(free)
# ==== [/gf2]
```

ldpc.py • lines 59--106

```
# ==== [code]
# ------------------------------------------------------------------ 2. the code object
class LDPCCode:
    """An LDPC code stored as the edge list of its Tanner graph.
    Edge e joins check node cn[e] and variable node vn[e]; edges are sorted
    by check node (then variable node)."""

    def __init__(self, m, n, cn, vn, name="LDPC"):
        order = np.lexsort((vn, cn))
        self.m, self.n, self.name = m, n, name
        self.cn, self.vn = np.asarray(cn)[order], np.asarray(vn)[order]
        self.E = len(self.cn)
        self.dc = np.bincount(self.cn, minlength=m)          # check degrees
        self.dv = np.bincount(self.vn, minlength=n)          # variable degrees
        self.cstart = np.r_[0, np.cumsum(self.dc)[:-1]]     # first edge of each check
        self.vperm = np.lexsort((self.cn, self.vn))         # edges sorted by variable
```

```
        self.vstart = np.r_[0, np.cumsum(self.dv)[:-1]]
        self.layers = None

    @classmethod
    def from_H(cls, H, name="LDPC"):
        cn, vn = np.nonzero(np.asarray(H) % 2)
        return cls(H.shape[0], H.shape[1], cn, vn, name)

    def H(self):
        """Dense parity-check matrix (use only for small codes)."""
        H = np.zeros((self.m, self.n), dtype=np.uint8)
        H[self.cn, self.vn] = 1
        return H

    def design_rate(self):
        return 1 - self.m / self.n

    def check_sum(self, x):
        """Per-check sums of edge values x (shape (B, E)) -> (B, m)."""
        return np.add.reduceat(x, self.cstart, axis=1)

    def var_sum(self, x):
        """Per-variable sums of edge values x (shape (B, E)) -> (B, n)."""
        return np.add.reduceat(x[:, self.vperm], self.vstart, axis=1)

    def syndrome(self, c):
        """Syndrome s = H c^T for a batch of words c (shape (B, n))."""
        c = np.atleast_2d(c)
        return self.check_sum(c[:, self.vn].astype(np.int32)) % 2
# ==== [/code]
```

ldpc.py • lines 107--152

```
# ==== [construct]
# ------------------------------------------------------------------ 3. constructions
def gallager_H(n, dv, dc, seed=0):
    """Gallager's (dv, dc)-regular construction: dv stacked bands, each a
    random column permutation of the first band."""
    rng = np.random.default_rng(seed)
    mb = n // dc                                    # checks per band
    band = np.kron(np.eye(mb, dtype=np.uint8), np.ones((1, dc), dtype=np.uint8))
    bands = [band] + [band[:, rng.permutation(n)] for _ in range(dv - 1)]
    return np.vstack(bands)


def peg(n, m, vn_degrees, seed=0):
    """Progressive edge-growth (Hu, Eleftheriou and Arnold, 2005).
    Each new edge goes to a check node as far as possible from the variable."""
    rng = np.random.default_rng(seed)
    vdeg = np.broadcast_to(vn_degrees, (n,))
    chk_nb = [[] for _ in range(m)]                 # neighbours of each check
    var_nb = [[] for _ in range(n)]
    cdeg = np.zeros(m, dtype=int)
    for j in np.argsort(vdeg, kind="stable"):       # low degrees first
```

```
        for k in range(vdeg[j]):
            if k == 0:
                cand = np.flatnonzero(cdeg == cdeg.min())
            else:                                   # breadth-first search from j
                seen_c = np.zeros(m, bool); seen_c[var_nb[j]] = True
                frontier, cand = list(var_nb[j]), None
                while True:
                    nxt_v = {v for c in frontier for v in chk_nb[c]}
                    nxt_c = {c for v in nxt_v for c in var_nb[v]} - set(np.flatnonzero(
    seen_c))
                    if not nxt_c or seen_c.all():  # cannot grow: take the unreached
                        cand = np.flatnonzero(~seen_c)
                        break
                    seen_c[list(nxt_c)] = True
                    frontier = list(nxt_c)
                    if seen_c.all():               # all reached: farthest layer
                        cand = np.array(sorted(nxt_c))
                        break
                cand = cand[cdeg[cand] == cdeg[cand].min()]
            c = rng.choice(cand)
            chk_nb[c].append(j); var_nb[j].append(c); cdeg[c] += 1
    cn = np.array([c for j in range(n) for c in var_nb[j]])
    vn = np.array([j for j in range(n) for _ in var_nb[j]])
    return LDPCCode(m, n, cn, vn, f"PEG({n},{n - m})")
```

ldpc.py • lines 153--200

```
# ==== [qclift]
def qc_lift(base, Z):
    """Expand a base (exponent) matrix: -1 -> Z x Z zero block, s >= 0 ->
    identity cyclically shifted to the right by s (row r has a 1 in column (r+s) mod Z)."""
    base = np.asarray(base)
    r = np.arange(Z)
    cn, vn = [], []
    for i, j in zip(*np.nonzero(base >= 0)):
        cn.append(i * Z + r)
        vn.append(j * Z + (r + base[i, j]) % Z)
    return LDPCCode(base.shape[0] * Z, base.shape[1] * Z,
                    np.concatenate(cn), np.concatenate(vn), f"QC(Z={Z})")
# ==== [/qclift]


def girth(code):
    """Length of the shortest cycle of the Tanner graph (BFS from each variable)."""
    vnb = [code.cn[code.vperm[s:s + d]] for s, d in zip(code.vstart, code.dv)]
    cnb = [code.vn[s:s + d] for s, d in zip(code.cstart, code.dc)]
    best = np.inf
    for root in range(code.n):
        dist = {("v", root): 0}; parent = {("v", root): None}
        queue = [("v", root)]
        for node in queue:
            kind, idx = node
            nbrs = [("c", c) for c in vnb[idx]] if kind == "v" else [("v", v) for v in cnb[
    idx]]
```

```
            for nb in nbrs:
                if nb == parent[node]:
                    continue
                if nb in dist:                        # a cycle closes here
                    best = min(best, dist[node] + dist[nb] + 1)
                else:
                    dist[nb] = dist[node] + 1; parent[nb] = node; queue.append(nb)
            if 2 * dist[node] >= best:
                break
    return best


def count_4cycles(code):
    """Number of 4-cycles = pairs of checks sharing two (or more) variables."""
    H = code.H().astype(np.int64)
    overlap = H @ H.T
    iu = np.triu_indices(code.m, 1)
    k = overlap[iu]
    return int(np.sum(k * (k - 1) // 2))
# ==== [/construct]
```

ldpc.py • lines 201--281

```
# ==== [nr5g]
# ------------------------------------------------------------------ 4. 5G NR base graphs
NR_LIFTING_SETS = [[2, 4, 8, 16, 32, 64, 128, 256], [3, 6, 12, 24, 48, 96, 192, 384],
                   [5, 10, 20, 40, 80, 160, 320], [7, 14, 28, 56, 112, 224],
                   [9, 18, 36, 72, 144, 288], [11, 22, 44, 88, 176, 352],
                   [13, 26, 52, 104, 208], [15, 30, 60, 120, 240]]   # TS 38.212 Table
    5.3.2-1
DATA = Path(__file__).resolve().parent.parent / "data"


def nr_base_graph(bg, ils):
    """Base matrix V (entries -1 or V_ij) of BG1 (46 x 68) or BG2 (42 x 52) for
    lifting-set index ils, read from TS 38.212 Tables 5.3.2-2/3 (csv)."""
    shape = (46, 68) if bg == 1 else (42, 52)
    V = -np.ones(shape, dtype=int)
    row = 0
    for line in open(DATA / f"5G_bg{bg}.csv").read().splitlines()[2:]:
        f = line.split(";")
        if f[0].strip():
            row = int(f[0])
        V[row, int(f[1])] = int(f[2 + ils])
    return V


def nr_select(K, R, bg=None):
    """Base graph and lifting size for K information bits (incl. CRC) and rate R
    (TS 38.212, Sections 7.2.2 and 5.2.2)."""
    if bg is None:
        bg = 2 if (K <= 292 or (K <= 3824 and R <= 0.67) or R <= 0.25) else 1
    if bg == 1:
        kb = 22
```

```
    else:
        kb = 10 if K > 640 else 9 if K > 560 else 8 if K > 192 else 6
    Z = min(z for s in NR_LIFTING_SETS for z in s if kb * z >= K)
    ils = next(i for i, s in enumerate(NR_LIFTING_SETS) if Z in s)
    return bg, Z, ils


class NRCode:
    """A 5G NR LDPC code: lifted base graph, encoder and simple rate matching
    (redundancy version 0, no interleaving). Only the rows of the base graph that
    are needed for the E transmitted bits are kept (the other parity bits are not
    sent, so their checks carry no information)."""

    def __init__(self, K, E, bg=None):
        self.bg, Z, ils = nr_select(K, K / E, bg)
        self.K, self.E, self.Z, self.ils = K, E, Z, ils
        self.V = nr_base_graph(self.bg, ils)
        self.P = np.where(self.V >= 0, self.V % Z, -1)      # shift values P = V mod Z
        self.kb = 22 if self.bg == 1 else 10                 # information columns
        self.Kp = self.kb * Z                                # K' incl. filler bits
        self.filler = np.arange(K, self.Kp)                  # known zero bits
        nfull = self.V.shape[1] * Z                          # 68Z or 52Z
        tx = np.setdiff1d(np.arange(2 * Z, nfull), self.filler)   # first 2Z punctured
        self.Ncb = len(tx)
        tx = tx[:E] if E <= len(tx) else tx                   # bits that are ever sent
        ncol = max(self.kb + 4, -(-(tx.max() + 1) // Z))      # base columns in use
        self.code = qc_lift(self.P[:ncol - self.kb, :ncol], Z)
        self.code.name = f"NR BG{self.bg} K={K} E={E} Z={Z}"
        self.n = ncol * Z
        self.enc = DualEncoder(self.code, self.Kp, 4 * Z)
        self.tx_pos0 = tx
        self.tx_pos = np.resize(tx, E)                       # circular buffer, rv = 0

    def encode(self, u):
        u = np.atleast_2d(u)
        s = np.zeros((u.shape[0], self.Kp), dtype=np.uint8)
        s[:, :self.K] = u                                    # filler bits stay 0
        c = self.enc.encode(s)
        return c, c[:, self.tx_pos]                          # full codeword, sent bits

    def llr_in(self, llr_tx):
        """Map received LLRs of the E sent bits to the full-length decoder input."""
        L = np.zeros((llr_tx.shape[0], self.n))
        for p in range(0, self.E, len(self.tx_pos0)):         # repeated bits add up
            q = min(p + len(self.tx_pos0), self.E)
            L[:, self.tx_pos[p:q]] += llr_tx[:, p:q]
        L[:, self.filler] = 1e3                              # filler bits are known zeros
        return L
# ==== [/nr5g]
```

ldpc.py • lines 282--322

```
# ==== [encode]
# ------------------------------------------------------------------ 5. encoders
```

```python
def encode_G(G, u):
    """Encoding with a generator matrix: c = u G (mod 2)."""
    return (np.atleast_2d(u).astype(np.int64) @ G) % 2


class DualEncoder:
    """Linear-time-style encoder for H = [[A, B, 0], [C1, C2, I]]:
    columns = k systematic bits | g core parity bits | (m-g) extension parity bits.
    Step 1: lam = A s.  Step 2: p1 = B^-1 lam.  Step 3: p2 = C1 s + C2 p1."""

    def __init__(self, code, k, g):
        self.code, self.k, self.g = code, k, g
        H = np.zeros((g, k + g), dtype=np.uint8)
        sel = (code.cn < g) & (code.vn < k + g)
        H[code.cn[sel], code.vn[sel]] = 1
        self.Binv = gf2_inv(H[:, k:k + g])                  # small g x g inverse
        diag = (code.cn >= g) & (code.vn == code.cn + k)
        self.ext_edges = (code.cn >= g) & ~diag             # edges of [C1 C2]
        self.core_edges = (code.cn < g) & (code.vn < k)     # edges of A

    def _rows(self, mask, c, rows):
        """Parity of the bits c[vn] over the selected edges, per check row."""
        B, m = c.shape[0], self.code.m
        idx = (np.arange(B)[:, None] * m + self.code.cn[mask][None, :]).ravel()
        out = np.bincount(idx, weights=c[:, self.code.vn[mask]].ravel(), minlength=B * m)
        return out.reshape(B, m)[:, rows].astype(np.int64) % 2

    def encode(self, s):
        s = np.atleast_2d(s).astype(np.int64)
        B, n = s.shape[0], self.code.n
        c = np.zeros((B, n), dtype=np.int64)
        c[:, :self.k] = s
        lam = self._rows(self.core_edges, c, np.arange(self.g))         # step 1
        c[:, self.k:self.k + self.g] = (lam @ self.Binv.T.astype(np.int64)) % 2   # step 2
        c[:, self.k + self.g:] = self._rows(self.ext_edges, c, np.arange(self.g, self.code.
    m))  # step 3
        return c.astype(np.uint8)
# ==== [/encode]
```

ldpc.py • lines 323--341

```python
# ==== [channel]
# ------------------------------------------------------------------ 6. channels
def bpsk(c):
    return 1.0 - 2.0 * c


def sigma_from_ebn0(ebn0_db, R):
    """Noise standard deviation for BPSK (Es = 1) at the given Eb/N0 and rate."""
    return np.sqrt(1.0 / (2 * R * 10 ** (ebn0_db / 10)))


def awgn_llr(c, ebn0_db, R, rng):
    """Send codewords c through BPSK + AWGN and return channel LLRs 2y/sigma^2."""
```

```
    sigma = sigma_from_ebn0(ebn0_db, R)
    y = bpsk(c) + sigma * rng.standard_normal(c.shape)
    return 2 * y / sigma ** 2
# ==== [/channel]
```

ldpc.py • lines 342--382

```
# ==== [hard]
# ------------------------------------------------------------- 7. hard-decision decoders
def bit_flip(code, r, max_iter=50):
    """Gallager-style parallel bit flipping on hard decisions r (B, n): in each iteration flip every bit that sits in the largest number of failed checks."""
    x = np.atleast_2d(r).astype(np.uint8).copy()
    iters = np.zeros(x.shape[0], dtype=int)
    for it in range(1, max_iter + 1):
        s = code.syndrome(x)                                    # (B, m)
        active = s.any(axis=1)
        if not active.any():
            break
        iters[active] = it
        fails = code.var_sum(s[:, code.cn])                     # failed checks per bit
        worst = fails.max(axis=1, keepdims=True)
        flip = (fails == worst) & (worst > 0) & active[:, None]
        x ^= flip.astype(np.uint8)
    return x, iters


def peel(code, y):
    """Peeling decoder for the binary erasure channel. y holds 0, 1 or -1 (erasure).
    A check with exactly one erased neighbour recovers it. Returns the word
    (remaining erasures = -1) and the number of rounds used."""
    y = np.atleast_2d(y).astype(np.int64).copy()
    rounds = np.zeros(y.shape[0], dtype=int)
    for it in range(1, code.n + 1):
        ev = y[:, code.vn]                                      # edge values
        n_er = code.check_sum((ev < 0).astype(np.int64))        # erasures per check
        par = code.check_sum(np.where(ev < 0, 0, ev)) % 2       # parity of known bits
        solve = n_er == 1
        if not solve.any():
            break
        e_solve = solve[:, code.cn] & (ev < 0)                  # the single erased edge
        b, e = np.nonzero(e_solve)
        y[b, code.vn[e]] = par[b, code.cn[e]]
        rounds[np.unique(b)] = it
    return y, rounds
# ==== [/hard]
```

ldpc.py • lines 383--439

```
# ==== [bp]
# ------------------------------------------------------------- 8. soft decoders (flooding)
def phi(x):
```

```python
    """Gallager's function phi(x) = -ln tanh(x/2); it is its own inverse."""
    x = np.clip(x, 1e-12, 50.0)
    return -np.log(np.tanh(x / 2))


def check_update(code, v2c, rule="spa", alpha=0.75, beta=0.5, starts=None, seg=None):
    """Check-node messages from variable-to-check messages v2c (B, E).
    rule: 'spa' (sum-product), 'ms' (min-sum), 'nms' (normalised, x alpha),
    'oms' (offset, - beta). starts/seg describe the checks (default: all)."""
    st = code.cstart if starts is None else starts
    seg = code.cn if seg is None else seg                          # check of each edge
    neg = (v2c < 0).astype(np.int64)
    sign_all = np.add.reduceat(neg, st, axis=1) % 2               # parity of negatives
    sign = 1 - 2 * ((sign_all[:, seg] + neg) % 2)                  # exclude own sign
    mag = np.abs(v2c)
    if rule == "spa":
        p = phi(mag)
        out = phi(np.add.reduceat(p, st, axis=1)[:, seg] - p)
    else:
        m1 = np.minimum.reduceat(mag, st, axis=1)                  # smallest magnitude
        is1 = mag == m1[:, seg]
        n1 = np.add.reduceat(is1.astype(np.int64), st, axis=1)     # how many equal it
        m2 = np.minimum.reduceat(np.where(is1, np.inf, mag), st, axis=1)
        m2 = np.where(n1 >= 2, m1, m2)                             # second smallest
        out = np.where(is1, m2[:, seg], m1[:, seg])
        if rule == "nms":
            out = alpha * out
        elif rule == "oms":
            out = np.maximum(out - beta, 0.0)
    return sign * out


def bp_decode(code, llr, max_iter=50, rule="spa", alpha=0.75, beta=0.5, early_stop=True):
    """Flooding belief propagation. Returns hard decisions (B, n), the number of
    iterations used per frame and the final a-posteriori LLRs."""
    llr = np.atleast_2d(llr).astype(float)
    B = llr.shape[0]
    app, iters = llr.copy(), np.full(B, max_iter)
    act = np.arange(B)                                         # frames still decoding
    c2v = np.zeros((B, code.E))
    for it in range(1, max_iter + 1):
        v2c = app[act][:, code.vn] - c2v                       # extrinsic: remove own message
        c2v = check_update(code, v2c, rule, alpha, beta)
        app[act] = llr[act] + code.var_sum(c2v)                # a-posteriori LLRs
        if early_stop:
            ok = ~code.syndrome((app[act] < 0).astype(np.uint8)).any(axis=1)
            iters[act[ok]] = it
            act, c2v = act[~ok], c2v[~ok]                      # stop decoding valid codewords
            if act.size == 0:
                break
    return (app < 0).astype(np.uint8), iters, app
# ==== [/bp]
```

ldpc.py • lines 440--490

```python
# ==== [layered]
# ------------------------------------------------------------------ 9. layered decoding
def make_layers(code, Z=None):
    """Split the checks into layers whose checks share no variable. For a QC code
    pass Z: every block row of Z checks is one layer."""
    if Z is not None:
        groups = [np.arange(r, r + Z) for r in range(0, code.m, Z)]
    else:                                                # greedy colouring of checks
        used, groups = [], []
        for c in range(code.m):
            vs = set(code.vn[code.cstart[c]:code.cstart[c] + code.dc[c]])
            for g, u in zip(groups, used):
                if not (vs & u):
                    g.append(c); u |= vs; break
            else:
                groups.append([c]); used.append(set(vs))
        groups = [np.array(g) for g in groups]
    layers = []
    for g in groups:
        e = np.concatenate([np.arange(code.cstart[c], code.cstart[c] + code.dc[c]) for c in g])
        layers.append((e, np.r_[0, np.cumsum(code.dc[g])[:-1]], np.repeat(np.arange(len(g)), code.dc[g])))
    code.layers = layers
    return layers


def layered_decode(code, llr, max_iter=50, rule="nms", alpha=0.75, beta=0.5):
    """Row-layered (turbo-decoding message passing) decoder: the a-posteriori
    LLRs are updated after every layer, so new information is used at once."""
    if code.layers is None:
        make_layers(code)
    app = np.atleast_2d(llr).astype(float).copy()
    B = app.shape[0]
    iters = np.full(B, max_iter)
    act = np.arange(B)
    a, c2v = app.copy(), np.zeros((B, code.E))          # working copies of active frames
    for it in range(1, max_iter + 1):
        for e, st, sg in code.layers:
            v2c = a[:, code.vn[e]] - c2v[:, e]              # remove the old message
            new = check_update(code, v2c, rule, alpha, beta, starts=st, seg=sg)
            a[:, code.vn[e]] = v2c + new                    # update APP immediately
            c2v[:, e] = new
        ok = ~code.syndrome((a < 0).astype(np.uint8)).any(axis=1)
        app[act] = a
        iters[act[ok]] = it
        act, a, c2v = act[~ok], a[~ok], c2v[~ok]
        if act.size == 0:
            break
    return (app < 0).astype(np.uint8), iters, app
# ==== [/layered]
```

ldpc.py • lines 491--596

```python
# ==== [de]
# ---------------------------------------------------------------- 10. density evolution and EXIT
def poly(coef, x):
    """Evaluate a degree distribution {degree: fraction of edges} as sum c x^(d-1)."""
    return sum(c * x ** (d - 1) for d, c in coef.items())


def de_bec(lam, rho, eps, iters=2000, tol=1e-12):
    """Density evolution on the BEC: x_{l+1} = eps * lam(1 - rho(1 - x_l)).
    Returns the list of erasure probabilities x_l."""
    x, hist = eps, [eps]
    for _ in range(iters):
        x = eps * poly(lam, 1 - poly(rho, 1 - x))
        hist.append(x)
        if x < tol:
            break
    return hist


def threshold_bec(lam, rho, iters=5000):
    """Largest erasure probability for which density evolution goes to zero."""
    lo, hi = 0.0, 1.0
    for _ in range(40):
        mid = (lo + hi) / 2
        if de_bec(lam, rho, mid, iters)[-1] < 1e-10:
            lo = mid
        else:
            hi = mid
    return lo


_gh_x, _gh_w = np.polynomial.hermite_e.hermegauss(80)      # Gauss-Hermite nodes


def phi_ga(m):
    """phi(m) = 1 - E[tanh(u/2)], u ~ N(m, 2m) (Chung et al., 2001), by quadrature."""
    m = np.atleast_1d(np.asarray(m, float))
    u = m[:, None] + np.sqrt(2 * m[:, None]) * _gh_x[None, :]
    val = 1 - (np.tanh(u / 2) * _gh_w).sum(axis=1) / np.sqrt(2 * np.pi)
    return np.where(m < 1e-10, 1.0, val)


_m_tab = np.logspace(-8, 3, 6000)                          # table for the inverse
_lphi_tab = np.log(np.maximum(phi_ga(_m_tab), 1e-300))


def phi_ga_inv(y):
    """Inverse of phi_ga by interpolation of ln(phi) on a log grid of m."""
    lm = np.interp(-np.log(max(y, 1e-300)), -_lphi_tab, np.log(_m_tab))
    return float(np.exp(lm))


```

```python
def de_ga(lam, rho, sigma, iters=20000, big=60.0):
    """Gaussian-approximation density evolution on BI-AWGN. Returns True if the
    mean of the check-to-variable messages grows without bound."""
    m0, mu = 2 / sigma ** 2, 0.0
    for _ in range(iters):
        s = sum(c * phi_ga(m0 + (d - 1) * mu)[0] for d, c in lam.items())
        mu = sum(c * phi_ga_inv(1 - (1 - s) ** (d - 1)) for d, c in rho.items())
        if mu > big:
            return True
    return False


def threshold_ga(lam, rho):
    lo, hi = 0.3, 1.5
    for _ in range(30):
        mid = (lo + hi) / 2
        lo, hi = (mid, hi) if de_ga(lam, rho, mid) else (lo, mid)
    return lo


def de_mc(dv, dc, sigma, iters=300, pop=100000, seed=1):
    """Monte-Carlo (population) density evolution for a (dv, dc)-regular code
    with sum-product decoding. Returns the bit error probability per iteration."""
    rng = np.random.default_rng(seed)
    ch = lambda: 2 / sigma ** 2 * (1 + sigma * rng.standard_normal(pop))
    v, pe = ch(), []
    for _ in range(iters):
        t = np.prod(np.tanh(v[rng.integers(0, pop, (dc - 1, pop))] / 2), axis=0)
        u = 2 * np.arctanh(np.clip(t, -1 + 1e-15, 1 - 1e-15))
        v = ch() + u[rng.integers(0, pop, (dv - 1, pop))].sum(axis=0)
        pe.append(np.mean(v < 0))
        if np.median(v) > 60:
            break
    return pe


def J(sigma):
    """Mutual information of a consistent Gaussian LLR with std sigma (EXIT charts)."""
    s = np.atleast_1d(np.asarray(sigma, float))
    m = s ** 2 / 2
    u = m[:, None] + s[:, None] * _gh_x[None, :]
    val = 1 - (np.logaddexp(0, -u) / np.log(2) * _gh_w).sum(axis=1) / np.sqrt(2 * np.pi)
    return np.where(s < 1e-8, 0.0, val)


def J_inv(I):
    lo, hi = 0.0, 60.0
    for _ in range(60):
        mid = (lo + hi) / 2
        lo, hi = (mid, hi) if J(mid)[0] < I else (lo, mid)
    return (lo + hi) / 2
# ==== [/de]
```

ldpc.py • lines 597--623

```python
# ==== [sim]
# ------------------------------------------------------------- 11. Monte-Carlo simulation
def simulate(encode, decode, k, R, ebn0_list, info=None, max_frames=100000,
             min_errors=100, batch=1000, seed=2026, stop_fer=0.0):
    """BER/FER versus Eb/N0. encode(u) -> (codeword, sent bits);
    decode(llr of sent bits) -> (decoded codeword, iterations); info = positions
    of the message bits inside the decoded codeword. The sweep stops after the
    first point with FER < stop_fer."""
    rng = np.random.default_rng(seed)
    res = {"ebn0": list(ebn0_list), "ber": [], "fer": [], "frames": [], "avg_iter": []}
    for eb in ebn0_list:
        bit_err = fr_err = frames = it_sum = 0
        while frames < max_frames and fr_err < min_errors:
            u = rng.integers(0, 2, (batch, k), dtype=np.uint8)
            c, x = encode(u)
            llr = awgn_llr(x, eb, R, rng)
            c_hat, iters = decode(llr)
            u_hat = c_hat[:, info] if info is not None else c_hat[:, :k]
            e = (u_hat != u).sum(axis=1)
            bit_err += e.sum(); fr_err += (e > 0).sum(); frames += batch; it_sum += iters.sum()
        res["ber"].append(bit_err / (frames * k)); res["fer"].append(fr_err / frames)
        res["frames"].append(frames); res["avg_iter"].append(it_sum / frames)
        if fr_err / frames < stop_fer:                    # low enough: stop the sweep
            res["ebn0"] = res["ebn0"][:len(res["fer"])]
            break
    return res
# ==== [/sim]
```

B.2 Unit tests `test_ldpc.py`

test_ldpc.py

```python
"""Unit tests of ldpc.py (run: python3 test_ldpc.py)."""
import itertools, numpy as np
from ldpc import *

H6 = np.array([[1, 1, 0, 1, 0, 0], [0, 1, 1, 0, 1, 0], [1, 0, 0, 0, 1, 1], [0, 0, 1, 1, 0, 1]], dtype=np.uint8)
rng = np.random.default_rng(1)
results = []

def test(name):
    def deco(f):
        f(); results.append(name); print("PASS", name)
    return deco

@test("GF(2): rank, inverse and generator matrix")
def _():
```

```
    assert gf2_rank(H6) == 3
    for _ in range(20):
        A = rng.integers(0, 2, (8, 8)).astype(np.uint8)
        if gf2_rank(A) == 8:
            assert ((A.astype(int) @ gf2_inv(A)) % 2 == np.eye(8)).all()
    for H in [H6, gallager_H(60, 3, 6, seed=2), peg(96, 48, 3, seed=0).H()]:
        G, info = generator_from_H(H)
        assert not ((H.astype(int) @ G.T) % 2).any()
        assert G.shape[0] == H.shape[1] - gf2_rank(H) == gf2_rank(G)
        assert (G[:, info] == np.eye(G.shape[0])).all()

@test("Code object: degrees, H and syndromes")
def _():
    c = LDPCCode.from_H(H6)
    assert (c.H() == H6).all() and c.E == 12 and (c.dv == 2).all() and (c.dc == 3).all()
    cws = [w for w in itertools.product([0, 1], repeat=6) if not ((H6 @ np.array(w)) % 2).
    any()]
    assert len(cws) == 8 and not c.syndrome(np.array(cws)).any()

@test("Girth and 4-cycles")
def _():
    assert girth(LDPCCode.from_H(H6)) == 6
    assert girth(qc_lift([[0, 0, 0, 0], [0, 1, 2, 3]], 5)) == 8
    q = qc_lift([[0, 0, 0, 0], [0, 1, 1, 3]], 5)
    assert girth(q) == 4 and count_4cycles(q) == 5
    tree = LDPCCode.from_H(np.array([[1, 1, 1, 0, 0, 0, 0], [0, 0, 1, 1, 1, 0, 0], [0, 0,
    0, 0, 1, 1, 1]]))
    assert girth(tree) == np.inf

@test("5G NR tables, parameter selection and encoder")
def _():
    for ils in range(8):
        assert (nr_base_graph(1, ils) >= 0).sum() == 316 and (nr_base_graph(2, ils) >= 0).
    sum() == 197
    assert nr_select(120, 1 / 3)[:2] == (2, 20) and nr_select(1024, 0.5)[:2] == (2, 104)
    assert nr_select(1024, 8 / 9)[:2] == (1, 48) and nr_select(8448, 1 / 3)[:2] == (1, 384)
    for K, E in [(40, 200), (120, 360), (500, 1250), (1024, 2048), (1024, 1152), (3840,
    19200), (8448, 25344), (100, 3000)]:
        nr = NRCode(K, E)
        c, x = nr.encode(rng.integers(0, 2, (3, K)))
        assert not nr.code.syndrome(c).any() and x.shape == (3, E)
        assert not c[:, nr.filler].any()

@test("Check-node update equals the brute-force formulas")
def _():
    c = LDPCCode.from_H(H6)
    v2c = rng.normal(0, 3, (50, c.E))
    spa = check_update(c, v2c, "spa"); ms = check_update(c, v2c, "ms")
    for e in range(c.E):
        others = [f for f in range(c.E) if c.cn[f] == c.cn[e] and f != e]
        t = np.prod(np.tanh(v2c[:, others] / 2), axis=1)
        assert np.allclose(spa[:, e], 2 * np.arctanh(t), atol=1e-8)
        assert np.allclose(ms[:, e], np.prod(np.sign(v2c[:, others]), 1) * np.abs(v2c[:,
    others]).min(1))
```

```
    assert np.allclose(check_update(c, v2c, "nms", alpha=0.75), 0.75 * ms)

@test("Sum-product is exact (bit-MAP) on a cycle-free graph")
def _():
    Ht = np.array([[1, 1, 1, 0, 0, 0, 0], [0, 0, 1, 1, 1, 0, 0], [0, 0, 0, 0, 1, 1, 1]],
    dtype=np.uint8)
    ct = LDPCCode.from_H(Ht)
    cw = np.array([w for w in itertools.product([0, 1], repeat=7) if not ((Ht @ np.array(w)
    ) % 2).any()])
    for _ in range(100):
        L = rng.normal(1, 2, 7)
        app = bp_decode(ct, L, 10, "spa", early_stop=False)[2][0]
        ll = (np.where(cw == 0, 1, -1) * L / 2).sum(1)
        mp = [np.logaddexp.reduce(ll[cw[:, i] == 0]) - np.logaddexp.reduce(ll[cw[:, i] ==
    1]) for i in range(7)]
        assert np.abs(app - mp).max() < 1e-9

@test("Decoders: noiseless frames and high-SNR frames (flooding and layered)")
def _():
    code = peg(504, 252, 3, seed=3); G, info = generator_from_H(code.H()); make_layers(code
    )
    c = encode_G(G, rng.integers(0, 2, (200, G.shape[0]))).astype(np.uint8)
    L = 10.0 * bpsk(c)
    for rule in ["spa", "ms", "nms", "oms"]:
        x, it, _ = bp_decode(code, L, 5, rule); assert (x == c).all() and (it == 1).all()
    L = awgn_llr(c, 4.0, 0.5, np.random.default_rng(9))
    for f in [lambda: bp_decode(code, L, 50, "spa"), lambda: layered_decode(code, L, 50, "
    nms")]:
        assert (f()[0] == c).all()

@test("Peeling and bit flipping")
def _():
    code = peg(96, 48, 3, seed=0); G, _ = generator_from_H(code.H())
    c = encode_G(G, rng.integers(0, 2, (50, G.shape[0]))).astype(int)
    y = c.copy(); y[:, :3] = -1                                  # 3 erasures
    out, _ = peel(code, y)
    assert ((out == c) | (out < 0)).all()                        # never wrong
    assert girth(code) >= 6
    for i in range(96):                                          # every single error
    corrected
        r = c[:1].copy(); r[0, i] ^= 1
        assert (bit_flip(code, r, 10)[0] == c[:1]).all()

@test("Density evolution thresholds")
def _():
    assert abs(threshold_bec({3: 1.0}, {6: 1.0}) - 0.4294) < 1e-4
    assert abs(threshold_bec({3: 1.0}, {4: 1.0}) - 0.6474) < 1e-4
    assert 0.870 < threshold_ga({3: 1.0}, {6: 1.0}) < 0.874
    assert de_mc(3, 6, 0.86, iters=100, pop=50000)[-1] == 0.0

@test("Monte-Carlo simulation runs and decodes at high SNR")
def _():
    nr = NRCode(256, 512); make_layers(nr.code, nr.Z)
    dec = lambda L: layered_decode(nr.code, nr.llr_in(L), 20, "nms")[:2]
```

```
    r = simulate(nr.encode, dec, 256, 0.5, [4.0], info=np.arange(256), max_frames=400, batch=200)
    assert r["fer"][0] == 0.0 and r["frames"][0] == 400

print(f"\n{len(results)} tests passed")
```

B.3 Irregular code design `design_bec.py`

design_bec.py

```
"""Linear-programming design of irregular LDPC degree distributions for the BEC."""
import numpy as np
from scipy.optimize import linprog
from ldpc import threshold_bec, poly

def feasible(eps, dc, dmax, rate, grid=np.linspace(1e-4, 1, 600)):
    """Is there lambda (degrees 2..dmax) with design rate >= rate and
    eps*lambda(1-rho(1-x)) < x on the grid? rho(x) = x^(dc-1)."""
    degs = np.arange(2, dmax + 1)
    y = 1 - (1 - grid) ** (dc - 1)                       # 1 - rho(1 - x)
    A = eps * y[:, None] ** (degs[None, :] - 1)          # eps * y^(i-1)
    b = grid * (1 - 1e-6)
    A_ub = np.vstack([A, -1.0 / degs[None, :]])          # rate: sum lam_i/i >= (1/dc)/(1-rate)
    b_ub = np.r_[b, -(1 / dc) / (1 - rate)]
    A_ub = np.vstack([A_ub, np.r_[eps * (dc - 1), np.zeros(len(degs) - 1)]])   # stability
    b_ub = np.r_[b_ub, 1 - 1e-6]
    r = linprog(-1.0 / degs, A_ub=A_ub, b_ub=b_ub, A_eq=np.ones((1, len(degs))), b_eq=[1],
                bounds=[(0, 1)] * len(degs), method="highs")
    return r.status == 0, (dict(zip(degs.tolist(), r.x)) if r.status == 0 else None)

if __name__ == "__main__":
    for dc, dmax in [(6, 3), (6, 6), (7, 10), (8, 16), (9, 30)]:
        lo, hi, best = 0.3, 0.5, None
        for _ in range(30):
            mid = (lo + hi) / 2
            ok, lam = feasible(mid, dc, dmax, 0.5)
            if ok:
                lo, best = mid, lam
            else:
                hi = mid
        lam = {d: c for d, c in best.items() if c > 1e-4}
        s = sum(lam.values()); lam = {d: c / s for d, c in lam.items()}
        rate = 1 - (1 / dc) / sum(c / d for d, c in lam.items())
        th = threshold_bec(lam, {dc: 1.0})
        print(f"dc={dc} dmax={dmax}: LP eps={lo:.4f}  checked threshold={th:.4f}  rate={rate:.4f}  lambda=",
              {d: round(c, 4) for d, c in lam.items()})
```

OUTPUT (real run)

```
dc=6 dmax=3: LP eps=0.4294  checked threshold=0.4294  rate=0.5000  lambda= {3: np.float64(1.0)}
dc=6 dmax=6: LP eps=0.4776  checked threshold=0.4757  rate=0.5000  lambda= {2: np.float64(0.4188), 3:
    np.float64(0.1624), 6: np.float64(0.4188)}
dc=7 dmax=10: LP eps=0.4855  checked threshold=0.4855  rate=0.5000  lambda= {2: np.float64(0.3224), 3:
     np.float64(0.2432), 10: np.float64(0.4344)}
dc=8 dmax=16: LP eps=0.4852  checked threshold=0.4852  rate=0.5000  lambda= {2: np.float64(0.2845), 3:
     np.float64(0.1421), 4: np.float64(0.131), 16: np.float64(0.4425)}
dc=9 dmax=30: LP eps=0.4892  checked threshold=0.4876  rate=0.5000  lambda= {2: np.float64(0.2552), 3:
     np.float64(0.1199), 5: np.float64(0.159), 6: np.float64(0.0549), 30: np.float64(0.411)}
```

B.4 LDPC versus polar `compare_polar.py`

compare_polar.py

```python
"""LDPC (5G NR, layered NMS / flooding SPA) versus polar (SC, CA-SCL) at equal
length and rate. polar.py is the library of Part One."""
import json, sys, time, numpy as np
import polar
from ldpc import *

out = {}
def save():
    json.dump(out, open("../data/sim_E.json", "w"), indent=1)

E = lambda a, b: list(np.round(np.arange(a, b + 1e-9, 0.25), 2))
for N in [512, 1024]:
    K = N // 2
    nr = NRCode(K, N); make_layers(nr.code, nr.Z)
    for tag, dec in [("LNMS", lambda L, nr=nr: layered_decode(nr.code, nr.llr_in(L), 50, "nms")[:2]),
                     ("SPA", lambda L, nr=nr: bp_decode(nr.code, nr.llr_in(L), 50, "spa")[:2])]:
        t = time.time()
        r = simulate(nr.encode, dec, K, 0.5, E(0.5, 3.5), info=np.arange(K), max_frames=40000,
                     min_errors=50, batch=500, seed=11, stop_fer=2e-4)
        out[f"N{N}_LDPC_{tag}"] = {k: [float(x) for x in v] for k, v in r.items()} | {"bg": nr.bg, "Z": nr.Z}
        print(f"N{N} LDPC {tag} BG{nr.bg} Z{nr.Z}", f"{time.time()-t:.0f}s", ["%.1e" % f for f in r["fer"]], flush=True)
        save()
    for tag, L, Kp, crc in [("SC", 1, K, None), ("CASCL8", 8, K + 11, "CRC11")]:
        t = time.time(); fer_all = []
        res = {"ebn0": [], "ber": [], "fer": []}
        for eb in E(0.5, 4.0):
            r = polar.simulate(N, Kp, [eb], L=L, crc_name=crc, max_frames=40000, min_errors=50, batch=500, seed=11)
            res["ebn0"].append(eb); res["ber"].append(float(r["ber"][0])); res["fer"].append(float(r["bler"][0]))
            if r["bler"][0] < 2e-4:
                break
```

```
        out[f"N{N}_POLAR_{tag}"] = res
        print(f"N{N} polar {tag}", f"{time.time()-t:.0f}s", ["%.1e" % f for f in res["fer"
    ]], flush=True)
        save()
```

Appendix C — MATLAB Implementation

C.1 Overview

This appendix gives the complete library in MATLAB. The algorithms, conventions and worked examples are the same as in the Python code; every function was run in GNU Octave 8.4 and checked against the Python results (Section C.9). Copy all `.m` files, the two CSV files `5G_bg1.csv` and `5G_bg2.csv`, and `pyref_ldpc.mat` (only for the tests) into one folder.

MATLAB files	Purpose	Book
`gf2_rref, gf2_rank, gf2_inv, generator_from_H`	GF(2) algebra, $\mathbf{G}$ from $\mathbf{H}$	Ch. 3
`ldpc_from_edges, ldpc_from_H, ldpc_H, ldpc_syndrome`	Tanner-graph object	Ch. 4, 12
`gallager_H, peg_code, qc_lift, ldpc_girth, count_4cycles`	constructions, girth	Ch. 5
`encode_G, dual_encoder, dual_encode`	encoders	Ch. 6
`bit_flip, peel`	hard-decision decoders	Ch. 7
`phi_fn, check_update, bp_decode`	sum-product and min-sum	Ch. 8, 9
`make_layers, layered_decode`	layered decoding	Ch. 9
`de_bec, threshold_bec, gh_nodes, phi_ga, threshold_ga`	density evolution	Ch. 10
`nr_lifting_sets, nr_base_graph, nr_select, nr_code, nr_encode, nr_llr_in`	5G NR codes	Ch. 11
`sigma_from_ebn0, awgn_llr, ldpc_simulate`	channel and simulation	Ch. 12
`demo_ldpc, test_ldpc_matlab, sim_ldpc_matlab, final_project_lab, crc_matrix, pass_fail`	scripts	Ch. 12–16

! MATLAB conventions

- **Indices start at 1.** Bit i, check j and base-graph entry (i, j) of the book (0-based) are `(i+1)`, `(j+1)` in MATLAB. The set index i_{LS} and the shifts stay as in the book (0-based numbers).
- A code is a `struct` with fields `m`, `n`, `E`, `cn`, `vn` (edges sorted by check, then by variable,

exactly as in Python), the sparse incidence matrices `Sc` ($E \times m$) and `Sv` ($E \times n$) and the table positions `epos` used by min-sum.

- Frames are *rows*: an LLR array is $B \times n$. Per-check sums are products with `Sc`, per-variable sums are products with `Sv`.
- Degree distributions are matrices of rows `[degree, edge fraction]`, e.g. `[3 1]` for $\lambda(x) = x^2$.
- MATLAB R2016b or newer (implicit expansion); no toolbox is needed. Random experiments use MATLAB's own generator, so their numbers differ from the Python runs but agree statistically; deterministic results are identical.

C.2 GF(2) algebra

gf2_rref.m

```
function [R, piv] = gf2_rref(A)
%GF2_RREF Reduced row echelon form over GF(2). piv = pivot columns (1-based).
R = mod(double(A), 2);
[m, n] = size(R);
piv = []; row = 1;
for col = 1:n
    if row > m, break; end
    hits = find(R(row:end, col), 1);
    if isempty(hits), continue; end
    p = row + hits - 1;
    R([row p], :) = R([p row], :);                % swap the pivot row up
    others = find(R(:, col)); others(others == row) = [];
    R(others, :) = mod(R(others, :) + R(row, :), 2);   % clear the column
    piv(end + 1) = col; %#ok<AGROW>
    row = row + 1;
end
R = R(1:row - 1, :);
end
```

gf2_rank.m

```
function r = gf2_rank(A)
%GF2_RANK Rank of a binary matrix over GF(2).
[~, piv] = gf2_rref(A);
r = numel(piv);
end
```

gf2_inv.m

```
function Ai = gf2_inv(A)
%GF2_INV Inverse of a square invertible matrix over GF(2).
n = size(A, 1);
[R, piv] = gf2_rref([mod(A, 2), eye(n)]);
if numel(piv) < n || any(piv(1:n) ~= 1:n)
```

```
    error('matrix is singular over GF(2)');
end
Ai = R(:, n + 1:end);
end
```

generator_from_H.m

```
function [G, info] = generator_from_H(H)
%GENERATOR_FROM_H Generator matrix G (k x n) with H*G' = 0 (mod 2).
% info = columns (1-based) where the codeword equals the message.
[R, piv] = gf2_rref(H);
n = size(H, 2);
info = setdiff(1:n, piv);
G = zeros(numel(info), n);
G(:, info) = eye(numel(info));
G(:, piv) = R(:, info).';           % c(piv_i) = sum_j R(i, info_j) c(info_j)
end
```

C.3 The Tanner-graph object

ldpc_from_edges.m

```
function code = ldpc_from_edges(m, n, cn, vn, name)
%LDPC_FROM_EDGES Tanner-graph object. cn, vn: check and variable of each edge
% (1-based). Edges are sorted by check, then by variable (same order as Python).
if nargin < 5, name = 'LDPC'; end
cv = sortrows([cn(:), vn(:)]);
code.m = m; code.n = n; code.name = name;
code.cn = cv(:, 1); code.vn = cv(:, 2); code.E = numel(code.cn);
code.dc = accumarray(code.cn, 1, [m 1]);          % check degrees
code.dv = accumarray(code.vn, 1, [n 1]);          % variable degrees
code.Sc = sparse(1:code.E, code.cn, 1, code.E, m); % edge -> check incidence
code.Sv = sparse(1:code.E, code.vn, 1, code.E, n); % edge -> variable incidence
code = add_padding(code);
code.layers = {};
end

function s = add_padding(s)
% position of every edge inside an (m x max degree) table, for min-sum
first = [1; cumsum(s.dc(1:end - 1)) + 1];
t = (1:s.E).' - first(s.cn) + 1;                  % slot of the edge in its check
s.dmax = max(s.dc);
s.epos = s.cn + (t - 1) * s.m;                    % linear index in m x dmax
end
```

ldpc_from_H.m

```
function code = ldpc_from_H(H, name)
%LDPC_FROM_H Build the Tanner-graph object from a dense parity-check matrix.
if nargin < 2, name = 'LDPC'; end
[cn, vn] = find(mod(H, 2));
```

```
code = ldpc_from_edges(size(H, 1), size(H, 2), cn, vn, name);
end
```

ldpc_H.m

```
function H = ldpc_H(code)
%LDPC_H Dense parity-check matrix (small codes only).
H = full(sparse(code.cn, code.vn, 1, code.m, code.n));
end
```

ldpc_syndrome.m

```
function s = ldpc_syndrome(code, C)
%LDPC_SYNDROME Syndromes H*c' for a batch of words C (B x n) -> (B x m).
s = mod(double(C(:, code.vn)) * code.Sc, 2);
end
```

C.4 Constructions

gallager_H.m

```
function H = gallager_H(n, dv, dc)
%GALLAGER_H Gallager's (dv,dc)-regular construction: dv stacked bands, each a
% random column permutation of the first band (uses MATLAB's random stream).
mb = n / dc;
band = kron(eye(mb), ones(1, dc));
H = band;
for b = 2:dv
    H = [H; band(:, randperm(n))]; %#ok<AGROW>
end
end
```

peg_code.m

```
function code = peg_code(n, m, dv)
%PEG_CODE Progressive edge growth (Hu, Eleftheriou, Arnold 2005). dv: scalar or
% vector of variable degrees. Ties are broken with MATLAB's random stream.
dv = dv(:).' .* ones(1, n);
chk = cell(m, 1); var = cell(n, 1); cdeg = zeros(m, 1);
[~, order] = sort(dv);
for j = order
    for k = 1:dv(j)
        if k == 1
            cand = find(cdeg == min(cdeg));
        else
            seen = false(m, 1); seen(var{j}) = true;
            frontier = var{j};
            while true
                nv = unique([chk{frontier}]);
                nc = setdiff(unique([var{nv}]), find(seen));
                if isempty(nc) || all(seen)
```

```
                    cand = find(~seen); break;
                end
                seen(nc) = true; frontier = nc;
                if all(seen)
                    cand = nc(:); break;
                end
            end
            cand = cand(cdeg(cand) == min(cdeg(cand)));
        end
        c = cand(randi(numel(cand)));
        chk{c}(end + 1) = j; var{j}(end + 1) = c; cdeg(c) = cdeg(c) + 1;
    end
end
cn = [var{:}].'; vn = repelem((1:n).', cellfun(@numel, var));
code = ldpc_from_edges(m, n, cn, vn, sprintf('PEG(%d,%d)', n, n - m));
end
```

qc_lift.m

```
function code = qc_lift(base, Z)
%QC_LIFT Expand a base matrix: -1 -> zero block, s >= 0 -> identity shifted
% right by s (row r has its one in column (r+s) mod Z, 0-based inside the block).
[I, J] = find(base >= 0);
r = (0:Z - 1).';
cn = zeros(Z * numel(I), 1); vn = cn;
for t = 1:numel(I)
    s = base(I(t), J(t));
    cn((t - 1) * Z + (1:Z)) = (I(t) - 1) * Z + r + 1;
    vn((t - 1) * Z + (1:Z)) = (J(t) - 1) * Z + mod(r + s, Z) + 1;
end
code = ldpc_from_edges(size(base, 1) * Z, size(base, 2) * Z, cn, vn, sprintf('QC(Z=%d)', Z)
    );
end
```

ldpc_girth.m

```
function g = ldpc_girth(code)
%LDPC_GIRTH Shortest cycle of the Tanner graph (breadth-first search from
% every variable node). Nodes 1..n are variables, n+1..n+m are checks.
A = [sparse(code.n, code.n), sparse(code.vn, code.cn, 1, code.n, code.m); ...
     sparse(code.cn, code.vn, 1, code.m, code.n), sparse(code.m, code.m)];
nb = cell(code.n + code.m, 1);
for v = 1:code.n + code.m, nb{v} = find(A(:, v)).'; end
g = inf;
for root = 1:code.n
    dist = -ones(code.n + code.m, 1); par = zeros(code.n + code.m, 1);
    dist(root) = 0; queue = root; head = 1;
    while head <= numel(queue)
        u = queue(head); head = head + 1;
        if 2 * dist(u) >= g, break; end
        for w = nb{u}
            if w == par(u), continue; end
            if dist(w) >= 0
                g = min(g, dist(u) + dist(w) + 1);
```

```
            else
                dist(w) = dist(u) + 1; par(w) = u; queue(end + 1) = w; %#ok<AGROW>
            end
        end
    end
end
end
```

count_4cycles.m

```
function c = count_4cycles(code)
%COUNT_4CYCLES Number of 4-cycles = pairs of checks sharing two or more variables.
H = sparse(code.cn, code.vn, 1, code.m, code.n);
O = triu(full(H * H.'), 1);
c = sum(O(:) .* (O(:) - 1) / 2);
end
```

C.5 Encoders and channel

encode_G.m

```
function c = encode_G(G, u)
%ENCODE_G Encoding with a generator matrix: c = u*G (mod 2).
c = mod(double(u) * G, 2);
end
```

dual_encoder.m

```
function enc = dual_encoder(code, k, g)
%DUAL_ENCODER Prepare the encoder for H = [A B 0; C1 C2 I]:
% k systematic bits, g core parity bits, m-g extension parity bits.
enc.code = code; enc.k = k; enc.g = g;
sel = code.cn <= g & code.vn <= k + g;
Hc = full(sparse(code.cn(sel), code.vn(sel), 1, g, k + g));
enc.Binv = gf2_inv(Hc(:, k + 1:k + g));                % small g x g inverse
diagE = code.cn > g & code.vn == code.cn + k;          % the identity part
enc.ext = code.cn > g & ~diagE;                        % edges of [C1 C2]
enc.core = code.cn <= g & code.vn <= k;                % edges of A
end
```

dual_encode.m

```
function c = dual_encode(enc, s)
%DUAL_ENCODE Three-step encoding: lam = A s; p1 = B^-1 lam; p2 = C1 s + C2 p1.
code = enc.code; k = enc.k; g = enc.g;
c = zeros(size(s, 1), code.n);
c(:, 1:k) = s;
lam = rowsum(code, enc.core, c);                       % step 1
c(:, k + 1:k + g) = mod(lam(:, 1:g) * enc.Binv.', 2);  % step 2
p2 = rowsum(code, enc.ext, c);                         % step 3
c(:, k + g + 1:end) = p2(:, g + 1:code.m);
```

```
end

function out = rowsum(code, mask, c)
e = find(mask);
S = sparse(1:numel(e), code.cn(e), 1, numel(e), code.m);
out = mod(c(:, code.vn(e)) * S, 2);
end
```

sigma_from_ebn0.m

```
function s = sigma_from_ebn0(ebn0_db, R)
%SIGMA_FROM_EBN0 Noise standard deviation for BPSK (Es = 1).
s = sqrt(1 ./ (2 * R * 10 .^ (ebn0_db / 10)));
end
```

awgn_llr.m

```
function L = awgn_llr(c, ebn0_db, R)
%AWGN_LLR BPSK (0 -> +1, 1 -> -1) over AWGN; returns channel LLRs 2y/sigma^2.
sigma = sigma_from_ebn0(ebn0_db, R);
y = (1 - 2 * double(c)) + sigma * randn(size(c));
L = 2 * y / sigma ^ 2;
end
```

C.6 Decoders

phi_fn.m

```
function y = phi_fn(x)
%PHI_FN Gallager's function phi(x) = -ln tanh(x/2); it is its own inverse.
x = min(max(x, 1e-12), 50);
y = -log(tanh(x / 2));
end
```

check_update.m

```
function out = check_update(seg, v2c, rule, alpha, beta)
%CHECK_UPDATE Check-node messages from variable-to-check messages v2c (B x E).
% seg describes the checks: fields cn (check of each edge), m, Sc, dmax, epos.
% rule: 'spa' (sum-product), 'ms', 'nms' (x alpha) or 'oms' (- beta).
if nargin < 4, alpha = 0.75; end
if nargin < 5, beta = 0.5; end
B = size(v2c, 1);
neg = double(v2c < 0);
sign_all = mod(neg * seg.Sc, 2);                    % parity of negatives per check
sgn = 1 - 2 * mod(sign_all(:, seg.cn) + neg, 2);    % exclude own sign
mag = abs(v2c);
if strcmp(rule, 'spa')
    p = phi_fn(mag);
    S = p * seg.Sc;                                  % sum of phi per check
    out = phi_fn(S(:, seg.cn) - p);
```

```
else
    m1 = groupmin(seg, mag, B);                          % smallest magnitude
    is1 = mag == m1(:, seg.cn);
    n1 = double(is1) * seg.Sc;                           % how many equal it
    mag2 = mag; mag2(is1) = inf;
    m2 = groupmin(seg, mag2, B);
    m2(n1 >= 2) = m1(n1 >= 2);                           % second smallest
    out = m1(:, seg.cn);
    M2 = m2(:, seg.cn);
    out(is1) = M2(is1);
    if strcmp(rule, 'nms')
        out = alpha * out;
    elseif strcmp(rule, 'oms')
        out = max(out - beta, 0);
    end
end
out = sgn .* out;
end

function mn = groupmin(seg, x, B)
T = inf(B, seg.m * seg.dmax);
T(:, seg.epos) = x;
mn = min(reshape(T, B, seg.m, seg.dmax), [], 3);
if B == 1, mn = mn(:).'; end
end
```

bp_decode.m

```
function [chat, iters, app] = bp_decode(code, llr, max_iter, rule, alpha, beta, early_stop)
%BP_DECODE Flooding belief propagation on a batch of frames (rows of llr).
% Returns hard decisions, iterations used per frame and a-posteriori LLRs.
if nargin < 4, rule = 'spa'; end
if nargin < 5, alpha = 0.75; end
if nargin < 6, beta = 0.5; end
if nargin < 7, early_stop = true; end
B = size(llr, 1);
app = llr; iters = max_iter * ones(B, 1);
act = (1:B).';                                           % frames still decoding
c2v = zeros(B, code.E);
for it = 1:max_iter
    v2c = app(act, code.vn) - c2v;                       % extrinsic: remove own message
    c2v = check_update(code, v2c, rule, alpha, beta);
    app(act, :) = llr(act, :) + c2v * code.Sv;           % a-posteriori LLRs
    if early_stop
        ok = ~any(ldpc_syndrome(code, app(act, :) < 0), 2);
        iters(act(ok)) = it;
        act = act(~ok); c2v = c2v(~ok, :);
        if isempty(act), break; end
    end
end
chat = double(app < 0);
end
```

make_layers.m

```
function code = make_layers(code, Z)
%MAKE_LAYERS Groups of checks that share no variable. With Z: every block row of
% Z checks is one layer (QC codes); otherwise greedy colouring (same as Python).
first = [1; cumsum(code.dc(1:end - 1)) + 1];
if nargin > 1
    groups = arrayfun(@(r) (r:r + Z - 1).', 1:Z:code.m, 'UniformOutput', false);
else
    groups = {}; used = {};
    for c = 1:code.m
        vs = code.vn(first(c):first(c) + code.dc(c) - 1);
        placed = false;
        for g = 1:numel(groups)
            if ~any(used{g}(vs))
                groups{g}(end + 1, 1) = c; used{g}(vs) = true; placed = true; break;
            end
        end
        if ~placed
            groups{end + 1} = c; u = false(code.n, 1); u(vs) = true; used{end + 1} = u; %#
    ok<AGROW>
        end
    end
end
code.layers = cell(numel(groups), 1);
for g = 1:numel(groups)
    cs = groups{g}; e = [];
    for c = cs.', e = [e; (first(c):first(c) + code.dc(c) - 1).']; end %#ok<AGROW>
    L.e = e; L.m = numel(cs); L.cn = repelem((1:L.m).', code.dc(cs));
    L.Sc = sparse(1:numel(e), L.cn, 1, numel(e), L.m);
    L.dmax = max(code.dc(cs));
    t = (1:numel(e)).' - [0; cumsum(code.dc(cs(1:end - 1)))](L.cn);
    L.epos = L.cn + (t - 1) * L.m;
    code.layers{g} = L;
end
end
```

layered_decode.m

```
function [chat, iters, app] = layered_decode(code, llr, max_iter, rule, alpha, beta)
%LAYERED_DECODE Row-layered decoder: the a-posteriori LLRs are updated after
% every layer, so new information is used at once. Call make_layers first.
if nargin < 4, rule = 'nms'; end
if nargin < 5, alpha = 0.75; end
if nargin < 6, beta = 0.5; end
B = size(llr, 1); app = llr; iters = max_iter * ones(B, 1);
act = (1:B).'; a = app; c2v = zeros(B, code.E);
for it = 1:max_iter
    for g = 1:numel(code.layers)
        L = code.layers{g}; e = L.e;
        v2c = a(:, code.vn(e)) - c2v(:, e);            % remove the old message
        nw = check_update(L, v2c, rule, alpha, beta);
        a(:, code.vn(e)) = v2c + nw;                   % update APP immediately
        c2v(:, e) = nw;
```

```
    end
    ok = ~any(ldpc_syndrome(code, a < 0), 2);
    app(act, :) = a;
    iters(act(ok)) = it;
    act = act(~ok); a = a(~ok, :); c2v = c2v(~ok, :);
    if isempty(act), break; end
end
chat = double(app < 0);
end
```

bit_flip.m

```
function [x, iters] = bit_flip(code, r, max_iter)
%BIT_FLIP Parallel bit flipping: flip every bit that sits in the largest number
% of failed checks. r: hard decisions (B x n).
x = double(r); iters = zeros(size(x, 1), 1);
for it = 1:max_iter
    s = ldpc_syndrome(code, x);
    active = any(s, 2);
    if ~any(active), break; end
    iters(active) = it;
    fails = s(:, code.cn) * code.Sv;                    % failed checks per bit
    worst = max(fails, [], 2);
    flip = (fails == worst) & (worst > 0) & active;
    x = mod(x + flip, 2);
end
end
```

peel.m

```
function [y, rounds] = peel(code, y)
%PEEL Peeling decoder for the BEC. y holds 0, 1 or -1 (erasure). A check with
% exactly one erased neighbour recovers it (all checks in parallel each round).
rounds = zeros(size(y, 1), 1);
for it = 1:code.n
    ev = y(:, code.vn);
    n_er = double(ev < 0) * code.Sc;                    % erasures per check
    par = mod(max(ev, 0) * code.Sc, 2);                 % parity of known bits
    solve = n_er == 1;
    if ~any(solve(:)), break; end
    es = solve(:, code.cn) & (ev < 0);                  % the single erased edge
    [b, e] = find(es); b = b(:); e = e(:);
    y(sub2ind(size(y), b, code.vn(e))) = par(sub2ind(size(par), b, code.cn(e)));
    rounds(unique(b)) = it;
end
end
```

C.7 Density evolution

de_bec.m

```
function hist = de_bec(lam, rho, eps, iters, tol)
%DE_BEC Density evolution on the BEC: x(l+1) = eps*lambda(1 - rho(1 - x(l))).
% lam, rho: [degree, edge fraction] rows (edge perspective).
if nargin < 4, iters = 2000; end
if nargin < 5, tol = 1e-12; end
x = eps; hist = eps;
for l = 1:iters
    x = eps * dpoly(lam, 1 - dpoly(rho, 1 - x));
    hist(end + 1) = x; %#ok<AGROW>
    if x < tol, break; end
end
end

function y = dpoly(d, x)
y = sum(d(:, 2) .* x .^ (d(:, 1) - 1));
end
```

threshold_bec.m

```
function t = threshold_bec(lam, rho, iters)
%THRESHOLD_BEC Largest erasure probability for which DE goes to zero (bisection).
if nargin < 3, iters = 5000; end
lo = 0; hi = 1;
for k = 1:40
    mid = (lo + hi) / 2;
    h = de_bec(lam, rho, mid, iters);
    if h(end) < 1e-10, lo = mid; else, hi = mid; end
end
t = lo;
end
```

gh_nodes.m

```
function [x, w] = gh_nodes(N)
%GH_NODES Gauss-Hermite nodes and weights for the weight exp(-x^2/2)
% (probabilists' Hermite polynomials), by the Golub-Welsch method.
J = diag(sqrt(1:N - 1), 1) + diag(sqrt(1:N - 1), -1);
[V, D] = eig(J);
[x, i] = sort(diag(D));
w = sqrt(2 * pi) * V(1, i).' .^ 2;
end
```

phi_ga.m

```
function y = phi_ga(m)
%PHI_GA phi(m) = 1 - E[tanh(u/2)], u ~ N(m, 2m) (Chung et al. 2001), by quadrature.
persistent x w
if isempty(x), [x, w] = gh_nodes(80); end
m = m(:);
```

```
u = m + sqrt(2 * m) .* x.';
y = 1 - (tanh(u / 2) * w) / sqrt(2 * pi);
y(m < 1e-10) = 1;
end
```

threshold_ga.m

```
function s = threshold_ga(lam, rho)
%THRESHOLD_GA Gaussian-approximation threshold (noise sigma) on BI-AWGN.
mt = logspace(-8, 3, 6000).';                          % table for the inverse
lp = log(max(phi_ga(mt), 1e-300));
[xl, ia] = unique(-lp);                                % increasing, no repeated values
lm = log(mt(ia));
inv = @(y) exp(interp1(xl, lm, -log(max(y, 1e-300)), 'linear', 'extrap'));
lo = 0.3; hi = 1.5;
for k = 1:30
    mid = (lo + hi) / 2;
    if de_ga(lam, rho, mid, inv), lo = mid; else, hi = mid; end
end
s = lo;
end

function ok = de_ga(lam, rho, sigma, inv)
m0 = 2 / sigma ^ 2; mu = 0; ok = false;
for it = 1:20000
    s = sum(lam(:, 2) .* phi_ga(m0 + (lam(:, 1) - 1) * mu));
    mu = 0;
    for r = 1:size(rho, 1)
        mu = mu + rho(r, 2) * inv(1 - (1 - s) ^ (rho(r, 1) - 1));
    end
    if mu > 60, ok = true; return; end
end
end
```

C.8 5G NR codes and simulation

nr_lifting_sets.m

```
function S = nr_lifting_sets()
%NR_LIFTING_SETS The eight sets of lifting sizes of TS 38.212 Table 5.3.2-1.
S = {[2 4 8 16 32 64 128 256], [3 6 12 24 48 96 192 384], [5 10 20 40 80 160 320], ...
     [7 14 28 56 112 224], [9 18 36 72 144 288], [11 22 44 88 176 352], ...
     [13 26 52 104 208], [15 30 60 120 240]};
end
```

nr_base_graph.m

```
function V = nr_base_graph(bg, ils)
%NR_BASE_GRAPH Base matrix of 5G NR BG1 (46 x 68) or BG2 (42 x 52) for the
% lifting-set index ils (0..7), read from the TS 38.212 tables (csv files).
if bg == 1, V = -ones(46, 68); else, V = -ones(42, 52); end
```

```
fid = fopen(fullfile(fileparts(mfilename('fullpath')), sprintf('5G_bg%d.csv', bg)));
fgetl(fid); fgetl(fid);                                % two header lines
row = 0;
while true
    line = fgetl(fid);
    if ~ischar(line), break; end
    f = strsplit(line, ';');
    if ~isempty(strtrim(f{1})), row = str2double(f{1}); end
    V(row + 1, str2double(f{2}) + 1) = str2double(f{3 + ils});
end
fclose(fid);
end
```

nr_select.m

```
function [bg, Z, ils] = nr_select(K, R, bg)
%NR_SELECT Base graph, lifting size and set index for K bits (data + CRC) at
% code rate R (TS 38.212, Sections 7.2.2 and 5.2.2).
if nargin < 3 || isempty(bg)
    if K <= 292 || (K <= 3824 && R <= 0.67) || R <= 0.25, bg = 2; else, bg = 1; end
end
if bg == 1
    kb = 22;
elseif K > 640, kb = 10; elseif K > 560, kb = 9; elseif K > 192, kb = 8; else, kb = 6;
end
S = nr_lifting_sets(); Z = inf; ils = -1;
for i = 1:8
    z = S{i}(find(kb * S{i} >= K, 1));
    if ~isempty(z) && z < Z, Z = z; ils = i - 1; end
end
end
```

nr_code.m

```
function nr = nr_code(K, E, bg)
%NR_CODE 5G NR LDPC code block: lifted base graph (only the rows needed for the
% E transmitted bits), encoder and rate matching (rv = 0, no interleaving).
if nargin < 3, bg = []; end
[nr.bg, Z, ils] = nr_select(K, K / E, bg);
nr.K = K; nr.E = E; nr.Z = Z; nr.ils = ils;
V = nr_base_graph(nr.bg, ils);
P = V; P(V >= 0) = mod(V(V >= 0), Z);                  % shift values P = V mod Z
if nr.bg == 1, nr.kb = 22; else, nr.kb = 10; end
nr.Kp = nr.kb * Z;                                     % encoder input incl. fillers
nr.filler = K + 1:nr.Kp;
nfull = size(V, 2) * Z;
tx = setdiff(2 * Z + 1:nfull, nr.filler);              % first 2Z bits punctured
nr.Ncb = numel(tx);
if E <= numel(tx), tx = tx(1:E); end
ncol = max(nr.kb + 4, ceil(max(tx) / Z));              % base columns in use
nr.code = qc_lift(P(1:ncol - nr.kb, 1:ncol), Z);
nr.code.name = sprintf('NR BG%d K=%d E=%d Z=%d', nr.bg, K, E, Z);
nr.n = ncol * Z;
nr.enc = dual_encoder(nr.code, nr.Kp, 4 * Z);
```

```
nr.tx_pos0 = tx;
nr.tx_pos = tx(mod(0:E - 1, numel(tx)) + 1);           % circular buffer
end
```

nr_encode.m

```
function [c, x] = nr_encode(nr, u)
%NR_ENCODE Encode messages u (B x K): full codewords c and the E sent bits x.
s = zeros(size(u, 1), nr.Kp);
s(:, 1:nr.K) = u;                                      % filler bits stay 0
c = dual_encode(nr.enc, s);
x = c(:, nr.tx_pos);
end
```

nr_llr_in.m

```
function L = nr_llr_in(nr, llr_tx)
%NR_LLR_IN Decoder input: LLRs of the sent bits at their positions (repeated bits
% add up), 0 for punctured / unsent bits, a large value for filler bits.
L = zeros(size(llr_tx, 1), nr.n);
N0 = numel(nr.tx_pos0);
for p = 1:N0:nr.E
    q = min(p + N0 - 1, nr.E);
    L(:, nr.tx_pos(p:q)) = L(:, nr.tx_pos(p:q)) + llr_tx(:, p:q);
end
L(:, nr.filler) = 1e3;
end
```

ldpc_simulate.m

```
function res = ldpc_simulate(encode, decode, k, R, ebn0_list, info, max_frames, min_errors,
     batch)
%LDPC_SIMULATE BER/FER versus Eb/N0. encode(u) -> [codeword, sent bits];
% decode(llr of sent bits) -> [decoded codeword, iterations]; info = positions of
% the message bits in the decoded codeword (1-based).
res.ebn0 = ebn0_list; res.ber = []; res.fer = []; res.frames = []; res.avg_iter = [];
for eb = ebn0_list
    be = 0; fe = 0; fr = 0; its = 0;
    while fr < max_frames && fe < min_errors
        u = randi([0 1], batch, k);
        [~, x] = encode(u);
        [ch, it] = decode(awgn_llr(x, eb, R));
        e = sum(ch(:, info) ~= u, 2);
        be = be + sum(e); fe = fe + sum(e > 0); fr = fr + batch; its = its + sum(it);
    end
    res.ber(end + 1) = be / (fr * k); res.fer(end + 1) = fe / fr;
    res.frames(end + 1) = fr; res.avg_iter(end + 1) = its / fr;
    fprintf('Eb/N0 = %4.2f dB  FER = %.2e  BER = %.2e  frames = %d  avg it = %.1f\n', ...
            eb, res.fer(end), res.ber(end), fr, res.avg_iter(end));
end
end
```

C.9 Worked examples and tests

demo_ldpc.m

```
% DEMO_LDPC  Worked examples of the book, computed with the MATLAB library.
H = [1 1 0 1 0 0; 0 1 1 0 1 0; 1 0 0 0 1 1; 0 0 1 1 0 1];
code = ldpc_from_H(H, 'toy');
fprintf('Toy code: rank %d, girth %d\n', gf2_rank(H), ldpc_girth(code));
[G, info] = generator_from_H(H);
fprintf('G = %s\n', mat2str(G)); fprintf('information positions (1-based): %s\n', mat2str(
    info));
fprintf('u = [1 0 1] -> c = %s\n', mat2str(encode_G(G, [1 0 1])));

% Chapter 7: peeling and bit flipping
y = [-1 0 -1 0 1 -1];
[x, rounds] = peel(code, y);
fprintf('peeling (?,0,?,0,1,?) -> %s in %d rounds\n', mat2str(x), rounds);
[x, it] = bit_flip(code, [0 0 1 1 1 1], 10);
fprintf('bit flipping 001111 -> %s in %d iteration(s)\n', mat2str(x), it);

% Chapter 8: sum-product example
L = 4 * [0.9 1.2 -0.8 -0.3 -1.1 -0.7];
c2v = check_update(code, L(code.vn), 'spa');
fprintf('SPA iteration-1 check messages: %s\n', mat2str(round(c2v * 1e4) / 1e4));
[x, it, app] = bp_decode(code, L, 20, 'spa');
fprintf('SPA: decoded %s after %d iteration(s); APP = %s\n', mat2str(x), it, mat2str(round(
    app * 1e4) / 1e4));

% Chapter 9: min-sum variants and layered decoding
for r = {'ms', 'nms', 'oms'}
    c2v = check_update(code, L(code.vn), r{1}, 0.75, 0.5);
    fprintf('%-3s check messages: %s\n', r{1}, mat2str(round(c2v * 1e4) / 1e4));
end
code = make_layers(code);
[x, it, app] = layered_decode(code, L, 20, 'spa');
fprintf('layered SPA: %s, %d iteration(s), APP = %s\n', mat2str(x), it, mat2str(round(app *
     1e4) / 1e4));

% Chapter 10: thresholds
fprintf('BEC thresholds: (3,6) %.4f  (3,4) %.4f  (4,8) %.4f\n', threshold_bec([3 1], [6 1])
    , ...
        threshold_bec([3 1], [4 1]), threshold_bec([4 1], [8 1]));
fprintf('GA threshold (3,6): sigma = %.4f\n', threshold_ga([3 1], [6 1]));

% Chapter 11: 5G NR parameters
for KE = [120 360; 256 512; 1024 2048; 1024 1152; 8448 25344].'
    nr = nr_code(KE(1), KE(2));
    fprintf('K=%5d E=%5d: BG%d Z=%3d iLS=%d fillers=%3d rows used=%2d\n', KE(1), KE(2), nr.
    bg, nr.Z, ...
            nr.ils, nr.Kp - nr.K, nr.code.m / nr.Z);
end
```

OUTPUT (real run)

```
Toy code: rank 3, girth 6
G = [0 1 1 1 0 0;1 1 0 0 1 0;1 1 1 0 0 1]
information positions (1-based): [4 5 6]
u = [1 0 1] -> c = [1 0 0 1 0 1]
peeling (?,0,?,0,1,?) -> [0 0 1 0 1 1] in 2 rounds
bit flipping 001111 -> [0 0 1 0 1 1] in 1 iteration(s)
SPA iteration-1 check messages: [-1.1755 -1.1214 3.3369 2.9372 -3.8871 -3.0164 2.6168 -2.4306 -3.2292
    1.0342 2.2895 1.0853]
SPA: decoded [0 0 1 0 1 1] after 1 iteration(s); APP = [5.0413 6.6159 -6.0528 4.4264 -9.847 -4.944]
ms  check messages: [-1.2 -1.2 3.6 3.2 -4.4 -3.2 2.8 -2.8 -3.6 1.2 2.8 1.2]
nms check messages: [-0.9 -0.9 2.7 2.4 -3.3 -2.4 2.1 -2.1 -2.7 0.9 2.1 0.9]
oms check messages: [-0.7 -0.7 3.1 2.7 -3.9 -2.7 2.3 -2.3 -3.1 0.7 2.3 0.7]
layered SPA: [0 0 1 0 1 1], 1 iteration(s), APP = [5.2113 6.6159 -8.5754 7.1043 -9.0258 -7.3397]
BEC thresholds: (3,6) 0.4294  (3,4) 0.6474  (4,8) 0.3834
GA threshold (3,6): sigma = 0.8719
K=  120 E=  360: BG2 Z= 20 iLS=2 fillers= 80 rows used=14
K=  256 E=  512: BG2 Z= 32 iLS=0 fillers= 64 rows used=10
K= 1024 E= 2048: BG2 Z=104 iLS=6 fillers= 16 rows used=12
K= 1024 E= 1152: BG1 Z= 48 iLS=1 fillers= 32 rows used= 5
K= 8448 E=25344: BG1 Z=384 iLS=1 fillers=  0 rows used=46
```

Every number agrees with the Python results of Chapters 3–11.

test_ldpc_matlab.m

```
% TEST_LDPC_MATLAB  Compare the MATLAB library with the Python reference results.
ref = load('pyref_ldpc.mat');
d = @(x) double(x);
npass = 0;
report = @(name, ok) fprintf('%-62s %s\n', name, pass_fail(ok));

% 1. GF(2) and generator matrix
H6 = [1 1 0 1 0 0; 0 1 1 0 1 0; 1 0 0 0 1 1; 0 0 1 1 0 1];
[G6, info6] = generator_from_H(H6);
ok = gf2_rank(H6) == 3 && isequal(G6, [0 1 1 1 0 0; 1 1 0 0 1 0; 1 1 1 0 0 1]) && isequal(
    info6, [4 5 6]);
report('GF(2): toy code rank, G and information positions', ok); npass = npass + ok;
H = d(ref.H_peg); code = ldpc_from_H(H, 'PEG');
[G, info] = generator_from_H(H);
ok = isequal(G, d(ref.G_peg)) && isequal(info - 1, d(ref.info_peg)) && ~any(any(mod(H * G
    .', 2)));
report('GF(2): generator matrix of PEG(1008,504) identical to Python', ok); npass = npass +
     ok;

% 2. structure
ok = ldpc_girth(code) == 8 && count_4cycles(code) == 0 && ldpc_girth(ldpc_from_H(H6)) == 6;
q = qc_lift([0 0 0 0; 0 1 1 3], 5);
ok = ok && ldpc_girth(q) == 4 && count_4cycles(q) == 5 && ldpc_girth(qc_lift([0 0 0 0; 0 1
    2 3], 5)) == 8;
report('Girth and 4-cycles (PEG 8, toy 6, QC 4 and 8)', ok); npass = npass + ok;

% 3. check-node update on random inputs
ok = true;
rules = {'spa', 'ms', 'nms', 'oms'};
```

```
for r = 1:4
    out = check_update(code, ref.v2c, rules{r}, 0.75, 0.5);
    ok = ok && max(abs(out(:) - ref.(['cu_' rules{r}])(:))) < 1e-9;
end
report('Check-node update (SPA, MS, NMS, OMS) equals Python', ok); npass = npass + ok;

% 4. flooding decoders on 200 noisy frames
L = ref.L_peg; c = d(ref.c_peg); ok = true; agree = 0;
for r = 1:4
    [x, it] = bp_decode(code, L, 50, rules{r});
    same = all(x == d(ref.(['x_' rules{r}])), 2) & it == reshape(d(ref.(['it_' rules{r}])),
     [], 1);
    agree = agree + sum(same);
end
report(sprintf('Flooding decoders: %d of 800 frames identical (decisions and iterations)',
    agree), agree == 800);
npass = npass + (agree == 800);

% 5. layered decoders
code = make_layers(code); agree = 0;
for r = {'nms', 'spa'}
    [x, it] = layered_decode(code, L, 50, r{1});
    agree = agree + sum(all(x == d(ref.(['xl_' r{1}])), 2) & it == reshape(d(ref.(['itl_' r
    {1}])), [], 1));
end
report(sprintf('Layered decoders (8 layers): %d of 400 frames identical', agree), agree ==
    400);
npass = npass + (agree == 400);

% 6. bit flipping and peeling
[x, it] = bit_flip(code, d(ref.r_bf), 50);
ok = isequal(x, d(ref.x_bf)) && isequal(it, reshape(d(ref.it_bf), [], 1));
[y, rd] = peel(code, d(ref.y_peel));
ok = ok && isequal(y, d(ref.x_peel)) && isequal(rd, reshape(d(ref.rd_peel), [], 1));
report('Bit flipping (50 frames) and peeling (50 frames) identical', ok); npass = npass +
    ok;

% 7. 5G NR codes: parameters and codewords
ok = true;
for i = 0:7
    KE = d(ref.(sprintf('nr%d_KE', i)));
    nr = nr_code(KE(1), KE(2));
    [cc, xx] = nr_encode(nr, d(ref.(sprintf('nr%d_u', i))));
    par = [nr.bg nr.Z nr.ils nr.code.m nr.code.n nr.code.E];
    ok = ok && isequal(par, d(ref.(sprintf('nr%d_par', i)))) && isequal(cc, d(ref.(sprintf(
    'nr%d_c', i)))) ...
            && isequal(xx, d(ref.(sprintf('nr%d_x', i)))) && ~any(any(ldpc_syndrome(nr.code
    , cc)));
end
report('5G NR: parameters and codewords identical for 8 (K,E) pairs', ok); npass = npass +
    ok;

% 8. 5G NR layered decoding
nr = nr_code(1024, 2048); nr.code = make_layers(nr.code, nr.Z);
```

```
[xh, it] = layered_decode(nr.code, nr_llr_in(nr, ref.nr_L), 50, 'nms');
agree = sum(all(xh == d(ref.nr_xh), 2) & it == reshape(d(ref.nr_it), [], 1));
report(sprintf('5G NR layered NMS (K=1024, R=1/2): %d of 100 frames identical', agree),
    agree == 100);
npass = npass + (agree == 100);

% 9. density evolution
t1 = threshold_bec([3 1], [6 1]); t2 = threshold_bec([3 1], [4 1]); t3 = threshold_ga([3
    1], [6 1]);
ok = abs(t1 - ref.th_bec36) < 1e-9 && abs(t2 - ref.th_bec34) < 1e-9 && abs(t3 - ref.th_ga36
    ) < 2e-4;
report(sprintf('Thresholds: BEC %.4f %.4f, GA sigma %.4f', t1, t2, t3), ok); npass = npass
    + ok;
% 11. constructions that use MATLAB's own random stream
rand('seed', 3); randn('seed', 3); %#ok<RAND>
pc = peg_code(96, 48, 3); Hg = gallager_H(60, 3, 6);
ok = ldpc_girth(pc) >= 6 && all(pc.dv == 3) && gf2_rank(Hg) <= 30 - 2 && all(sum(Hg, 1) ==
    3) && all(sum(Hg, 2) == 6);
report(sprintf('PEG(96,48): girth %d; Gallager (3,6), n=60: rank %d', ldpc_girth(pc),
    gf2_rank(Hg)), ok);
npass = npass + ok;
fprintf('\n%d of 11 tests passed\n', npass);
```

pass_fail.m

```
function s = pass_fail(ok)
%PASS_FAIL 'PASS' or 'FAIL'.
if ok, s = 'PASS'; else, s = 'FAIL'; end
end
```

OUTPUT (real run)

```
GF(2): toy code rank, G and information positions              PASS
GF(2): generator matrix of PEG(1008,504) identical to Python   PASS
Girth and 4-cycles (PEG 8, toy 6, QC 4 and 8)                  PASS
Check-node update (SPA, MS, NMS, OMS) equals Python            PASS
Flooding decoders: 800 of 800 frames identical (decisions and iterations) PASS
Layered decoders (8 layers): 400 of 400 frames identical       PASS
Bit flipping (50 frames) and peeling (50 frames) identical     PASS
5G NR: parameters and codewords identical for 8 (K,E) pairs    PASS
5G NR layered NMS (K=1024, R=1/2): 100 of 100 frames identical PASS
Thresholds: BEC 0.4294 0.6474, GA sigma 0.8719                 PASS
PEG(96,48): girth 6; Gallager (3,6), n=60: rank 28             PASS

11 of 11 tests passed
```

`pyref_ldpc.mat` holds the PEG$(1008, 504)$ matrix, 200 noisy frames at $E_b/N_0 = 2$ dB with the Python decisions and iteration counts of six decoders, bit-flipping and peeling cases, eight 5G NR codewords and 100 decoded 5G frames. The MATLAB functions reproduce all of them exactly: same decisions *and* same numbers of iterations.

C.10 Simulation in MATLAB

sim_ldpc_matlab.m

```
% SIM_LDPC_MATLAB  FER of the 5G NR code K = 1024, R = 1/2 (BG2, Z = 104) with
% layered normalised min-sum (alpha = 0.75, 50 iterations) in MATLAB/Octave.
rng_seed = 2026; randn('seed', rng_seed); rand('seed', rng_seed); %#ok<RAND>
nr = nr_code(1024, 2048);
nr.code = make_layers(nr.code, nr.Z);
enc = @(u) nr_encode(nr, u);
dec = @(L) layered_decode(nr.code, nr_llr_in(nr, L), 50, 'nms');
res = ldpc_simulate(enc, dec, 1024, 0.5, 0.5:0.25:2.0, 1:1024, 4000, 50, 200);
save('-v7', 'sim_matlab.mat', 'res');
```

OUTPUT (real run)

```
Eb/N0 = 0.50 dB  FER = 9.05e-01  BER = 1.48e-01  frames = 200  avg it = 47.4
Eb/N0 = 0.75 dB  FER = 6.90e-01  BER = 1.02e-01  frames = 200  avg it = 42.1
Eb/N0 = 1.00 dB  FER = 2.50e-01  BER = 3.71e-02  frames = 200  avg it = 24.9
Eb/N0 = 1.25 dB  FER = 5.10e-02  BER = 6.31e-03  frames = 1000  avg it = 14.7
Eb/N0 = 1.50 dB  FER = 3.00e-03  BER = 3.78e-04  frames = 4000  avg it = 9.7
Eb/N0 = 1.75 dB  FER = 0.00e+00  BER = 0.00e+00  frames = 4000  avg it = 7.5
Eb/N0 = 2.00 dB  FER = 0.00e+00  BER = 0.00e+00  frames = 4000  avg it = 6.3
```

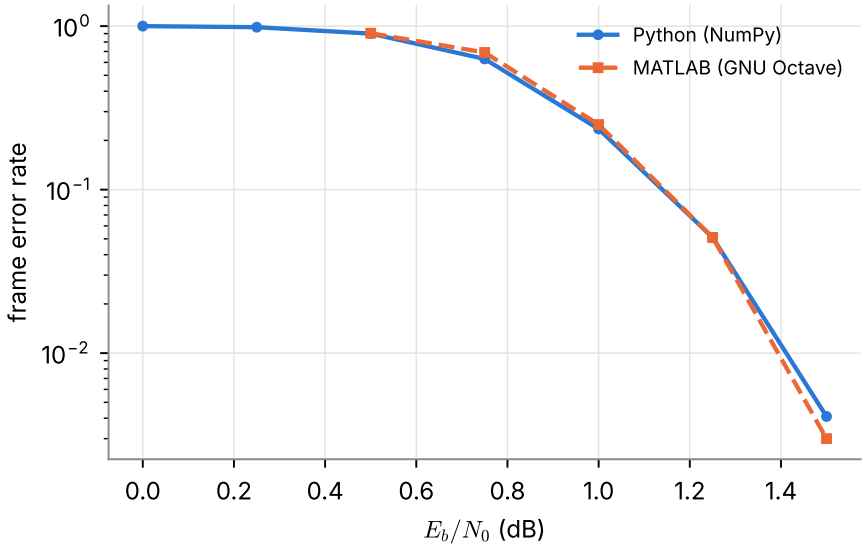


Figure 16.3. 5G NR code $K = 1024$, $R = 1/2$, layered NMS: MATLAB (Octave) and Python simulations with independent random streams.

C.11 Final project in MATLAB

crc_matrix.m

```
function P = crc_matrix(poly, k)
%CRC_MATRIX crc(m) = mod(m*P, 2) for a k-bit message. poly lists the exponents
```

```
% of the generator, e.g. [16 12 5 0] for CRC16 of TS 38.212.
r = poly(1);
g = zeros(1, r + 1); g(r - poly + 1) = 1;            % g(1) = coefficient of x^r
P = zeros(k, r);
cur = g(2:end);                                      % x^r mod g
for i = k:-1:1
    P(i, :) = cur;
    top = cur(1);
    cur = [cur(2:end), 0];                           % multiply by x
    if top, cur = mod(cur + g(2:end), 2); end
end
end
```

final_project_lab.m

```
% FINAL_PROJECT_LAB  MATLAB version of the final project (Chapter 16): one 5G NR
% transport block with TB CRC, segmentation, LDPC coding, two redundancy
% versions (HARQ-IR) and layered decoding.
A = 1000; R = 1/2; ebn0 = 0.75:0.25:1.75; frames = 600; batch = 100;
L_tb = 16 + 8 * (A > 3824);                          % CRC16 or CRC24A
if A > 3824, poly = [24 23 18 17 14 11 10 7 6 5 4 3 1 0]; else, poly = [16 12 5 0]; end
P = crc_matrix(poly, A);
B = A + L_tb;
if A <= 292 || (A <= 3824 && R <= 0.67) || R <= 0.25, bg = 2; else, bg = 1; end
nr = nr_code(B, 1e6, bg);                            % mother code (single code block here)
nr.code = make_layers(nr.code, nr.Z);
Z = nr.Z;
if bg == 1, N = 66 * Z; f = [0 17 33 56]; else, N = 50 * Z; f = [0 13 25 43]; end
buf = 2 * Z + 1:2 * Z + N; buf = buf(~ismember(buf, nr.filler));
G = round(A / R);
pos = @(rv) buf(mod(find(buf >= 2 * Z + 1 + f(rv + 1) * Z, 1) - 1 + (0:G - 1), numel(buf))
    + 1);
fprintf('A = %d, R = %.2f: TB CRC %d bits, BG%d, Z = %d, fillers %d, k0 = %s\n', A, R, L_tb
    , bg, Z, ...
        nr.Kp - B, mat2str(f * Z));
randn('seed', 7); rand('seed', 7); %#ok<RAND>
for eb = ebn0
    e1 = 0; e2 = 0; n = 0;
    while n < frames
        a = randi([0 1], batch, A);
        b = [a, mod(a * P, 2)];                      % attach the TB CRC
        c = nr_encode(nr, b);
        soft = zeros(batch, nr.n);
        for t = 1:2                                  % rv0, then rv2
            p = pos(2 * (t - 1));
            soft(:, p) = soft(:, p) + awgn_llr(c(:, p), eb, R);
            Ld = soft; Ld(:, nr.filler) = 1e3;
            x = layered_decode(nr.code, Ld, 25, 'nms');
            bh = x(:, 1:B);
            ok = all(mod(bh(:, 1:A) * P, 2) == bh(:, A + 1:B), 2);
            if t == 1, e1 = e1 + sum(~ok); else, e2 = e2 + sum(~ok); end
        end
        n = n + batch;
    end
```

```
38      fprintf('Eb/N0 = %4.2f dB: TB error after rv0 %.4f, after rv0+rv2 %.4f\n', eb, e1 / n,
        e2 / n);
39  end
```

OUTPUT (real run)

```
A = 1000, R = 0.50: TB CRC 16 bits, BG2, Z = 104, fillers 24, k0 = [0 1352 2600 4472]
Eb/N0 = 0.75 dB: TB error after rv0 0.8433, after rv0+rv2 0.0000
Eb/N0 = 1.00 dB: TB error after rv0 0.4800, after rv0+rv2 0.0000
Eb/N0 = 1.25 dB: TB error after rv0 0.1550, after rv0+rv2 0.0000
Eb/N0 = 1.50 dB: TB error after rv0 0.0233, after rv0+rv2 0.0000
Eb/N0 = 1.75 dB: TB error after rv0 0.0017, after rv0+rv2 0.0000
```

Appendix D — Glossary

A-posteriori LLR	L_v^{app}: channel LLR plus all incoming check messages; its sign is the decision.
ALT form	Approximate lower-triangular form of $\mathbf{H}$ used by the Richardson–Urbanke encoder.
Base graph / base matrix	Small matrix of shifts (-1 = zero block) that is lifted to a QC code; 5G NR has BG1 and BG2.
BEC, BSC, BI-AWGN	Binary erasure, binary symmetric and binary-input AWGN channels.
Bit flipping	Hard-decision decoder that flips bits involved in many unsatisfied checks.
Box-plus ($\boxplus$)	LLR of the XOR of independent bits: $2\tanh^{-1}\prod\tanh(L_i/2)$.
Check node	Node of the Tanner graph for one parity check (one row of $\mathbf{H}$).
Circulant permutation matrix	Identity matrix cyclically shifted by s; written $\mathbf{P}^s$.
Circular buffer	5G rate-matching memory from which E bits are read starting at k_0.
Compressed message	Min-sum check storage: m_1, m_2, index of m_1 and the signs.
Computation tree	The neighbourhood of a node unrolled over iterations.
Concentration	Property that almost all codes of an ensemble behave like the average for large n.
Degree distribution	$\lambda(x), \rho(x)$ (edge perspective) or $L(x), R(x)$ (node perspective).
Density evolution (DE)	Tracking of the message distributions of an ensemble; gives the threshold.
Design rate	$1 - m/n = 1 - \int\rho/\int\lambda$; the true rate can be higher if $\mathbf{H}$ is rank deficient.
Dual-diagonal	Staircase parity structure that allows linear-time encoding.
Early termination	Stopping the decoder when the syndrome (or a CRC) is satisfied.
Edge perspective	Fractions counted over edges: λ_i, ρ_i.
Error floor	Flattening of the error-rate curve at high SNR caused by small harmful sub-graphs.
EXIT chart	Mutual-information transfer curves of variable and check nodes; decoding needs an open tunnel.
Extrinsic information	Information sent on an edge that does not include what was received on that edge.
Filler bits	Known zero bits that fill a 5G code block up to $22Z$ or $10Z$; not transmitted.
Flooding schedule	All check nodes, then all variable nodes, in each iteration.
Gaussian approximation (GA)	DE in which every message density is a consistent Gaussian described by its mean.
Girth	Length of the shortest cycle of the Tanner graph.
HARQ-IR	Hybrid ARQ with incremental redundancy: retransmissions carry new parity bits.
Irregular code	Code whose nodes have different degrees.
Layered decoding	Schedule that updates the a-posteriori LLRs after each group of checks.

Lifting size Z	Size of the circulant blocks of a QC code; 51 values in 5G NR.
LDPC code	Linear code with a sparse parity-check matrix.
Min-sum (MS)	Check rule sign $\cdot$ min; normalised (NMS, $\times\alpha$) and offset (OMS, $-\beta$) versions.
Peeling decoder	BEC decoder that solves checks with exactly one erased bit.
PEG	Progressive edge growth: greedy construction maximising local girth.
Protograph	Small base graph (possibly with parallel edges) copied and permuted to build a code.
Puncturing	Not transmitting some code bits; in 5G the first $2Z$ systematic bits.
QC-LDPC code	Code whose $\mathbf{H}$ consists of circulant blocks.
Redundancy version (RV)	One of four starting points in the 5G circular buffer.
Regular code	Every variable node has degree d_v and every check node degree d_c.
Stability condition	$\epsilon\lambda_2\rho'(1) < 1$ (BEC): limit on degree-2 nodes.
Stopping set	Variable set such that every neighbouring check sees it at least twice; peeling fails on it.
Sum-product algorithm (SPA)	Belief propagation with the exact tanh / ϕ check rule.
Syndrome	$\mathbf{H}\hat{\mathbf{c}}^T$; zero iff $\hat{\mathbf{c}}$ is a codeword.
Tanner graph	Bipartite graph of variable and check nodes defined by $\mathbf{H}$.
Threshold	Worst channel for which DE converges to zero error probability.
Trapping set	Small variable set with few odd-degree neighbouring checks; causes error floors.
Variable node	Node of the Tanner graph for one code bit (one column of $\mathbf{H}$).
Waterfall	Region where the error rate falls steeply with SNR.

Appendix E — Bibliography

Textbooks

[1] T. Richardson and R. Urbanke, *Modern Coding Theory*. Cambridge University Press, 2008.

[2] W. E. Ryan and S. Lin, *Channel Codes: Classical and Modern*. Cambridge University Press, 2009.

[3] S. J. Johnson, *Iterative Error Correction: Turbo, Low-Density Parity-Check and Repeat-Accumulate Codes*. Cambridge University Press, 2010.

[4] D. J. C. MacKay, *Information Theory, Inference, and Learning Algorithms*. Cambridge University Press, 2003.

[5] S. Lin and D. J. Costello, *Error Control Coding*, 2nd ed. Pearson, 2004.

[6] T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed. Wiley, 2006.

Foundational papers

[7] R. G. Gallager, "Low-density parity-check codes," *IRE Trans. Inf. Theory*, vol. 8, no. 1, pp. 21–28, 1962; and *Low-Density Parity-Check Codes*, MIT Press, 1963.

[8] R. M. Tanner, "A recursive approach to low complexity codes," *IEEE Trans. Inf. Theory*, vol. 27, no. 5, pp. 533–547, 1981.

[9] D. J. C. MacKay and R. M. Neal, "Near Shannon limit performance of low density parity check codes," *Electronics Letters*, vol. 32, no. 18, pp. 1645–1646, 1996.

[10] D. J. C. MacKay, "Good error-correcting codes based on very sparse matrices," *IEEE Trans. Inf. Theory*, vol. 45, no. 2, pp. 399–431, 1999.

[11] C. Berrou, A. Glavieux and P. Thitimajshima, "Near Shannon limit error-correcting coding and decoding: Turbo-codes," in *Proc. IEEE ICC*, 1993, pp. 1064–1070.

[12] F. R. Kschischang, B. J. Frey and H.-A. Loeliger, "Factor graphs and the sum-product algorithm," *IEEE Trans. Inf. Theory*, vol. 47, no. 2, pp. 498–519, 2001.

Analysis and design

[13] T. J. Richardson and R. L. Urbanke, "The capacity of low-density parity-check codes under message-passing decoding," *IEEE Trans. Inf. Theory*, vol. 47, no. 2, pp. 599–618, 2001.

[14] T. J. Richardson, M. A. Shokrollahi and R. L. Urbanke, "Design of capacity-approaching irregular low-density parity-check codes," *IEEE Trans. Inf. Theory*, vol. 47, no. 2, pp. 619–637, 2001.

[15] T. J. Richardson and R. L. Urbanke, "Efficient encoding of low-density parity-check codes," *IEEE Trans. Inf. Theory*, vol. 47, no. 2, pp. 638–656, 2001.

[16] S.-Y. Chung, T. J. Richardson and R. L. Urbanke, "Analysis of sum-product decoding of low-density parity-check codes using a Gaussian approximation," *IEEE Trans. Inf. Theory*, vol. 47, no. 2, pp. 657–670, 2001.

[17] S.-Y. Chung, G. D. Forney, T. J. Richardson and R. Urbanke, "On the design of low-density parity-check codes within 0.0045 dB of the Shannon limit," *IEEE Communications Letters*, vol. 5, no. 2, pp. 58–60, 2001.

[18] M. G. Luby, M. Mitzenmacher, M. A. Shokrollahi and D. A. Spielman, "Efficient erasure correcting codes," *IEEE Trans. Inf. Theory*, vol. 47, no. 2, pp. 569–584, 2001.

[19] S. ten Brink, G. Kramer and A. Ashikhmin, "Design of low-density parity-check codes for modulation and detection," *IEEE Trans. Commun.*, vol. 52, no. 4, pp. 670–678, 2004.

[20] C. Di, D. Proietti, I. E. Telatar, T. J. Richardson and R. L. Urbanke, "Finite-length analysis of low-density parity-check codes on the binary erasure channel," *IEEE Trans. Inf. Theory*, vol. 48, no. 6, pp. 1570–1579, 2002.

[21] T. Richardson, "Error floors of LDPC codes," in *Proc. 41st Allerton Conf.*, 2003.

[22] T. Etzion, A. Trachtenberg and A. Vardy, "Which codes have cycle-free Tanner graphs?" *IEEE Trans. Inf. Theory*, vol. 45, no. 6, pp. 2173–2181, 1999.

Construction and decoding algorithms

[23] X.-Y. Hu, E. Eleftheriou and D.-M. Arnold, "Regular and irregular progressive edge-growth Tanner graphs," *IEEE Trans. Inf. Theory*, vol. 51, no. 1, pp. 386–398, 2005.

[24] M. P. C. Fossorier, "Quasi-cyclic low-density parity-check codes from circulant permutation matrices," *IEEE Trans. Inf. Theory*, vol. 50, no. 8, pp. 1788–1793, 2004.

[25] J. Thorpe, "Low-density parity-check (LDPC) codes constructed from protographs," JPL IPN Progress Report 42-154, 2003.

[26] M. P. C. Fossorier, M. Mihaljević and H. Imai, "Reduced complexity iterative decoding of low-density parity check codes based on belief propagation," *IEEE Trans. Commun.*, vol. 47, no. 5, pp. 673–680, 1999.

[27] J. Chen and M. P. C. Fossorier, "Near optimum universal belief propagation based decoding of low-density parity check codes," *IEEE Trans. Commun.*, vol. 50, no. 3, pp. 406–414, 2002.

[28] M. M. Mansour and N. R. Shanbhag, "High-throughput LDPC decoders," *IEEE Trans. VLSI Systems*, vol. 11, no. 6, pp. 976–996, 2003.

Standards and 5G NR

[29] 3GPP TS 38.212, *NR; Multiplexing and channel coding*, Release 15 and later.

[30] T. Richardson and S. Kudekar, "Design of low-density parity check codes for 5G new radio," *IEEE Communications Magazine*, vol. 56, no. 3, pp. 28–34, 2018.

[31] J. H. Bae, A. Abotabl, H.-P. Lin, K.-B. Song and J. Lee, "An overview of channel coding for 5G NR cellular communications," *APSIPA Trans. Signal and Information Processing*, vol. 8, e17, 2019.

[32] ETSI EN 302 307, *Digital Video Broadcasting (DVB); Second generation framing structure, channel coding and modulation systems (DVB-S2).*

[33] IEEE Std 802.11n-2009, amendment for higher throughput (LDPC codes of length 648, 1296, 1944).

[34] J. Hoydis et al., "Sionna: An open-source library for next-generation physical layer research," arXiv:2203.11854, 2022 (source of the machine-readable base-graph tables used in this book).

[35] M. Abdelaziz, *Polar Codes: From Zero to Expert* (Part One of this series), 2026.